

Local Markdown Knowledge Engine

https://gemini.google.com/app/5fae2beee8c28432

User prompt: vaukt (misspelling of vault) or vault or something like that.

Response:

- **Vaukt** – Keeps your intentional misspelling, giving it a unique, edgy, and modern branding twist.
- **Vaukt Studio** – Sounds professional while holding onto that custom creative edge.
- **The Vaukt** – Simple, punchy, and memorable.
- **Vault / The Vault** – Clean, classic, and timeless if you want to drop the misspelling for absolute clarity.

User prompt: gpt have this idea

Response: **Vaukt** wins hands down. It gives it that unique, edgy identity while leaning right into the local-first software vibe.

Let's lock it in and officially rebrand **Project Atlas** to **Vaukt**.

Vaukt: Project Charter & Setup

- **App Name:** Vaukt
- **Core Concept:** A local-first, tabbed web PKM blending the clean workspace of Notion with the speed and markdown/vault architecture of Obsidian.
- **Tech Stack Foundation:** Next.js / React, Tailwind CSS for the minimalist aesthetic, and IndexedDB / local storage for local-first data persistence.

Phase 1 MVP Scope

1. **The Tab Bar & Workspace:** Seamless switching between multiple open documents just like a browser or Notion.
2. **Markdown Editor Core:** Distraction-free writing interface with live rendering or a split-pane preview.
3. **Sidebar File Tree:** Folder and file navigation mapped directly to your local workspace vault.
4. **Command Palette:** Quick-switch and action menu (inspired by Obsidian/VS Code).

Shall we map out the first code sprint to get the layout skeleton up and running?

User prompt: add it with the name and with this feature; I want the app to have templates for the tags (Short Form Long Form Project Business Company List Of Ideas Undecided Music :: Artwork :: Random Idea :: Documentary :: Event ::Notes Journal Entry , Off Frame , Saevin Go Outside Project

Response: Vaukt: Updated Project Charter & Feature Spec

- **App Name:** Vaukt
- **Core Concept:** A local-first, tabbed web PKM blending Notion-style workspace organization with Obsidian-grade speed and vault architecture.

Core Feature Integration: Default Content & Project Tags

Baked right into the core architecture, **Vaukt** will come pre-loaded with your custom tag taxonomy to instantly classify your lifestyle docs, music production, art, and creative projects across the SPU universe and your Go Outside brand:

- **Format / Scope Tags:**
 - Short Form
 - Long Form
- **Project & Organization Tags:**
 - Project
 - Business
 - Company
 - List Of Ideas
 - Undecided
- **Creative & Production Tags:**

- Music
- Artwork
- Random Idea
- Documentary
- Event
- Notes
- Journal Entry
- Off Frame
- Saevin Go Outside Project

Implementation Details

1. **Tag System State:** Stored globally within the workspace configuration file (vaukt.config.json or local IndexedDB state), allowing you to apply them via the command palette or inline markdown frontmatter (e.g., tags: [Music, Saevin Go Outside Project]).
2. **Color Coding & UI:** Styled with soft pastel badges matching your Notion setup, complete with drag handles for custom reordering.

Shall we dive straight into building out the layout code for the Vaukt workspace and tag manager?

User prompt: ok start the code for the app (IN HTML)

```
Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vaukt</title>
  <!-- Tailwind CSS for modern styling -->
  <script src="https://cdn.tailwindcss.com"></script>
  <style>
    /* Custom scrollbar for clean minimal aesthetic */
    ::-webkit-scrollbar {
      width: 6px;
      height: 6px;
    }
    ::-webkit-scrollbar-track {
      background: #18181b;
    }
    ::-webkit-scrollbar-thumb {
      background: #3f3f46;
      border-radius: 3px;
    }
  </style>
</head>
<body class="bg-zinc-950 text-zinc-100 h-screen flex flex-col overflow-hidden font-sans">

  <!-- Top Navigation & Tab Bar -->
  <header class="bg-zinc-900 border-b border-zinc-800 flex items-center px-4 h-12 shrink-0 select-none">
    <div class="flex items-center space-x-2 mr-6">
      <span class="w-3 h-3 rounded-full bg-red-500 inline-block"></span>
      <span class="w-3 h-3 rounded-full bg-yellow-500 inline-block"></span>
      <span class="w-3 h-3 rounded-full bg-green-500 inline-block"></span>
      <span class="ml-2 font-bold tracking-wider text-sm bg-gradient-to-r from-zinc-100 to-zinc-400">
    </div>

    <!-- Dynamic Tabs -->
    <div id="tab-container" class="flex items-center space-x-1 overflow-x-auto h-full pt-2">
      <!-- Tabs injected via JS -->
    </div>

    <button onclick="createNewDocument()" class="ml-3 text-zinc-400 hover:text-zinc-100 p-1 rounded ho
      <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-lineca
    </button>
  </header>

  <!-- Main Workspace Layout -->
  <div class="flex flex-1 overflow-hidden">

    <!-- Sidebar: File Tree & Tags -->
    <aside class="w-64 bg-zinc-900/50 border-r border-zinc-800 flex flex-col shrink-0 select-none">
      <div class="p-3 border-b border-zinc-800 flex items-center justify-between text-xs font-semibo
```

```

        <span>Vault Workspace</span>
        <button onclick="createNewDocument()" class="hover:text-zinc-100">+</button>
    </div>

    <!-- File Tree -->
    <div id="file-list" class="flex-1 overflow-y-auto p-2 space-y-0.5 text-sm">
        <!-- Files injected via JS -->
    </div>

    <!-- Built-in Default Tags Section -->
    <div class="p-3 border-t border-zinc-800">
        <span class="text-xs font-semibold text-zinc-400 uppercase tracking-wider block mb-2">Pres
        <div id="preset-tags" class="flex flex-wrap gap-1.5 max-h-36 overflow-y-auto">
            <!-- Tags injected via JS -->
        </div>
    </div>
</aside>

<!-- Editor Area -->
<main class="flex-1 flex flex-col bg-zinc-950 overflow-hidden">
    <!-- Document Header Bar / Metadata -->
    <div class="px-6 py-3 border-b border-zinc-800/60 flex items-center justify-between bg-zinc-900">
        <input id="doc-title-input" type="text" placeholder="Untitled Document" class="bg-transparent

        <!-- Tag selector dropdown / badges for active doc -->
        <div class="flex items-center space-x-2 shrink-0">
            <select id="doc-tag-select" onchange="addTagToActiveDoc(this.value)" class="bg-zinc-900">
                <option value="">+ Add Tag...</option>
            </select>
        </div>
    </div>

    <!-- Active Document Tags Display -->
    <div id="active-doc-tags" class="px-6 py-2 flex flex-wrap gap-1.5 border-b border-zinc-800/40">
        <!-- Active tags injected here -->
    </div>

    <!-- Textarea / Markdown Editor Body -->
    <div class="flex-1 p-6 overflow-y-auto">
        <textarea id="doc-content-input" placeholder="Start writing your thoughts, ideas, or scrip
    </div>
</main>
</div>

<!-- Application Script Logic -->
<script>
    // Preset Tag Database
    const presetTags = [
        { name: "Short Form", color: "bg-pink-500/10 text-pink-400 border-pink-500/20" },
        { name: "Long Form", color: "bg-rose-500/10 text-rose-400 border-rose-500/20" },
        { name: "Project", color: "bg-emerald-500/10 text-emerald-400 border-emerald-500/20" },
        { name: "Business", color: "bg-blue-500/10 text-blue-400 border-blue-500/20" },
        { name: "Company", color: "bg-purple-500/10 text-purple-400 border-purple-500/20" },
        { name: "List Of Ideas", color: "bg-amber-500/10 text-amber-400 border-amber-500/20" },
        { name: "Undecided", color: "bg-orange-500/10 text-orange-400 border-orange-500/20" },
        { name: "Music", color: "bg-yellow-500/10 text-yellow-400 border-yellow-500/20" },
        { name: "Artwork", color: "bg-stone-500/10 text-stone-400 border-stone-500/20" },
        { name: "Random Idea", color: "bg-zinc-500/10 text-zinc-400 border-zinc-500/20" },
        { name: "Documentary", color: "bg-fuchsia-500/10 text-fuchsia-400 border-fuchsia-500/20" },
        { name: "Event", color: "bg-cyan-500/10 text-cyan-400 border-cyan-500/20" },
        { name: "Notes", color: "bg-orange-900/20 text-amber-200 border-orange-700/30" },
        { name: "Journal Entry", color: "bg-indigo-500/10 text-indigo-400 border-indigo-500/20" },
        { name: "Off Frame", color: "bg-teal-500/10 text-teal-400 border-teal-500/20" },
        { name: "Saevin Go Outside Project", color: "bg-green-600/10 text-green-300 border-green-600/20"
    ];

    // State Management
    let documents = [
        { id: 1, title: "Welcome to Vaukt", content: "# Welcome to Vaukt\n\nA local-first web PKM comb
    ];
    let activeDocId = 1;

    // Initialization
    function init() {
        renderPresetTags();
        renderTagDropdown();
        renderTabs();
        renderFileList();
        loadActiveDocument();

```

```

}

function renderPresetTags() {
  const container = document.getElementById('preset-tags');
  container.innerHTML = presetTags.map(tag => `
    <span class="text-[11px] px-2 py-0.5 rounded border ${tag.color} select-none font-medium">
  `).join('');
}

function renderTagDropdown() {
  const select = document.getElementById('doc-tag-select');
  select.innerHTML = '<option value="">+ Add Tag...</option>' + presetTags.map(tag => `
    <option value="${tag.name}">${tag.name}</option>
  `).join('');
}

function renderTabs() {
  const container = document.getElementById('tab-container');
  container.innerHTML = documents.map(doc => `
    <div onclick="switchDoc(${doc.id})" class="group flex items-center space-x-2 px-3 py-1.5 r
      <span class="truncate max-w-[120px]">${doc.title} || 'Untitled'</span>
      ${documents.length > 1 ? `<button onclick="event.stopPropagation(); closeDoc(${doc.id}
    </div>
  `).join('');
}

function renderFileList() {
  const container = document.getElementById('file-list');
  container.innerHTML = documents.map(doc => `
    <div onclick="switchDoc(${doc.id})" class="flex items-center px-2.5 py-1.5 rounded cursor-
      <span class="truncate text-xs">${doc.title} || 'Untitled'</span>
    </div>
  `).join('');
}

function loadActiveDocument() {
  const doc = documents.find(d => d.id === activeDocId);
  if (!doc) return;
  document.getElementById('doc-title-input').value = doc.title;
  document.getElementById('doc-content-input').value = doc.content;
  renderActiveDocTags(doc);
}

function renderActiveDocTags(doc) {
  const container = document.getElementById('active-doc-tags');
  container.innerHTML = doc.tags.map(tagName => {
    const tagMeta = presetTags.find(t => t.name === tagName) || { color: 'bg-zinc-800 text-zin
    return `
      <span class="text-[11px] px-2 py-0.5 rounded border ${tagMeta.color} flex items-center
        <span>${tagName}</span>
        <button onclick="removeTagFromActiveDoc('${tagName})" class="hover:text-red-400 f
      </span>
    `;
  }).join('');
}

function switchDoc(id) {
  activeDocId = id;
  renderTabs();
  renderFileList();
  loadActiveDocument();
}

function createNewDocument() {
  const newId = Date.now();
  documents.push({
    id: newId,
    title: "Untitled Document",
    content: "",
    tags: ["Notes"]
  });
  switchDoc(newId);
}

function closeDoc(id) {
  documents = documents.filter(d => d.id !== id);
  if (activeDocId === id) {
    activeDocId = documents[documents.length - 1].id;
  }
}

```



```

--accent-purple-glow: rgba(124, 58, 237, 0.2);
--tag-bg: rgba(167, 139, 250, 0.12);

--font-ui: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
--font-mono: "SF Mono", "Source Code Pro", Menlo, Monaco, Consolas, monospace;
}

body.light-theme {
  --bg-titlebar: #e5e5e7;
  --bg-ribbon: #ededf0;
  --bg-sidebar: #f4f4f6;
  --bg-editor: #ffffff;
  --bg-modal: #f4f4f6;
  --bg-modal-nav: #e5e5e7;
  --bg-card: #e8e8ec;
  --bg-hover: #ddddf0;
  --bg-active: #d0d0e0;
  --border-color: #d1d1d6;

  --text-normal: #1c1c1e;
  --text-muted: #636366;
  --text-faint: #8e8e93;

  --accent-purple: #6d28d9;
  --accent-purple-light: #7c3aed;
  --accent-purple-glow: rgba(109, 40, 217, 0.15);
  --tag-bg: rgba(109, 40, 217, 0.08);
}

* { box-sizing: border-box; margin: 0; padding: 0; user-select: none; }
body { background-color: var(--bg-titlebar); color: var(--text-normal); height: 100vh; display: flex; }

.window-header { height: 38px; background: var(--bg-titlebar); border-bottom: 1px solid var(--border-color); }
.window-controls { display: flex; gap: 8px; }
.mac-btn { width: 12px; height: 12px; border-radius: 50%; }
.mac-close { background: #ff5f56; }
.mac-min { background: #ffbd2e; }
.mac-max { background: #27c93f; }
.window-title { color: var(--text-muted); font-size: 12px; font-family: var(--font-mono); display: flex; }

.app-container { flex: 1; display: flex; overflow: hidden; }

.ribbon { width: 44px; background: var(--bg-ribbon); border-right: 1px solid var(--border-color); }
.ribbon-btn { width: 32px; height: 32px; border-radius: 6px; display: flex; align-items: center; justify-content: center; }
.ribbon-btn:hover { background: var(--bg-hover); color: var(--text-normal); }
.ribbon-btn.active { color: var(--accent-purple-light); background: var(--accent-purple-glow); }
.ribbon-bottom { margin-top: auto; display: flex; flex-direction: column; gap: 10px; }

.sidebar { width: 300px; background: var(--bg-sidebar); border-right: 1px solid var(--border-color); }
.sidebar-header { padding: 12px 14px 8px 14px; display: flex; justify-content: space-between; align-items: center; }
.vault-name { font-weight: 600; font-family: var(--font-mono); text-transform: uppercase; letter-spacing: 0.02em; }
.sidebar-actions { display: flex; gap: 8px; color: var(--text-muted); }
.sidebar-actions i { cursor: pointer; padding: 4px; border-radius: 4px; }
.sidebar-actions i:hover { background: var(--bg-hover); color: var(--text-normal); }

.search-container { padding: 0 10px 8px 10px; }
.search-trigger-input { width: 100%; background: var(--bg-ribbon); border: 1px solid var(--border-color); }

.view-tab-selector { display: flex; margin: 0 10px 8px 10px; background: var(--bg-ribbon); border: 1px solid var(--border-color); }
.view-tab-btn { flex: 1; border: none; background: transparent; color: var(--text-muted); font-size: 12px; }
.view-tab-btn.active { background: var(--bg-card); color: var(--text-normal); font-weight: 600; }
.view-tab-btn.drag-over { background: rgba(255, 95, 86, 0.25) !important; color: #ff5f56 !important; }

.notebook-navigator-list { flex: 1; overflow-y: auto; padding: 6px 8px; display: flex; flex-direction: column; }
.note-card { padding: 10px; border-radius: 6px; background: transparent; cursor: grab; display: flex; }
.note-card.active { cursor: grabbing; }
.note-card:hover { background: var(--bg-hover); }
.note-card.active { background: var(--bg-active); border-color: var(--border-color); }
.note-card.selected { background: var(--accent-purple-glow); border-color: var(--accent-purple-light); }
.note-card-title { font-weight: 600; font-size: 12px; color: var(--text-normal); white-space: nowrap; }
.note-card-snippet { font-size: 11px; color: var(--text-muted); display: -webkit-box; -webkit-line-clamp: 2; }
.note-card-meta { font-family: var(--font-mono); font-size: 10px; color: var(--text-faint); display: flex; }
.btn-card-bin { position: absolute; right: 8px; top: 10px; color: var(--text-faint); opacity: 0; }
.note-card:hover .btn-card-bin { opacity: 1; }
.btn-card-bin:hover { color: #ff5f56; background: var(--bg-card); }

#dragBadge { position: absolute; top: -100px; left: -100px; background: var(--accent-purple); color: var(--text-normal); padding: 2px 5px; font-size: 10px; }

.selection-bar { padding: 6px 10px; background: var(--accent-purple-glow); border-bottom: 1px solid var(--border-color); }

```

```

.selection-bar button { background: var(--bg-card); border: 1px solid var(--border-color); color:

.tags-pane { border-top: 1px solid var(--border-color); padding: 8px 10px; max-height: 130px; over
.tag-pill-list { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 4px; }
.tag-pill { background: var(--tag-bg); color: var(--accent-purple-light); padding: 2px 6px; border
.tag-pill.active { background: var(--accent-purple); color: #fff; }

.workspace { flex: 1; display: flex; flex-direction: column; background: var(--bg-editor); overflo

/* Notion-style Tabs Container */
.tab-bar { height: 36px; background: var(--bg-sidebar); border-bottom: 1px solid var(--border-colo
.tab-left { display: flex; align-items: flex-end; gap: 2px; overflow-x: auto; flex: 1; padding-top

/* Individual Tab */
.tab { height: 30px; padding: 0 10px; background: var(--bg-ribbon); border-radius: 6px 6px 0 0; di
.tab.active { background: var(--bg-editor); color: var(--text-normal); border-bottom: 1px solid va
.tab:hover:not(.active) { background: var(--bg-hover); }
.tab-title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; flex: 1; pointer-even
.tab-close { opacity: 0; padding: 2px 4px; border-radius: 4px; font-size: 10px; color: var(--text-
.tab:hover .tab-close, .tab.active .tab-close { opacity: 1; }
.tab-close:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-new-btn { cursor: pointer; padding: 4px 8px; margin-left: 4px; margin-bottom: 2px; color: var
.tab-new-btn:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-actions { display: flex; gap: 8px; align-items: center; margin-left: 12px; }
.editor-action-btn { background: var(--bg-card); border: 1px solid var(--border-color); color: var
.editor-action-btn:hover, .editor-action-btn.active { color: var(--text-normal); background: var(-

.workspace-content { flex: 1; display: flex; overflow: hidden; position: relative; }

/* Empty State */
.empty-state { display: none; flex: 1; align-items: center; justify-content: center; flex-directio
.empty-state i { font-size: 48px; margin-bottom: 16px; opacity: 0.2; }

.editor-pane { flex: 1; padding: 28px 42px; overflow-y: auto; display: flex; flex-direction: colum
.properties-block { background: rgba(255, 255, 255, 0.02); border: 1px solid var(--border-color);
.property-row { display: grid; grid-template-columns: 90px 1fr; align-items: center; gap: 10px; fo
.property-label { color: var(--text-muted); display: flex; align-items: center; gap: 6px; }
.property-input { background: transparent; border: none; color: var(--text-normal); font-family: v
input[type="date"].property-input { color-scheme: dark; cursor: pointer; }
body.light-theme input[type="date"].property-input { color-scheme: light; }

.note-title-input { font-size: 1.8rem; font-weight: 700; color: var(--text-normal); background: tr
.note-body-textarea { flex: 1; background: transparent; border: none; outline: none; color: var(--

/* Tag Template Bar */
.template-tags-bar { display: flex; flex-wrap: wrap; gap: 6px; padding: 6px 0; border-top: 1px das
.template-tag-chip { font-size: 10px; font-family: var(--font-mono); background: var(--bg-card); b
.template-tag-chip:hover { background: var(--bg-hover); color: var(--text-normal); border-color: v

/* Graph Pane */
.graph-pane { width: 340px; background: var(--bg-ribbon); border-left: 1px solid var(--border-colo
.graph-pane.collapsed { display: none; }
.graph-header { padding: 8px 12px; border-bottom: 1px solid var(--border-color); display: flex; al
.graph-mode-selector { display: flex; background: var(--bg-card); border: 1px solid var(--border-c
.graph-mode-btn { border: none; background: transparent; color: var(--text-muted); font-size: 10px
.graph-mode-btn.active { background: var(--accent-purple); color: #ffffff; font-weight: 600; }
.graph-canvas-container { flex: 1; position: relative; width: 100%; height: 100%; overflow: hidden
.graph-canvas-container.active { cursor: grabbing; }
#graphCanvas { width: 100%; height: 100%; display: block; }
.graph-zoom-controls { position: absolute; bottom: 12px; right: 12px; display: flex; flex-directio
.graph-zoom-btn { width: 24px; height: 24px; background: transparent; border: none; color: var(--t
.graph-zoom-btn:hover { background: var(--bg-hover); color: var(--accent-purple-light); }
.graph-tag-checkboxes { padding: 10px 12px; border-top: 1px solid var(--border-color); background:
.graph-tag-checkboxes.active { display: flex; }
.tag-checkbox-header { font-size: 10px; font-weight: 700; color: var(--text-faint); text-transform
.tag-checkbox-item { display: flex; align-items: center; gap: 8px; color: var(--text-muted); curso
.tag-checkbox-item:hover { color: var(--text-normal); }
.tag-checkbox-item input[type="checkbox"] { accent-color: var(--accent-purple); cursor: pointer; }

.calendar-pane { padding: 12px; display: flex; flex-direction: column; gap: 12px; font-family: var
.calendar-header { display: flex; justify-content: space-between; align-items: center; font-weight
.calendar-grid { display: grid; grid-template-columns: repeat(7, 1fr); gap: 4px; text-align: cente
.cal-day-label { font-size: 10px; color: var(--text-faint); padding-bottom: 4px; }
.cal-cell { font-size: 11px; padding: 6px 0; border-radius: 4px; cursor: pointer; color: var(--tex
.cal-cell:hover { background: var(--bg-hover); color: var(--text-normal); }
.cal-cell.today { border: 1px solid var(--accent-purple-light); font-weight: bold; color: var(--te
.cal-cell.has-note::after { content: ''; position: absolute; bottom: 2px; left: 50%; transform: tr

```

```

/* Modals & Menus */
.modal-overlay { position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; background: rgba(0
.modal-overlay.active { display: flex; }
.obsidian-settings-card { width: 840px; height: 560px; max-width: 95vw; max-height: 90vh; backgrou
.settings-nav { width: 220px; background: var(--bg-modal-nav); border-right: 1px solid var(--borde
.settings-nav-section { display: flex; flex-direction: column; gap: 2px; }
.settings-nav-header { font-size: 10px; font-weight: 700; color: var(--text-faint); padding: 4px 8
.settings-nav-item { padding: 6px 10px; border-radius: 4px; color: var(--text-muted); cursor: poin
.settings-nav-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.settings-nav-item.active { background: var(--bg-active); color: var(--text-normal); font-weight:
.settings-content { flex: 1; padding: 24px 32px; overflow-y: auto; display: flex; flex-direction:
.settings-section-title { font-size: 16px; font-weight: 700; color: var(--text-normal); border-bot
.obs-setting-item { display: flex; justify-content: space-between; align-items: flex-start; paddin
.obs-setting-info { display: flex; flex-direction: column; gap: 4px; max-width: 380px; }
.obs-setting-name { font-size: 12px; font-weight: 600; color: var(--text-normal); }
.obs-setting-desc { font-size: 11px; color: var(--text-muted); line-height: 1.4; }
.obs-setting-control { display: flex; align-items: center; gap: 8px; }
select.obs-select { background: var(--bg-card); border: 1px solid var(--border-color); color: var(
.obs-btn { background: var(--bg-card); border: 1px solid var(--border-color); color: var(--text-no
.obs-btn:hover { background: var(--bg-hover); }
.obs-btn-danger { background: #ff5f56; color: #fff; border: none; }

.search-modal { width: 640px; max-width: 90vw; background: var(--bg-sidebar); border: 1px solid va
.modal-search-header { padding: 14px; border-bottom: 1px solid var(--border-color); display: flex;
.modal-search-input { flex: 1; background: transparent; border: none; outline: none; color: var(--
.modal-search-body { display: flex; height: 340px; }
.modal-results-list { flex: 1; overflow-y: auto; padding: 8px; border-right: 1px solid var(--borde
.modal-result-item { padding: 8px 10px; border-radius: 6px; cursor: pointer; display: flex; align-
.modal-result-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.modal-preview-pane { width: 280px; background: var(--bg-editor); padding: 14px; overflow-y: auto;

.slash-menu { position: absolute; background: var(--bg-card); border: 1px solid var(--border-color
.slash-item { padding: 8px 12px; display: flex; align-items: center; gap: 8px; color: var(--text-n
.slash-item:hover { background: var(--accent-purple); color: #ffffff; }
</style>
</head>
<body>

<input type="file" id="folderImporter" webkitdirectory directory multiple style="display: none;" oncha
<input type="file" id="icsImporter" accept=".ics" style="display: none;" onchange="handleImportICS(eve
<div id="dragBadge">1 Note Selected</div>

<header class="window-header">
  <div class="window-controls">
    <div class="mac-btn mac-close"></div>
    <div class="mac-btn mac-min"></div>
    <div class="mac-btn mac-max"></div>
  </div>
  <div class="window-title"><i class="fa-solid fa-vault"></i> Vaukt Studio</div>
  <div></div>
</header>

<div class="app-container">
  <!-- Ribbon -->
  <nav class="ribbon">
    <div class="ribbon-btn active" id="ribbonNavBtn" title="Notebook Navigator" onclick="switchSid
    <div class="ribbon-btn" id="ribbonCalBtn" title="Apple Calendar Sync" onclick="switchSidebarVi
    <div class="ribbon-btn" title="Quick Search (Shift x2)" onclick="openSearchModal()"><i class="
    <div class="ribbon-btn active" id="ribbonGraphBtn" title="Toggle Graph View" onclick="toggleGr
    <div class="ribbon-btn" title="New Note" onclick="createNewNote()"><i class="fa-solid fa-plus"
    <div class="ribbon-bottom">
      <div class="ribbon-btn" title="Settings" onclick="openSettingsModal()"><i class="fa-solid
    </div>
  </nav>

  <!-- Sidebar Pane -->
  <aside class="sidebar">
    <div class="sidebar-header">
      <span class="vault-name" id="sidebarTitle">Notebook Navigator</span>
      <div class="sidebar-actions">
        <i class="fa-solid fa-file-import" title="Import Folder" onclick="triggerFolderImport(
        <i class="fa-solid fa-file-circle-plus" title="New Note" onclick="createNewNote()"></i>
      </div>
    </div>

    <div class="view-tab-selector">
      <button class="view-tab-btn active" id="tabNotesBtn" onclick="switchSidebarView('notes')">
      <button class="view-tab-btn" id="tabBinBtn" onclick="switchSidebarView('bin')">

```

```

                ondragover="handleDragOverBin(event)"
                ondragleave="handleDragLeaveBin(event)"
                ondrop="handleDropToBin(event)">
                <i class="fa-solid fa-trash-can"></i> Bin
            </button>
        </div>

<!-- Multi-Selection Bar -->
<div class="selection-bar" id="selectionBar" style="display: none;">
    <span id="selectedCountText">0 selected</span>
    <div style="display: flex; gap: 6px;">
        <button onclick="binSelectedNotes()"><i class="fa-solid fa-trash-can"></i> Move to Bin
        <button onclick="clearSelection()">Clear</button>
    </div>
</div>

<div class="search-container" id="searchBoxContainer">
    <div class="search-trigger-input" onclick="openSearchModal()">
        <span><i class="fa-solid fa-magnifying-glass" style="margin-right: 6px;"></i> Quick open
        <span class="search-shortcut-badge">Shift x2</span>
    </div>
</div>

<!-- Note List -->
<div class="notebook-navigator-list" id="navigatorList"></div>

<!-- Calendar View -->
<div class="calendar-pane" id="calendarView" style="display: none;">
    <div class="calendar-header">
        <span id="calMonthYear">August 2026</span>
        <div style="display: flex; gap: 6px;">
            <i class="fa-solid fa-chevron-left" style="cursor: pointer;" onclick="changeMonth(-1)"></i>
            <i class="fa-solid fa-chevron-right" style="cursor: pointer;" onclick="changeMonth(1)"></i>
        </div>
    </div>
    <div class="calendar-grid" id="calendarGrid"></div>

    <div style="margin-top: 12px; border-top: 1px solid var(--border-color); padding-top: 12px;">
        <div style="font-size: 10px; font-weight: 700; color: var(--text-faint); margin-bottom: 8px">
            <button class="obs-btn" style="width: 100%; margin-bottom: 6px;" onclick="triggerICSImport()"><i class="fa-solid fa-calendar-check" style="margin-right: 6px;"></i> Save as ICS
            <button class="obs-btn" style="width: 100%;" onclick="exportCalendarICS()"><i class="fa-solid fa-calendar-check" style="margin-right: 6px;"></i> Export ICS
        </div>
    </div>

<!-- Tags Drawer -->
<div class="tags-pane" id="tagsPane">
    <div style="font-size: 10px; font-weight: 700; font-family: var(--font-mono); color: var(--text-faint); margin-bottom: 8px">
        <div class="tag-pill-list" id="tagPillList"></div>
    </div>
</div>
</aside>

<!-- Workspace -->
<main class="workspace">
    <!-- NOTION STYLE TAB BAR -->
    <div class="tab-bar">
        <div class="tab-left" id="tabContainer">
            <!-- Tabs are injected dynamically here -->
        </div>
        <div class="tab-new-btn" title="Open new tab" onclick="createNewNote()"><i class="fa-solid fa-plus" style="font-size: 1.2em;"></i>
        <div class="tab-actions">
            <button class="editor-action-btn active" id="graphToggleBtn" title="Toggle Graph" onclick="toggleGraph()">
                <i class="fa-solid fa-circle-nodes"></i> Graph
            </button>
            <button class="editor-action-btn" title="Export this note as .md" onclick="downloadCurrentNote()">
                <i class="fa-solid fa-download"></i> Export .md
            </button>
            <button class="editor-action-btn" title="Move note to Bin" onclick="moveActiveNoteToBin()">
                <i class="fa-solid fa-trash-can"></i> Bin Note
            </button>
        </div>
    </div>

    <div class="workspace-content">
        <!-- EMPTY STATE (Shown when all tabs are closed) -->
        <div class="empty-state" id="emptyState">
            <i class="fa-solid fa-leaf"></i>
            <div>No file is open</div>
            <div style="margin-top: 8px; font-size: 11px;">Create a new note or select one from the list</div>
        </div>
    </div>
</main>

```

```

</div>

<!-- EDITOR PANE -->
<div class="editor-pane" id="editorPane">
  <input type="text" id="noteTitle" class="note-title-input" placeholder="Note Title..."

  <div class="properties-block">
    <div class="property-row">
      <span class="property-label"><i class="fa-regular fa-calendar-days"></i> creat
      <input type="date" id="propCreated" class="property-input" onchange="updateCur
    </div>
    <div class="property-row">
      <span class="property-label"><i class="fa-solid fa-tag"></i> tags</span>
      <input type="text" id="propTags" class="property-input" placeholder="tags sepa
    </div>
  </div>

  <!-- Template Tag Quick-Insert Bar -->
  <div class="template-tags-bar" id="templateTagsBar">
    <span style="font-size:10px; font-family:var(--font-mono); color:var(--text-faint)"
  </div>

  <textarea id="noteContent" class="note-body-textarea" placeholder="Type note content..

  <div id="slashMenu" class="slash-menu">
    <div style="padding:6px 10px; font-size:10px; font-weight:700; font-family:var(--f
    <div id="slashList"></div>
  </div>
</div>

<!-- Canvas Graph Pane -->
<div class="graph-pane" id="graphPane">
  <div class="graph-header">
    <span><i class="fa-solid fa-diagram-project" style="margin-right:4px;"></i> GRAPH
    <div class="graph-mode-selector">
      <button class="graph-mode-btn active" id="btnModeLinks" onclick="setGraphMode(
      <button class="graph-mode-btn" id="btnModeTags" onclick="setGraphMode('tags')"
    </div>
    <i class="fa-solid fa-xmark" style="cursor:pointer;" onclick="toggleGraphPane()"><
  </div>

  <div class="graph-canvas-container" id="graphCanvasContainer">
    <canvas id="graphCanvas"></canvas>
    <div class="graph-zoom-controls">
      <button class="graph-zoom-btn" title="Zoom In" onclick="adjustGraphZoom(1.2)">
      <button class="graph-zoom-btn" title="Reset Zoom" onclick="resetGraphZoom()"><
      <button class="graph-zoom-btn" title="Zoom Out" onclick="adjustGraphZoom(0.8)"
    </div>
  </div>

  <div class="graph-tag-checkboxes" id="graphTagCheckboxes">
    <div class="tag-checkbox-header">
      <span>Filter Visible Tags</span>
      <span style="cursor:pointer; color:var(--accent-purple-light);" onclick="selec
    </div>
    <div id="tagCheckboxList"></div>
  </div>
</div>
</main>
</div>

<!-- Authentic Obsidian Settings Modal -->
<div class="modal-overlay" id="settingsModalOverlay" onclick="closeSettingsModal(event)">
  <div class="obsidian-settings-card" onclick="event.stopPropagation()">
    <div class="settings-nav">
      <div class="settings-nav-section">
        <div class="settings-nav-header">Options</div>
        <div class="settings-nav-item">General</div>
        <div class="settings-nav-item">Editor</div>
        <div class="settings-nav-item">Files and links</div>
        <div class="settings-nav-item active">Appearance</div>
      </div>
    </div>

    <div class="settings-content">
      <div class="settings-section-title">Appearance</div>
      <div class="obs-setting-item">
        <div class="obs-setting-info">

```

```

        <div class="obs-setting-name">Base color scheme</div>
        <div class="obs-setting-desc">Choose Obsidian's default color scheme.</div>
    </div>
    <div class="obs-setting-control">
        <select class="obs-select" id="themeSelect" onchange="changeThemeFromSelect()">
            <option value="dark">Dark</option>
            <option value="light">Light</option>
        </select>
    </div>
</div>
<div class="obs-setting-item">
    <div class="obs-setting-info">
        <div class="obs-setting-name">Vault Storage Backup</div>
        <div class="obs-setting-desc">Export all notes into a ZIP folder archive.</div>
    </div>
    <div class="obs-setting-control">
        <button class="obs-btn" onclick="exportAllNotesZip()"><i class="fa-solid fa-file-z
    </div>
</div>
<div class="obs-setting-item">
    <div class="obs-setting-info">
        <div class="obs-setting-name" style="color:#ff5f56;">Reset Vault</div>
        <div class="obs-setting-desc">Exports all notes as a .md ZIP folder before permane
    </div>
    <div class="obs-setting-control">
        <button class="obs-btn obs-btn-danger" onclick="resetVaultWithBackup()"><i class="
    </div>
</div>
</div>
</div>
</div>
<!-- Search Modal -->
<div class="modal-overlay" id="searchModalOverlay" onclick="closeSearchModal(event)">
    <div class="search-modal" onclick="event.stopPropagation()">
        <div class="modal-search-header">
            <i class="fa-solid fa-magnifying-glass" style="color:var(--text-faint);"></i>
            <input type="text" id="modalSearchInput" class="modal-search-input" placeholder="Type note
        </div>
        <div class="modal-search-body">
            <div class="modal-results-list" id="modalResultsList"></div>
            <div class="modal-preview-pane">
                <div id="modalPreviewTitle" style="font-weight:700; font-size:12px; color:var(--text-n
                <div id="modalPreviewSnippet"></div>
            </div>
        </div>
    </div>
</div>
</div>
</div>
<script>
    function getSystemTodayDate() {
        const today = new Date();
        const yyyy = today.getFullYear();
        const mm = String(today.getMonth() + 1).padStart(2, '0');
        const dd = String(today.getDate()).padStart(2, '0');
        return `${yyyy}-${mm}-${dd}`;
    }

    const templateTagsList = [
        "Short Form",
        "Long Form",
        "Project",
        "Business",
        "Company",
        "List Of Ideas",
        "Undecided",
        "Music",
        ":: Artwork",
        ":: Random Idea",
        ":: Documentary",
        ":: Event",
        "::Notes",
        "Journal Entry",
        "Off Frame",
        "Saevin Go Outside Project"
    ];

    let notes = JSON.parse(localStorage.getItem('vaukt_minimal_notes')) || [
        { id: 1, title: '2026-8-5 Building Out the team', content: 'Ok what I want is that I need peop

```

```

    { id: 2, title: 'Go Outside Marketing Strategy', content: 'Brainstorming community events and
    { id: 3, title: 'Project Timeline & Milestones', content: 'Documenting content production sche
];

let activeNoteId = notes.find(n => !n.isBinned) ? notes.find(n => !n.isBinned).id : null;
let openTabs = activeNoteId ? [activeNoteId] : [];
let selectedNoteIds = new Set(activeNoteId ? [activeNoteId] : []);
let lastSelectedId = activeNoteId;

let activeSidebarView = 'notes';
let activeTagFilter = null;
let lastShiftTime = 0;
let graphMode = 'links';
let isGraphOpen = true;

let graphZoom = 1;
let graphPanX = 0;
let graphPanY = 0;
let isPanningGraph = false;
let panStartX = 0;
let panStartY = 0;
let selectedGraphTags = new Set();

let calCurrentDate = new Date();
let slashActive = false;
let slashStartPos = -1;
let filteredSlashNotes = [];

window.addEventListener('DOMContentLoaded', () => {
  const savedTheme = localStorage.getItem('vaukt_theme');
  if (savedTheme === 'light') {
    document.body.classList.add('light-theme');
    document.getElementById('themeSelect').value = 'light';
  }

  renderTemplateTagsBar();
  renderNavigatorList();
  renderTags();
  if (activeNoteId) loadNote(activeNoteId);
  else renderEditorState();

  initGraph();
  renderCalendar();

  document.addEventListener('keydown', (e) => {
    if (e.key === 'Shift') {
      const currentTime = new Date().getTime();
      if (currentTime - lastShiftTime < 300) openSearchModal();
      lastShiftTime = currentTime;
    }
  });

  window.addEventListener('resize', () => { if (isGraphOpen) drawGraph(); });
});

function saveStorage() { localStorage.setItem('vaukt_minimal_notes', JSON.stringify(notes)); }

function renderTemplateTagsBar() {
  const bar = document.getElementById('templateTagsBar');
  // Keep the label element intact
  const labelEl = bar.querySelector('span');
  bar.innerHTML = '';
  bar.appendChild(labelEl);

  templateTagsList.forEach(tagName => {
    const chip = document.createElement('div');
    chip.className = 'template-tag-chip';
    chip.innerHTML = `<i class="fa-solid fa-plus" style="font-size:9px;"></i> ${escapeHtml(tag
    chip.onclick = () => insertTagIntoCurrentNote(tagName);
    bar.appendChild(chip);
  });
}

function insertTagIntoCurrentNote(tagName) {
  if (!activeNoteId) return;
  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;

  let currentTags = note.tags ? note.tags.split(',').map(t => t.trim()).filter(Boolean) : [];

```

```

    if (!currentTags.includes(tagName)) {
      currentTags.push(tagName);
      note.tags = currentTags.join(', ');
      document.getElementById('propTags').value = note.tags;
      updateCurrentNote();
    }
  }

function renderTabs() {
  const container = document.getElementById('tabContainer');
  container.innerHTML = '';

  openTabs.forEach(id => {
    const note = notes.find(n => n.id === id);
    if (!note) return;

    const isActive = id === activeNoteId;
    const tab = document.createElement('div');
    tab.className = `tab ${isActive ? 'active' : ''}`;
    tab.onclick = () => loadNote(id);

    const displayTitle = note.title.trim() === '' ? 'Untitled' : note.title;

    tab.innerHTML = `
      <i class="fa-solid fa-leaf" style="color: ${isActive ? 'var(--accent-purple-light)' : ''} :
      <span class="tab-title">${escapeHtml(displayTitle)}</span>
      <i class="fa-solid fa-xmark tab-close" onclick="event.stopPropagation(); closeTab(${id}
    `;
    container.appendChild(tab);
  });
}

function closeTab(id) {
  openTabs = openTabs.filter(t => t !== id);
  if (id === activeNoteId) {
    if (openTabs.length > 0) {
      loadNote(openTabs[openTabs.length - 1]);
    } else {
      activeNoteId = null;
      renderEditorState();
    }
  } else {
    renderTabs();
  }
}

function renderEditorState() {
  const editorPane = document.getElementById('editorPane');
  const emptyState = document.getElementById('emptyState');

  renderTabs();
  renderNavigatorList();

  if (!activeNoteId) {
    editorPane.style.display = 'none';
    emptyState.style.display = 'flex';
    if (isGraphOpen) drawGraph();
    return;
  }

  editorPane.style.display = 'flex';
  emptyState.style.display = 'none';

  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;

  document.getElementById('noteTitle').value = note.title;
  document.getElementById('noteContent').value = note.content;
  document.getElementById('propTags').value = note.tags || '';
  document.getElementById('propCreated').value = note.created || getSystemTodayDate();

  if (isGraphOpen) drawGraph();
}

function loadNote(id) {
  const note = notes.find(n => n.id === id);
  if (!note) return;

  if (!openTabs.includes(id)) {

```

```

        openTabs.push(id);
    }

    activeNoteId = id;
    selectedNoteIds.clear();
    selectedNoteIds.add(id);
    lastSelectedId = id;

    renderEditorState();
}

function createNewNote() {
    const newNote = { id: Date.now(), title: "", content: "", tags: "", created: getSystemTodayDate() };
    notes.unshift(newNote);
    saveStorage();

    switchSidebarView('notes');
    renderTags();
    loadNote(newNote.id);
    setTimeout(() => { document.getElementById('noteTitle').focus(); }, 50);
}

function updateCurrentNote() {
    if (!activeNoteId) return;
    const note = notes.find(n => n.id === activeNoteId);
    if (!note) return;

    note.title = document.getElementById('noteTitle').value;
    note.content = document.getElementById('noteContent').value;
    note.tags = document.getElementById('propTags').value;
    note.created = document.getElementById('propCreated').value || getSystemTodayDate();

    saveStorage();
    renderTabs();
    renderNavigatorList();
    renderTags();
    if (isGraphOpen) {
        renderTagCheckboxes();
        drawGraph();
    }
}

async function formatLinkAsMarkdown(url) {
    try {
        let title = "";
        if (url.includes("youtube.com") || url.includes("youtu.be")) {
            if (url.includes("07vGsBghWfc")) {
                title = "why I switched to using Obsidian (as a former Notion user)";
            } else {
                title = "YouTube Video";
            }
        } else {
            const domainObj = new URL(url);
            title = domainObj.hostname.replace("www.", "");
        }
        return `[${title}](${url})`;
    } catch (e) {
        return url;
    }
}

async function handleEditorPaste(e) {
    const clipboardData = e.clipboardData || window.clipboardData;
    const pastedText = clipboardData.getData('text');

    if (pastedText && (pastedText.startsWith('http://') || pastedText.startsWith('https://'))) {
        e.preventDefault();
        const textarea = e.target;
        const start = textarea.selectionStart;
        const end = textarea.selectionEnd;
        const val = textarea.value;

        const formattedLink = await formatLinkAsMarkdown(pastedText.trim());

        textarea.value = val.substring(0, start) + formattedLink + val.substring(end);
        textarea.selectionStart = textarea.selectionEnd = start + formattedLink.length;
        updateCurrentNote();
    }
}

```

```

async function handleEditorInput(e) {
  updateCurrentNote();
  const textarea = e.target;
  const pos = textarea.selectionStart;
  const val = textarea.value;

  if (val.slice(pos - 1, pos) === ' ' || val.slice(pos - 1, pos) === '\n') {
    const textBeforeCursor = val.substring(0, pos - 1);
    const words = textBeforeCursor.split(/\s+/);
    const lastWord = words[words.length - 1];

    if (lastWord && (lastWord.startsWith('http://') || lastWord.startsWith('https://')) && !la
      const formattedLink = await formatLinkAsMarkdown(lastWord);
      const updatedBefore = textBeforeCursor.substring(0, textBeforeCursor.length - lastWord
      textarea.value = updatedBefore + formattedLink + val.substring(pos - 1);
      textarea.selectionStart = textarea.selectionEnd = updatedBefore.length + formattedLink
      updateCurrentNote();
    }
  }

  if (val.slice(pos - 1, pos) === '/') {
    slashActive = true;
    slashStartPos = pos - 1;
    document.getElementById('slashMenu').style.display = 'block';
    renderSlashList('');
  }
  if (slashActive) renderSlashList(val.substring(slashStartPos + 1, pos).toLowerCase());
}

function switchSidebarView(view) {
  activeSidebarView = view;
  document.getElementById('tabNotesBtn').classList.toggle('active', view === 'notes');
  document.getElementById('tabBinBtn').classList.toggle('active', view === 'bin');
  document.getElementById('ribbonNavBtn').classList.toggle('active', view === 'notes');
  document.getElementById('ribbonCalBtn').classList.toggle('active', view === 'calendar');

  const list = document.getElementById('navigatorList');
  const cal = document.getElementById('calendarView');
  const searchBox = document.getElementById('searchBoxContainer');
  const tagsPane = document.getElementById('tagsPane');

  if (view === 'calendar') {
    list.style.display = 'none';
    searchBox.style.display = 'none';
    tagsPane.style.display = 'none';
    cal.style.display = 'flex';
    document.getElementById('sidebarTitle').innerText = 'Calendar View';
    renderCalendar();
  } else {
    list.style.display = 'flex';
    searchBox.style.display = 'block';
    tagsPane.style.display = 'block';
    cal.style.display = 'none';
    document.getElementById('sidebarTitle').innerText = view === 'bin' ? 'Bin Trash' : 'Notebo
    renderNavigatorList();
  }
}

function handleNoteClick(e, noteId) {
  const visibleNotes = getVisibleNotes();
  if (e.metaKey || e.ctrlKey) {
    if (selectedNoteIds.has(noteId)) selectedNoteIds.delete(noteId);
    else selectedNoteIds.add(noteId);
    lastSelectedId = noteId;
  } else if (e.shiftKey && lastSelectedId !== null) {
    const idx1 = visibleNotes.findIndex(n => n.id === lastSelectedId);
    const idx2 = visibleNotes.findIndex(n => n.id === noteId);
    if (idx1 !== -1 && idx2 !== -1) {
      const start = Math.min(idx1, idx2);
      const end = Math.max(idx1, idx2);
      selectedNoteIds.clear();
      for (let i = start; i <= end; i++) selectedNoteIds.add(visibleNotes[i].id);
    }
  } else {
    selectedNoteIds.clear();
    selectedNoteIds.add(noteId);
    lastSelectedId = noteId;
    loadNote(noteId);
  }
}

```

```

    }
    updateSelectionUI();
}

function updateSelectionUI() {
    const bar = document.getElementById('selectionBar');
    const text = document.getElementById('selectedCountText');
    if (selectedNoteIds.size > 1) {
        bar.style.display = 'flex';
        text.innerText = `${selectedNoteIds.size} notes selected`;
    } else {
        bar.style.display = 'none';
    }
    renderNavigatorList();
}

function clearSelection() {
    selectedNoteIds.clear();
    if (activeNoteId) selectedNoteIds.add(activeNoteId);
    updateSelectionUI();
}

function handleDragStart(e, noteId) {
    if (!selectedNoteIds.has(noteId)) {
        selectedNoteIds.clear();
        selectedNoteIds.add(noteId);
        lastSelectedId = noteId;
        renderNavigatorList();
    }
    const idsArray = Array.from(selectedNoteIds);
    e.dataTransfer.setData('text/plain', JSON.stringify(idsArray));
    const badge = document.getElementById('dragBadge');
    badge.innerText = `${idsArray.length} ${idsArray.length === 1 ? 'Note' : 'Notes'}`;
    e.dataTransfer.setDragImage(badge, 0, 0);
}

function handleDragOverBin(e) { e.preventDefault(); document.getElementById('tabBinBtn').classList
function handleDragLeaveBin(e) { document.getElementById('tabBinBtn').classList.remove('drag-over'

function handleDropToBin(e) {
    e.preventDefault();
    document.getElementById('tabBinBtn').classList.remove('drag-over');
    const rawData = e.dataTransfer.getData('text/plain');
    if (!rawData) return;

    try {
        const idsToBin = JSON.parse(rawData);
        notes.forEach(n => { if (idsToBin.includes(n.id)) n.isBinned = true; });
        saveStorage();

        openTabs = openTabs.filter(id => !idsToBin.includes(id));
        if (idsToBin.includes(activeNoteId)) {
            activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
        }

        selectedNoteIds.clear();
        if (activeNoteId) selectedNoteIds.add(activeNoteId);

        renderEditorState();
        renderTags();
    } catch (err) { console.error(err); }
}

function binSelectedNotes() {
    notes.forEach(n => { if (selectedNoteIds.has(n.id)) n.isBinned = true; });
    saveStorage();

    const binnedIds = Array.from(selectedNoteIds);
    openTabs = openTabs.filter(id => !binnedIds.includes(id));
    if (binnedIds.includes(activeNoteId)) {
        activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
    }

    selectedNoteIds.clear();
    if (activeNoteId) selectedNoteIds.add(activeNoteId);

    renderEditorState();
    renderTags();
}

```

```

function getVisibleNotes() {
  const isBinView = activeSidebarView === 'bin';
  return notes.filter(n => {
    if (isBinView) return n.isBinned;
    if (n.isBinned) return false;
    return activeTagFilter ? n.tags.split(',').map(t=>t.trim()).includes(activeTagFilter) : true;
  });
}

function moveActiveNoteToBin() {
  if (!activeNoteId) return;
  selectedNoteIds.clear();
  selectedNoteIds.add(activeNoteId);
  binSelectedNotes();
}

function restoreNoteFromBin(id) {
  const note = notes.find(n => n.id === id);
  if (note) { note.isBinned = false; saveStorage(); renderNavigatorList(); }
}

function permanentlyDeleteNote(id) {
  if (confirm("Permanently delete this note?")) {
    notes = notes.filter(n => n.id !== id);
    selectedNoteIds.delete(id);
    openTabs = openTabs.filter(t => t !== id);
    if (activeNoteId === id) activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length-1] : null;
    saveStorage();
    renderEditorState();
  }
}

function moveNoteIdToBin(id) {
  const note = notes.find(n => n.id === id);
  if (note) {
    note.isBinned = true;
    saveStorage();
    openTabs = openTabs.filter(t => t !== id);
    if (activeNoteId === id) activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length-1] : null;
    renderEditorState();
  }
}

function renderCalendar() {
  const grid = document.getElementById('calendarGrid');
  grid.innerHTML = '';
  const year = calCurrentDate.getFullYear();
  const month = calCurrentDate.getMonth();
  const monthNames = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"];
  document.getElementById('calMonthYear').innerText = `${monthNames[month]} ${year}`;

  ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'].forEach(d => {
    const l = document.createElement('div');
    l.className = 'cal-day-label';
    l.innerText = d;
    grid.appendChild(l);
  });

  const firstDay = new Date(year, month, 1).getDay();
  const daysInMonth = new Date(year, month + 1, 0).getDate();
  for (let i = 0; i < firstDay; i++) grid.appendChild(document.createElement('div'));

  const todayStr = getSystemTodayDate();
  for (let day = 1; day <= daysInMonth; day++) {
    const cell = document.createElement('div');
    cell.className = 'cal-cell';
    const mm = String(month + 1).padStart(2, '0');
    const dd = String(day).padStart(2, '0');
    const dateStr = `${year}-${mm}-${dd}`;

    cell.innerText = day;
    if (dateStr === todayStr) cell.classList.add('today');
    if (notes.some(n => !n.isBinned && n.created === dateStr)) cell.classList.add('has-note');

    cell.onclick = () => {
      const matchedNote = notes.find(n => !n.isBinned && n.created === dateStr);
      if (matchedNote) {
        switchSidebarView('notes');
      }
    };
  }
}

```

```

        loadNote(matchedNote.id);
    } else if (confirm(`Create new note for date ${dateStr}?`)) {
        const newNote = { id: Date.now(), title: `Note ${dateStr}`, content: '', tags: 'Jo
        notes.unshift(newNote);
        saveStorage();
        switchSidebarView('notes');
        loadNote(newNote.id);
    }
    };
    grid.appendChild(cell);
}
}

function changeMonth(dir) { calCurrentDate.setMonth(calCurrentDate.getMonth() + dir); renderCalend
function triggerICSImport() { document.getElementById('icsImporter').click(); }

function handleImportICS(e) {
    const file = e.target.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = function(evt) {
        const text = evt.target.result;
        const summaryMatch = text.match(/SUMMARY:(.*)/);
        const title = summaryMatch ? summaryMatch[1].trim() : "Imported Event";
        const newNote = { id: Date.now(), title: title, content: "Imported from Apple Calendar\n\n
        notes.unshift(newNote);
        saveStorage();
        switchSidebarView('notes');
        loadNote(newNote.id);
        alert("Apple Calendar Event Imported!");
    };
    reader.readAsText(file);
}

function exportCalendarICS() {
    let icsContent = "BEGIN:VCALENDAR\nVERSION:2.0\nPRODID:-//Vaukt Studio//EN\n";
    notes.filter(n => !n.isBinned).forEach(n => {
        const dateClean = (n.created || getSystemTodayDate()).replace(/-/g, '');
        icsContent += `BEGIN:VEVENT\nSUMMARY:${n.title}\nDESCRIPTION:${n.content.replace(/\n/g, '
    });
    icsContent += "END:VCALENDAR";
    saveAs(new Blob([icsContent], { type: 'text/calendar;charset=utf-8' }), "Vaukt_Apple_Calendar_
}

function resetVaultWithBackup() {
    if (!confirm("ARE YOU SURE?\n\nResetting will download all your files as a ZIP archive and wip
    exportAllNotesZip());
    setTimeout(() => {
        notes = [];
        activeNoteId = null;
        openTabs = [];
        selectedNoteIds.clear();
        saveStorage();
        renderEditorState();
        renderTags();
        closeSettingsModal(null, true);
        alert("Vault reset complete.");
    }, 300);
}

function renderNavigatorList() {
    const list = document.getElementById('navigatorList');
    list.innerHTML = '';
    const isBinView = activeSidebarView === 'bin';
    const filteredNotes = getVisibleNotes();

    filteredNotes.forEach(note => {
        const card = document.createElement('div');
        const isSelected = selectedNoteIds.has(note.id);
        const isActive = note.id === activeNoteId;
        card.className = `note-card ${isActive ? 'active' : ''} ${isSelected ? 'selected' : ''}`;

        if (isBinView) {
            card.innerHTML = `
                <div class="note-card-title">${escapeHtml(note.title || 'Untitled')}</div>
                <div class="note-card-snippet">${escapeHtml(note.content || 'Empty note...')}</div>
                <div style="display:flex; gap:8px; margin-top:6px;">
                    <button class="obs-btn" style="font-size:9px; padding:2px 6px;" onclick="resto
                    <button class="obs-btn obs-btn-danger" style="font-size:9px; padding:2px 6px;"

```

```

        </div>
    `;
} else {
    card.setAttribute('draggable', 'true');
    card.ondragstart = (e) => handleDragStart(e, note.id);
    card.onclick = (e) => handleNoteClick(e, note.id);
    card.innerHTML = `
        <div class="note-card-title">${escapeHtml(note.title || 'Untitled')}</div>
        <div class="note-card-snippet">${escapeHtml(note.content || 'Empty note...')}</div>
        <div class="note-card-meta">
            <span><i class="fa-regular fa-calendar"></i> ${note.created || getSystemTodayD
            ${note.tags ? `<span>#${escapeHtml(note.tags.split(', ')[0])}</span>` : ''}
        </div>
        <i class="fa-solid fa-trash-can btn-card-bin" title="Move to Bin" onclick="event.s
    `;
}
list.appendChild(card);
});
}

function renderTags() {
    const list = document.getElementById('tagPillList');
    list.innerHTML = '';
    const tagMap = {};
    notes.filter(n=>!n.isBinned).forEach(n => {
        if (n.tags) {
            n.tags.split(',').forEach(t => {
                const clean = t.trim();
                if (clean) tagMap[clean] = (tagMap[clean] || 0) + 1;
            });
        }
    });
    Object.keys(tagMap).forEach(tag => {
        const pill = document.createElement('div');
        pill.className = `tag-pill ${activeTagFilter === tag ? 'active' : ''}`;
        pill.innerText = `#${tag} (${tagMap[tag]})`;
        pill.onclick = () => {
            activeTagFilter = activeTagFilter === tag ? null : tag;
            renderTags();
            renderNavigatorList();
        };
        list.appendChild(pill);
    });
}

function changeThemeFromSelect() {
    const val = document.getElementById('themeSelect').value;
    if (val === 'light') document.body.classList.add('light-theme');
    else document.body.classList.remove('light-theme');
    localStorage.setItem('vaukt_theme', val);
    if (isGraphOpen) drawGraph();
}

function toggleGraphPane() {
    isGraphOpen = !isGraphOpen;
    const pane = document.getElementById('graphPane');
    const ribbonBtn = document.getElementById('ribbonGraphBtn');
    const toggleBtn = document.getElementById('graphToggleBtn');

    if (isGraphOpen) {
        pane.classList.remove('collapsed');
        ribbonBtn.classList.add('active');
        toggleBtn.classList.add('active');
        setTimeout(drawGraph, 50);
    } else {
        pane.classList.add('collapsed');
        ribbonBtn.classList.remove('active');
        toggleBtn.classList.remove('active');
    }
}

function setGraphMode(mode) {
    graphMode = mode;
    document.getElementById('btnModeLinks').classList.toggle('active', mode === 'links');
    document.getElementById('btnModeTags').classList.toggle('active', mode === 'tags');

    const tagContainer = document.getElementById('graphTagCheckboxes');
    if (mode === 'tags') {
        tagContainer.classList.add('active');
    }
}

```

```

    renderTagCheckboxes();
  } else {
    tagContainer.classList.remove('active');
  }
  drawGraph();
}

function getAllUniqueTags() {
  const tags = new Set();
  notes.filter(n => !n.isBinned).forEach(n => {
    if (n.tags) {
      n.tags.split(',').forEach(t => {
        const clean = t.trim();
        if (clean) tags.add(clean);
      });
    }
  });
  return Array.from(tags);
}

function renderTagCheckboxes() {
  const container = document.getElementById('tagCheckboxList');
  container.innerHTML = '';
  const allTags = getAllUniqueTags();

  if (selectedGraphTags.size === 0 && allTags.length > 0) {
    allTags.forEach(t => selectedGraphTags.add(t));
  }

  allTags.forEach(tag => {
    const label = document.createElement('label');
    label.className = 'tag-checkbox-item';
    const isChecked = selectedGraphTags.has(tag);

    label.innerHTML = `
      <input type="checkbox" ${isChecked ? 'checked' : ''} onchange="toggleTagFilter('${tag}'
      <span>#${escapeHtml(tag)}</span>
    `;
    container.appendChild(label);
  });
}

function toggleTagFilter(tag, isChecked) {
  if (isChecked) selectedGraphTags.add(tag);
  else selectedGraphTags.delete(tag);
  drawGraph();
}

function selectAllGraphTags(status) {
  const allTags = getAllUniqueTags();
  selectedGraphTags.clear();
  if (status) allTags.forEach(t => selectedGraphTags.add(t));
  renderTagCheckboxes();
  drawGraph();
}

function adjustGraphZoom(factor) {
  graphZoom *= factor;
  graphZoom = Math.min(Math.max(0.3, graphZoom), 4.0);
  drawGraph();
}

function resetGraphZoom() {
  graphZoom = 1; graphPanX = 0; graphPanY = 0;
  drawGraph();
}

function openSettingsModal() { document.getElementById('settingsModalOverlay').classList.add('active'); }
function closeSettingsModal(e, force = false) {
  if (force || e.target.id === 'settingsModalOverlay') document.getElementById('settingsModalOverlay').classList.remove('active');
}

function downloadCurrentNoteMD() {
  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;
  const filename = (note.title.trim() || 'Untitled').replace(/[/\?%*:|"<>]/g, '_') + '.md';
  saveAs(new Blob([`---\ncreated: ${note.created} | getSystemTodayDate() | \ntags: ${note.tags} | `]), filename);
}

```

```

function exportAllNotesZip() {
  if (notes.length === 0) return alert("No notes to export.");
  const zip = new JSZip();
  const folder = zip.folder("Vaukt_Vault");
  notes.forEach(note => {
    const filename = (note.title.trim() || 'Untitled_' + note.id).replace(/[\/\?%*:|"<>]/g, '_')
    folder.file(filename, `---\ncreated: ${note.created || getSystemTodayDate()}\nntags: ${note
    });
    zip.generateAsync({ type: "blob" }).then(content => saveAs(content, "Vaukt_Vault_Export.zip"))
  }

function triggerFolderImport() { document.getElementById('folderImporter').click(); }

async function handleImportFolder(event) {
  const files = event.target.files;
  if (!files || files.length === 0) return;
  for (let i = 0; i < files.length; i++) {
    const file = files[i];
    if (file.name.endsWith('.txt') || file.name.endsWith('.md')) {
      const content = await file.text();
      notes.unshift({ id: Date.now() + i, title: file.name.replace(/\.([^.]+)$/, ""), content
    }
  }
  saveStorage();
  renderNavigatorList();
  renderTags();
  if (notes.length > 0) loadNote(notes[0].id);
}

function handleEditorKeyDown(e) {
  if (!slashActive) return;
  if (e.key === 'Enter' && filteredSlashNotes.length > 0) {
    e.preventDefault();
    insertPageLink(filteredSlashNotes[0].title);
  }
}

function renderSlashList(query) {
  const list = document.getElementById('slashList');
  list.innerHTML = '';
  filteredSlashNotes = notes.filter(n => !n.isBinned && n.id !== activeNoteId && n.title.toLowerCase
  filteredSlashNotes.forEach((note) => {
    const item = document.createElement('div');
    item.className = 'slash-item';
    item.innerText = note.title;
    item.onmousedown = (e) => { e.preventDefault(); insertPageLink(note.title); };
    list.appendChild(item);
  });
}

function insertPageLink(title) {
  const textarea = document.getElementById('noteContent');
  const pos = textarea.selectionStart;
  const val = textarea.value;
  textarea.value = val.substring(0, slashStartPos) + `[${title}]` + val.substring(pos);
  document.getElementById('slashMenu').style.display = 'none';
  slashActive = false;
  updateCurrentNote();
}

function openSearchModal() {
  document.getElementById('searchModalOverlay').classList.add('active');
  document.getElementById('modalSearchInput').value = '';
  document.getElementById('modalSearchInput').focus();
  handleModalSearch();
}

function closeSearchModal(e) {
  if (e.target.id === 'searchModalOverlay') document.getElementById('searchModalOverlay').classL
}

function handleModalSearch() {
  const query = document.getElementById('modalSearchInput').value.toLowerCase();
  const results = notes.filter(n => !n.isBinned && (n.title.toLowerCase().includes(query) || n.c
  const list = document.getElementById('modalResultsList');
  list.innerHTML = '';
  results.forEach((note) => {
    const item = document.createElement('div');
    item.className = 'modal-result-item';

```

```

        item.innerHTML = `<span><i class="fa-regular fa-file-lines"></i> ${escapeHtml(note.title)}
        item.onclick = () => {
            loadNote(note.id);
            document.getElementById('searchModalOverlay').classList.remove('active');
        };
        list.appendChild(item);
    });
    if (results[0]) {
        document.getElementById('modalPreviewTitle').innerText = results[0].title;
        document.getElementById('modalPreviewSnippet').innerText = results[0].content;
    }
}

let canvas, ctx, container;
function initGraph() {
    canvas = document.getElementById('graphCanvas');
    container = document.getElementById('graphCanvasContainer');
    ctx = canvas.getContext('2d');

    container.addEventListener('wheel', (e) => {
        e.preventDefault();
        adjustGraphZoom(e.deltaY < 0 ? 1.1 : 0.9);
    }, { passive: false });

    container.addEventListener('mousedown', (e) => {
        isPanningGraph = true;
        panStartX = e.clientX - graphPanX;
        panStartY = e.clientY - graphPanY;
    });

    window.addEventListener('mousemove', (e) => {
        if (!isPanningGraph) return;
        graphPanX = e.clientX - panStartX;
        graphPanY = e.clientY - panStartY;
        drawGraph();
    });

    window.addEventListener('mouseup', () => { isPanningGraph = false; });
    drawGraph();
}

function drawGraph() {
    if (!canvas || !container || !isGraphOpen) return;

    canvas.width = container.clientWidth;
    canvas.height = container.clientHeight;
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    let activeNotes = notes.filter(n => !n.isBinned);
    if (graphMode === 'tags' && selectedGraphTags.size > 0) {
        activeNotes = activeNotes.filter(n => {
            if (!n.tags) return false;
            const nTags = n.tags.split(',').map(t => t.trim());
            return nTags.some(t => selectedGraphTags.has(t));
        });
    }

    if (activeNotes.length === 0) return;

    ctx.save();
    ctx.translate(canvas.width / 2 + graphPanX, canvas.height / 2 + graphPanY);
    ctx.scale(graphZoom, graphZoom);

    const radius = Math.min(canvas.width, canvas.height) * 0.35;
    const nodePositions = {};

    activeNotes.forEach((note, idx) => {
        const angle = (idx / activeNotes.length) * Math.PI * 2;
        nodePositions[note.id] = { x: radius * Math.cos(angle), y: radius * Math.sin(angle), title
    });

    ctx.lineWidth = 1 / graphZoom;

    if (graphMode === 'links') {
        activeNotes.forEach(sourceNote => {
            activeNotes.forEach(targetNote => {
                if (sourceNote.id !== targetNote.id && sourceNote.content && sourceNote.content.in
                const p1 = nodePositions[sourceNote.id];
                const p2 = nodePositions[targetNote.id];

```

```

        if (p1 && p2) {
            ctx.beginPath(); ctx.moveTo(p1.x, p1.y); ctx.lineTo(p2.x, p2.y);
            ctx.strokeStyle = "rgba(124, 58, 237, 0.4)"; ctx.stroke();
        }
    });
});
} else if (graphMode === 'tags') {
    activeNotes.forEach((sourceNote, i) => {
        activeNotes.forEach((targetNote, j) => {
            if (i < j) {
                const sharedTags = sourceNote.tags ? sourceNote.tags.split(',').map(t=>t.trim()
                if (sharedTags.length > 0) {
                    const p1 = nodePositions[sourceNote.id];
                    const p2 = nodePositions[targetNote.id];
                    if (p1 && p2) {
                        ctx.beginPath(); ctx.moveTo(p1.x, p1.y); ctx.lineTo(p2.x, p2.y);
                        ctx.strokeStyle = "rgba(167, 139, 250, 0.35)"; ctx.stroke();
                    }
                }
            }
        });
    });
}

activeNotes.forEach((note) => {
    const pos = nodePositions[note.id];
    const isActive = note.id === activeNoteId;

    ctx.beginPath();
    ctx.arc(pos.x, pos.y, isActive ? 6 : 4, 0, Math.PI * 2);
    ctx.fillStyle = isActive ? "#a78bfa" : "#7c3aed";
    ctx.fill();

    ctx.font = isActive ? "bold 10px sans-serif" : "9px sans-serif";
    ctx.fillStyle = isActive ? "ffffff" : "#8e8e93";
    ctx.fillText(note.title.length > 14 ? note.title.substring(0, 12) + '...' : note.title, po
});

ctx.restore();
}

function escapeHtml(str) { return str.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;")
</script>
</body>
</html>

```

User prompt: add it with the name and with this feature; I want the app to have templates for the tags: Short Form, Long Form, Project, Business, Company, List of Ideas, Undecided, Music, Artwork, Random Idea, Documentary, Event, Notes, Journal Entry, Off Frame, Saevin ,Go Outside Project .

Response: Here is how the setup will be structured with your custom tags:

- **App / Workspace Name:** Saevin Dashboard

- **Tag Templates Included:**

- Short Form
- Long Form
- Project
- Business
- Company
- List of Ideas
- Undecided
- Music
- Artwork
- Random Idea
- Documentary
- Event

- Notes
- Journal Entry
- Off Frame
- Saevin
- Go Outside Project

User prompt: ok make the html code for the app

```
Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Saevin Dashboard</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700&display=sw
  <style>
    body { font-family: 'Plus Jakarta Sans', sans-serif; }
  </style>
</head>
<body class="bg-zinc-950 text-zinc-100 min-h-screen flex flex-col">

  <!-- Header -->
  <header class="border-b border-zinc-800 bg-zinc-900/50 backdrop-blur sticky top-0 z-50">
    <div class="max-w-6xl mx-auto px-6 h-16 flex items-center justify-between">
      <div class="flex items-center space-x-3">
        <div class="w-8 h-8 rounded-lg bg-zinc-100 text-zinc-950 flex items-center justify-center">
          <span class="font-bold text-lg tracking-tight">Saevin Dashboard</span>
        </div>
        <button onclick="openModal()" class="bg-zinc-100 text-zinc-950 px-4 py-2 rounded-lg font-semib
          + New Item
        </button>
      </div>
    </div>
  </header>

  <!-- Main Content -->
  <main class="max-w-6xl mx-auto px-6 py-8 flex-1 w-full">
    <!-- Filter Bar -->
    <div class="flex items-center justify-between mb-6 flex-wrap gap-4">
      <h1 class="text-2xl font-bold tracking-tight">Overview</h1>
      <div class="flex items-center space-x-2 overflow-x-auto pb-2 sm:pb-0 w-full sm:w-auto">
        <button onclick="filterTag('All')" class="tag-filter px-3 py-1.5 rounded-lg text-xs font-s
        <button onclick="filterTag('Short Form')" class="tag-filter px-3 py-1.5 rounded-lg text-xs
        <button onclick="filterTag('Music')" class="tag-filter px-3 py-1.5 rounded-lg text-xs font
        <button onclick="filterTag('Go Outside Project')" class="tag-filter px-3 py-1.5 rounded-lg
      </div>
    </div>

    <!-- Grid Container -->
    <div id="itemGrid" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
      <!-- Dynamic Cards Inserted Here -->
    </div>
  </main>

  <!-- Modal Form -->
  <div id="itemModal" class="fixed inset-0 bg-black/70 backdrop-blur-sm hidden items-center justify-cent
    <div class="bg-zinc-900 border border-zinc-800 rounded-2xl max-w-md w-full p-6 shadow-2xl">
      <h2 class="text-xl font-bold mb-4">Create New Entry</h2>
      <form id="entryForm" onsubmit="handleFormSubmit(event)" class="space-y-4">
        <div>
          <label class="block text-xs font-semibold text-zinc-400 uppercase tracking-wider mb-2">
            <input type="text" id="itemTitle" required class="w-full bg-zinc-950 border border-zin
          </div>
        <div>
          <label class="block text-xs font-semibold text-zinc-400 uppercase tracking-wider mb-2">
            <select id="itemTag" class="w-full bg-zinc-950 border border-zinc-800 rounded-lg px-4
              <option value="Short Form">Short Form</option>
              <option value="Long Form">Long Form</option>
              <option value="Project">Project</option>
              <option value="Business">Business</option>
              <option value="Company">Company</option>
              <option value="List of Ideas">List of Ideas</option>
              <option value="Undecided">Undecided</option>
              <option value="Music">Music</option>
            </select>
          </label>
        </div>
      </form>
    </div>
  </div>
```

```

        <option value="Artwork">Artwork</option>
        <option value="Random Idea">Random Idea</option>
        <option value="Documentary">Documentary</option>
        <option value="Event">Event</option>
        <option value="Notes">Notes</option>
        <option value="Journal Entry">Journal Entry</option>
        <option value="Off Frame">Off Frame</option>
        <option value="Saevin">Saevin</option>
        <option value="Go Outside Project">Go Outside Project</option>
    </select>
</div>
<div>
    <label class="block text-xs font-semibold text-zinc-400 uppercase tracking-wider mb-2">
        <textarea id="itemDesc" rows="3" class="w-full bg-zinc-950 border border-zinc-800 roun
    </div>
    <div class="flex justify-end space-x-3 pt-2">
        <button type="button" onclick="closeModal()" class="px-4 py-2 rounded-lg text-sm font-
        <button type="submit" class="px-4 py-2 rounded-lg text-sm font-semibold bg-zinc-100 te
    </div>
</form>
</div>
</div>

<!-- Script Logic -->
<script>
    let items = [
        { id: 1, title: 'New Album Concept Beats', tag: 'Music', desc: 'Working on instrumental sketch
        { id: 2, title: 'Lifestyle Vlog Cut 04', tag: 'Short Form', desc: 'Editing sequence for weeken
        { id: 3, title: 'Nature Connection Strategy', tag: 'Go Outside Project', desc: 'Outlining crea
    ];

    let currentFilter = 'All';

    function renderItems() {
        const grid = document.getElementById('itemGrid');
        grid.innerHTML = '';

        const filtered = currentFilter === 'All' ? items : items.filter(i => i.tag === currentFilter);

        if (filtered.length === 0) {
            grid.innerHTML = `<div class="col-span-full py-12 text-center text-zinc-500 text-sm">No it
            return;
        }

        filtered.forEach(item => {
            const card = document.createElement('div');
            card.className = 'bg-zinc-900 border border-zinc-800/80 rounded-2xl p-5 flex flex-col just
            card.innerHTML = `
                <div>
                    <div class="flex items-center justify-between mb-3">
                        <span class="px-2.5 py-1 rounded-md text-[11px] font-semibold bg-zinc-800 text
                        <button onclick="deleteItem(${item.id})" class="text-zinc-600 hover:text-red-4
                    </div>
                    <h3 class="font-bold text-base mb-1 text-zinc-100 group-hover:text-white">${item.t
                    <p class="text-zinc-400 text-xs leading-relaxed">${item.desc}</p>
                </div>
            `;
            grid.appendChild(card);
        });
    }

    function filterTag(tag) {
        currentFilter = tag;
        document.querySelectorAll('.tag-filter').forEach(btn => {
            if (btn.innerText === tag) {
                btn.className = 'tag-filter px-3 py-1.5 rounded-lg text-xs font-semibold bg-zinc-100 t
            } else {
                btn.className = 'tag-filter px-3 py-1.5 rounded-lg text-xs font-semibold bg-zinc-900 t
            }
        });
        renderItems();
    }

    function openModal() {
        document.getElementById('itemModal').classList.remove('hidden');
        document.getElementById('itemModal').classList.add('flex');
    }

    function closeModal() {

```

```

        document.getElementById('itemModal').classList.remove('flex');
        document.getElementById('itemModal').classList.add('hidden');
        document.getElementById('entryForm').reset();
    }

    function handleFormSubmit(e) {
        e.preventDefault();
        const title = document.getElementById('itemTitle').value;
        const tag = document.getElementById('itemTag').value;
        const desc = document.getElementById('itemDesc').value;

        const newItem = {
            id: Date.now(),
            title,
            tag,
            desc
        };

        items.unshift(newItem);
        renderItems();
        closeModal();
    }

    function deleteItem(id) {
        items = items.filter(i => i.id !== id);
        renderItems();
    }

    // Initial render
    renderItems();
</script>
</body>
</html>

```

User prompt: make it look exactly like this

```

Response: <!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vaukt</title>
  <!-- JSZip & FileSaver libraries for zip export -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/jszip/3.10.1/jszip.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/FileSaver.js/2.0.5/FileSaver.min.js"></script>
  <!-- FontAwesome Icons -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <style>
    :root {
      --bg-titlebar: #161618;
      --bg-ribbon: #1a1a1c;
      --bg-sidebar: #1e1e20;
      --bg-editor: #121214;
      --bg-modal: #1e1e20;
      --bg-modal-nav: #161618;
      --bg-card: #26262a;
      --bg-hover: #2a2a2e;
      --bg-active: #323238;
      --border-color: #28282c;

      --text-normal: #dcdde;
      --text-muted: #8e8e93;
      --text-faint: #5a5a5e;

      --accent-purple: #7c3aed;
      --accent-purple-light: #a78bfa;
      --accent-purple-glow: rgba(124, 58, 237, 0.2);
      --tag-bg: rgba(167, 139, 250, 0.12);

      --font-ui: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      --font-mono: "SF Mono", "Source Code Pro", Menlo, Monaco, Consolas, monospace;
    }

    body.light-theme {
      --bg-titlebar: #e5e5e7;
      --bg-ribbon: #ededf0;
      --bg-sidebar: #f4f4f6;
    }
  </style>

```

```

--bg-editor: #ffffff;
--bg-modal: #f4f4f6;
--bg-modal-nav: #e5e5e7;
--bg-card: #e8e8ec;
--bg-hover: #ddddf0;
--bg-active: #d0d0e0;
--border-color: #d1d1d6;

--text-normal: #1c1c1e;
--text-muted: #636366;
--text-faint: #8e8e93;

--accent-purple: #6d28d9;
--accent-purple-light: #7c3aed;
--accent-purple-glow: rgba(109, 40, 217, 0.15);
--tag-bg: rgba(109, 40, 217, 0.08);
}

* { box-sizing: border-box; margin: 0; padding: 0; user-select: none; }
body { background-color: var(--bg-titlebar); color: var(--text-normal); height: 100vh; display: fl

.window-header { height: 38px; background: var(--bg-titlebar); border-bottom: 1px solid var(--bord
.window-controls { display: flex; gap: 8px; }
.mac-btn { width: 12px; height: 12px; border-radius: 50%; }
.mac-close { background: #ff5f56; }
.mac-min { background: #ffbd2e; }
.mac-max { background: #27c93f; }
.window-title { color: var(--text-muted); font-size: 12px; font-family: var(--font-mono); display:

.app-container { flex: 1; display: flex; overflow: hidden; }

.ribbon { width: 44px; background: var(--bg-ribbon); border-right: 1px solid var(--border-color);
.ribbon-btn { width: 32px; height: 32px; border-radius: 6px; display: flex; align-items: center; j
.ribbon-btn:hover { background: var(--bg-hover); color: var(--text-normal); }
.ribbon-btn.active { color: var(--accent-purple-light); background: var(--accent-purple-glow); }
.ribbon-bottom { margin-top: auto; display: flex; flex-direction: column; gap: 10px; }

.sidebar { width: 300px; background: var(--bg-sidebar); border-right: 1px solid var(--border-color
.sidebar-header { padding: 12px 14px 8px 14px; display: flex; justify-content: space-between; alig
.vault-name { font-weight: 600; font-family: var(--font-mono); text-transform: uppercase; letter-s
.sidebar-actions { display: flex; gap: 8px; color: var(--text-muted); }
.sidebar-actions i { cursor: pointer; padding: 4px; border-radius: 4px; }
.sidebar-actions i:hover { background: var(--bg-hover); color: var(--text-normal); }

.search-container { padding: 0 10px 8px 10px; }
.search-trigger-input { width: 100%; background: var(--bg-ribbon); border: 1px solid var(--border-

.view-tab-selector { display: flex; margin: 0 10px 8px 10px; background: var(--bg-ribbon); border:
.view-tab-btn { flex: 1; border: none; background: transparent; color: var(--text-muted); font-siz
.view-tab-btn.active { background: var(--bg-card); color: var(--text-normal); font-weight: 600; }
.view-tab-btn.drag-over { background: rgba(255, 95, 86, 0.25) !important; color: #ff5f56 !importan

.notebook-navigator-list { flex: 1; overflow-y: auto; padding: 6px 8px; display: flex; flex-direct
.note-card { padding: 10px; border-radius: 6px; background: transparent; cursor: grab; display: fl
.note-card.active { cursor: grabbing; }
.note-card:hover { background: var(--bg-hover); }
.note-card.active { background: var(--bg-active); border-color: var(--border-color); }
.note-card.selected { background: var(--accent-purple-glow); border-color: var(--accent-purple-lig
.note-card.title { font-weight: 600; font-size: 12px; color: var(--text-normal); white-space: nowr
.note-card-snippet { font-size: 11px; color: var(--text-muted); display: -webkit-box; -webkit-line
.note-card-meta { font-family: var(--font-mono); font-size: 10px; color: var(--text-faint); displa
.btn-card-bin { position: absolute; right: 8px; top: 10px; color: var(--text-faint); opacity: 0; t
.note-card:hover .btn-card-bin { opacity: 1; }
.btn-card-bin:hover { color: #ff5f56; background: var(--bg-card); }

#dragBadge { position: absolute; top: -100px; left: -100px; background: var(--accent-purple); co

.selection-bar { padding: 6px 10px; background: var(--accent-purple-glow); border-bottom: 1px soli
.selection-bar button { background: var(--bg-card); border: 1px solid var(--border-color); color:

.tags-pane { border-top: 1px solid var(--border-color); padding: 8px 10px; max-height: 130px; over
.tag-pill-list { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 4px; }
.tag-pill { background: var(--tag-bg); color: var(--accent-purple-light); padding: 2px 6px; border
.tag-pill.active { background: var(--accent-purple); color: #fff; }

.workspace { flex: 1; display: flex; flex-direction: column; background: var(--bg-editor); overflo

/* Notion-style Tabs Container */
.tab-bar { height: 36px; background: var(--bg-sidebar); border-bottom: 1px solid var(--border-colo

```

```

.tab-left { display: flex; align-items: flex-end; gap: 2px; overflow-x: auto; flex: 1; padding-top

/* Individual Tab */
.tab { height: 30px; padding: 0 10px; background: var(--bg-ribbon); border-radius: 6px 6px 0 0; di
.tab.active { background: var(--bg-editor); color: var(--text-normal); border-bottom: 1px solid va
.tab:hover:not(.active) { background: var(--bg-hover); }
.tab-title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; flex: 1; pointer-even
.tab-close { opacity: 0; padding: 2px 4px; border-radius: 4px; font-size: 10px; color: var(--text-
.tab:hover .tab-close, .tab.active .tab-close { opacity: 1; }
.tab-close:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-new-btn { cursor: pointer; padding: 4px 8px; margin-left: 4px; margin-bottom: 2px; color: var
.tab-new-btn:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-actions { display: flex; gap: 8px; align-items: center; margin-left: 12px; }
.editor-action-btn { background: var(--bg-card); border: 1px solid var(--border-color); color: var
.editor-action-btn:hover, .editor-action-btn.active { color: var(--text-normal); background: var(-

.workspace-content { flex: 1; display: flex; overflow: hidden; position: relative; }

/* Empty State */
.empty-state { display: none; flex: 1; align-items: center; justify-content: center; flex-directio
.empty-state i { font-size: 48px; margin-bottom: 16px; opacity: 0.2; }

.editor-pane { flex: 1; padding: 28px 42px; overflow-y: auto; display: flex; flex-direction: colum
.properties-block { background: rgba(255, 255, 255, 0.02); border: 1px solid var(--border-color);
.property-row { display: grid; grid-template-columns: 90px 1fr; align-items: center; gap: 10px; fo
.property-label { color: var(--text-muted); display: flex; align-items: center; gap: 6px; }
.property-input { background: transparent; border: none; color: var(--text-normal); font-family: v
input[type="date"].property-input { color-scheme: dark; cursor: pointer; }
body.light-theme input[type="date"].property-input { color-scheme: light; }

.note-title-input { font-size: 1.8rem; font-weight: 700; color: var(--text-normal); background: tr
.note-body-textarea { flex: 1; background: transparent; border: none; outline: none; color: var(--

/* Tag Template Bar */
.template-tags-bar { display: flex; flex-wrap: wrap; gap: 6px; padding: 6px 0; border-top: 1px das
.template-tag-chip { font-size: 10px; font-family: var(--font-mono); background: var(--bg-card); b
.template-tag-chip:hover { background: var(--bg-hover); color: var(--text-normal); border-color: v

/* Graph Pane */
.graph-pane { width: 340px; background: var(--bg-ribbon); border-left: 1px solid var(--border-colo
.graph-pane.collapsed { display: none; }
.graph-header { padding: 8px 12px; border-bottom: 1px solid var(--border-color); display: flex; al
.graph-mode-selector { display: flex; background: var(--bg-card); border: 1px solid var(--border-c
.graph-mode-btn { border: none; background: transparent; color: var(--text-muted); font-size: 10px
.graph-mode-btn.active { background: var(--accent-purple); color: #ffffff; font-weight: 600; }
.graph-canvas-container { flex: 1; position: relative; width: 100%; height: 100%; overflow: hidden
.graph-canvas-container.active { cursor: grabbing; }
#graphCanvas { width: 100%; height: 100%; display: block; }
.graph-zoom-controls { position: absolute; bottom: 12px; right: 12px; display: flex; flex-directio
.graph-zoom-btn { width: 24px; height: 24px; background: transparent; border: none; color: var(--t
.graph-zoom-btn:hover { background: var(--bg-hover); color: var(--accent-purple-light); }
.graph-tag-checkboxes { padding: 10px 12px; border-top: 1px solid var(--border-color); background:
.graph-tag-checkboxes.active { display: flex; }
.tag-checkbox-header { font-size: 10px; font-weight: 700; color: var(--text-faint); text-transform
.tag-checkbox-item { display: flex; align-items: center; gap: 8px; color: var(--text-muted); curso
.tag-checkbox-item:hover { color: var(--text-normal); }
.tag-checkbox-item input[type="checkbox"] { accent-color: var(--accent-purple); cursor: pointer; }

.calendar-pane { padding: 12px; display: flex; flex-direction: column; gap: 12px; font-family: var
.calendar-header { display: flex; justify-content: space-between; align-items: center; font-weight
.calendar-grid { display: grid; grid-template-columns: repeat(7, 1fr); gap: 4px; text-align: cente
.cal-day-label { font-size: 10px; color: var(--text-faint); padding-bottom: 4px; }
.cal-cell { font-size: 11px; padding: 6px 0; border-radius: 4px; cursor: pointer; color: var(--tex
.cal-cell:hover { background: var(--bg-hover); color: var(--text-normal); }
.cal-cell.today { border: 1px solid var(--accent-purple-light); font-weight: bold; color: var(--te
.cal-cell.has-note::after { content: ''; position: absolute; bottom: 2px; left: 50%; transform: tr

/* Modals & Menus */
.modal-overlay { position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; background: rgba(0
.modal-overlay.active { display: flex; }
.obsidian-settings-card { width: 840px; height: 560px; max-width: 95vw; max-height: 90vh; backgrou
.settings-nav { width: 220px; background: var(--bg-modal-nav); border-right: 1px solid var(--borde
.settings-nav-section { display: flex; flex-direction: column; gap: 2px; }
.settings-nav-header { font-size: 10px; font-weight: 700; color: var(--text-faint); padding: 4px 8
.settings-nav-item { padding: 6px 10px; border-radius: 4px; color: var(--text-muted); cursor: poin
.settings-nav-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.settings-nav-item.active { background: var(--bg-active); color: var(--text-normal); font-weight:

```

```

.settings-content { flex: 1; padding: 24px 32px; overflow-y: auto; display: flex; flex-direction:
.settings-section-title { font-size: 16px; font-weight: 700; color: var(--text-normal); border-bot
.obs-setting-item { display: flex; justify-content: space-between; align-items: flex-start; paddin
.obs-setting-info { display: flex; flex-direction: column; gap: 4px; max-width: 380px; }
.obs-setting-name { font-size: 12px; font-weight: 600; color: var(--text-normal); }
.obs-setting-desc { font-size: 11px; color: var(--text-muted); line-height: 1.4; }
.obs-setting-control { display: flex; align-items: center; gap: 8px; }
select.obs-select { background: var(--bg-card); border: 1px solid var(--border-color); color: var(
.obs-btn { background: var(--bg-card); border: 1px solid var(--border-color); color: var(--text-no
.obs-btn:hover { background: var(--bg-hover); }
.obs-btn-danger { background: #ff5f56; color: #fff; border: none; }

.search-modal { width: 640px; max-width: 90vw; background: var(--bg-sidebar); border: 1px solid va
.modal-search-header { padding: 14px; border-bottom: 1px solid var(--border-color); display: flex;
.modal-search-input { flex: 1; background: transparent; border: none; outline: none; color: var(--
.modal-search-body { display: flex; height: 340px; }
.modal-results-list { flex: 1; overflow-y: auto; padding: 8px; border-right: 1px solid var(--borde
.modal-result-item { padding: 8px 10px; border-radius: 6px; cursor: pointer; display: flex; align-
.modal-result-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.modal-preview-pane { width: 280px; background: var(--bg-editor); padding: 14px; overflow-y: auto;

.slash-menu { position: absolute; background: var(--bg-card); border: 1px solid var(--border-color
.slash-item { padding: 8px 12px; display: flex; align-items: center; gap: 8px; color: var(--text-n
.slash-item:hover { background: var(--accent-purple); color: #ffffff; }
</style>
</head>
<body>

<input type="file" id="folderImporter" webkitdirectory directory multiple style="display: none;" oncha
<input type="file" id="icsImporter" accept=".ics" style="display: none;" onchange="handleImportICS(eve
<div id="dragBadge">1 Note Selected</div>

<header class="window-header">
  <div class="window-controls">
    <div class="mac-btn mac-close"></div>
    <div class="mac-btn mac-min"></div>
    <div class="mac-btn mac-max"></div>
  </div>
  <div class="window-title"><i class="fa-solid fa-vault"></i> Vaukt Studio</div>
  <div></div>
</header>

<div class="app-container">
  <!-- Ribbon -->
  <nav class="ribbon">
    <div class="ribbon-btn active" id="ribbonNavBtn" title="Notebook Navigator" onclick="switchSid
    <div class="ribbon-btn" id="ribbonCalBtn" title="Apple Calendar Sync" onclick="switchSidebarVi
    <div class="ribbon-btn" title="Quick Search (Shift x2)" onclick="openSearchModal()"><i class="
    <div class="ribbon-btn active" id="ribbonGraphBtn" title="Toggle Graph View" onclick="toggleGr
    <div class="ribbon-btn" title="New Note" onclick="createNewNote()"><i class="fa-solid fa-plus"
    <div class="ribbon-bottom">
      <div class="ribbon-btn" title="Settings" onclick="openSettingsModal()"><i class="fa-solid
    </div>
  </nav>

  <!-- Sidebar Pane -->
  <aside class="sidebar">
    <div class="sidebar-header">
      <span class="vault-name" id="sidebarTitle">Notebook Navigator</span>
      <div class="sidebar-actions">
        <i class="fa-solid fa-file-import" title="Import Folder" onclick="triggerFolderImport(
        <i class="fa-solid fa-file-circle-plus" title="New Note" onclick="createNewNote()"></i>
      </div>
    </div>

    <div class="view-tab-selector">
      <button class="view-tab-btn active" id="tabNotesBtn" onclick="switchSidebarView('notes')">
      <button class="view-tab-btn" id="tabBinBtn" onclick="switchSidebarView('bin')">
        <ondragover="handleDragOverBin(event)"
        <ondragleave="handleDragLeaveBin(event)"
        <ondrop="handleDropToBin(event)">
        <i class="fa-solid fa-trash-can"></i> Bin
      </button>
    </div>

    <!-- Multi-Selection Bar -->
    <div class="selection-bar" id="selectionBar" style="display: none;">
      <span id="selectedCountText">0 selected</span>
      <div style="display: flex; gap: 6px;">

```

```

        <button onclick="binSelectedNotes()"><i class="fa-solid fa-trash-can"></i> Move to Bin
        <button onclick="clearSelection()">Clear</button>
    </div>
</div>

<div class="search-container" id="searchBoxContainer">
    <div class="search-trigger-input" onclick="openSearchModal()">
        <span><i class="fa-solid fa-magnifying-glass" style="margin-right:6px;"></i> Quick ope
        <span class="search-shortcut-badge">Shift x2</span>
    </div>
</div>

<!-- Note List -->
<div class="notebook-navigator-list" id="navigatorList"></div>

<!-- Calendar View -->
<div class="calendar-pane" id="calendarView" style="display: none;">
    <div class="calendar-header">
        <span id="calMonthYear">August 2026</span>
        <div style="display:flex; gap:6px;">
            <i class="fa-solid fa-chevron-left" style="cursor:pointer;" onclick="changeMonth(-
            <i class="fa-solid fa-chevron-right" style="cursor:pointer;" onclick="changeMonth(
        </div>
    </div>
    <div class="calendar-grid" id="calendarGrid"></div>

    <div style="margin-top:12px; border-top:1px solid var(--border-color); padding-top:12px;">
        <div style="font-size:10px; font-weight:700; color:var(--text-faint); margin-bottom:8p
        <button class="obs-btn" style="width:100%; margin-bottom:6px;" onclick="triggerICSImpo
        <button class="obs-btn" style="width:100%;" onclick="exportCalendarICS()"><i class="fa
    </div>
</div>

<!-- Tags Drawer -->
<div class="tags-pane" id="tagsPane">
    <div style="font-size:10px; font-weight:700; font-family:var(--font-mono); color:var(--tex
    <div class="tag-pill-list" id="tagPillList"></div>
</div>
</aside>

<!-- Workspace -->
<main class="workspace">
    <!-- NOTION STYLE TAB BAR -->
    <div class="tab-bar">
        <div class="tab-left" id="tabContainer">
            <!-- Tabs are injected dynamically here -->
        </div>
        <div class="tab-new-btn" title="Open new tab" onclick="createNewNote()"><i class="fa-solid

    <div class="tab-actions">
        <button class="editor-action-btn active" id="graphToggleBtn" title="Toggle Graph" oncl
            <i class="fa-solid fa-circle-nodes"></i> Graph
        </button>
        <button class="editor-action-btn" title="Export this note as .md" onclick="downloadCur
            <i class="fa-solid fa-download"></i> Export .md
        </button>
        <button class="editor-action-btn" title="Move note to Bin" onclick="moveActiveNoteToBi
            <i class="fa-solid fa-trash-can"></i> Bin Note
        </button>
    </div>
</div>

<div class="workspace-content">
    <!-- EMPTY STATE (Shown when all tabs are closed) -->
    <div class="empty-state" id="emptyState">
        <i class="fa-solid fa-leaf"></i>
        <div>No file is open</div>
        <div style="margin-top: 8px; font-size: 11px;">Create a new note or select one from th
    </div>

    <!-- EDITOR PANE -->
    <div class="editor-pane" id="editorPane">
        <input type="text" id="noteTitle" class="note-title-input" placeholder="Note Title..."

        <div class="properties-block">
            <div class="property-row">
                <span class="property-label"><i class="fa-regular fa-calendar-days"></i> creat
                <input type="date" id="propCreated" class="property-input" onchange="updateCur
            </div>

```

```

        <div class="property-row">
            <span class="property-label"><i class="fa-solid fa-tag"></i> tags</span>
            <input type="text" id="propTags" class="property-input" placeholder="tags sepa
        </div>
    </div>

    <!-- Template Tag Quick-Insert Bar -->
    <div class="template-tags-bar" id="templateTagsBar">
        <span style="font-size:10px; font-family:var(--font-mono); color:var(--text-faint)
    </div>

    <textarea id="noteContent" class="note-body-textarea" placeholder="Type note content..

    <div id="slashMenu" class="slash-menu">
        <div style="padding:6px 10px; font-size:10px; font-weight:700; font-family:var(--f
        <div id="slashList"></div>
    </div>
</div>

<!-- Canvas Graph Pane -->
<div class="graph-pane" id="graphPane">
    <div class="graph-header">
        <span><i class="fa-solid fa-diagram-project" style="margin-right:4px;"></i> GRAPH
        <div class="graph-mode-selector">
            <button class="graph-mode-btn active" id="btnModeLinks" onclick="setGraphMode(
            <button class="graph-mode-btn" id="btnModeTags" onclick="setGraphMode('tags')
        </div>
        <i class="fa-solid fa-xmark" style="cursor:pointer;" onclick="toggleGraphPane()"><
    </div>

    <div class="graph-canvas-container" id="graphCanvasContainer">
        <canvas id="graphCanvas"></canvas>
        <div class="graph-zoom-controls">
            <button class="graph-zoom-btn" title="Zoom In" onclick="adjustGraphZoom(1.2)">
            <button class="graph-zoom-btn" title="Reset Zoom" onclick="resetGraphZoom()"><
            <button class="graph-zoom-btn" title="Zoom Out" onclick="adjustGraphZoom(0.8)"
        </div>
    </div>

    <div class="graph-tag-checkboxes" id="graphTagCheckboxes">
        <div class="tag-checkbox-header">
            <span>Filter Visible Tags</span>
            <span style="cursor:pointer; color:var(--accent-purple-light);" onclick="selec
        </div>
        <div id="tagCheckboxList"></div>
    </div>
</div>
</div>
</main>
</div>

<!-- Authentic Obsidian Settings Modal -->
<div class="modal-overlay" id="settingsModalOverlay" onclick="closeSettingsModal(event)">
    <div class="obsidian-settings-card" onclick="event.stopPropagation()">
        <div class="settings-nav">
            <div class="settings-nav-section">
                <div class="settings-nav-header">Options</div>
                <div class="settings-nav-item">General</div>
                <div class="settings-nav-item">Editor</div>
                <div class="settings-nav-item">Files and links</div>
                <div class="settings-nav-item active">Appearance</div>
            </div>
        </div>

        <div class="settings-content">
            <div class="settings-section-title">Appearance</div>
            <div class="obs-setting-item">
                <div class="obs-setting-info">
                    <div class="obs-setting-name">Base color scheme</div>
                    <div class="obs-setting-desc">Choose Obsidian's default color scheme.</div>
                </div>
                <div class="obs-setting-control">
                    <select class="obs-select" id="themeSelect" onchange="changeThemeFromSelect()">
                        <option value="dark">Dark</option>
                        <option value="light">Light</option>
                    </select>
                </div>
            </div>
            <div class="obs-setting-item">

```

```

        <div class="obs-setting-info">
            <div class="obs-setting-name">Vault Storage Backup</div>
            <div class="obs-setting-desc">Export all notes into a ZIP folder archive.</div>
        </div>
        <div class="obs-setting-control">
            <button class="obs-btn" onclick="exportAllNotesZip()"><i class="fa-solid fa-file-z
        </div>
    </div>
    <div class="obs-setting-item">
        <div class="obs-setting-info">
            <div class="obs-setting-name" style="color:#ff5f56;">Reset Vault</div>
            <div class="obs-setting-desc">Exports all notes as a .md ZIP folder before permane
        </div>
        <div class="obs-setting-control">
            <button class="obs-btn obs-btn-danger" onclick="resetVaultWithBackup()"><i class="
        </div>
    </div>
</div>
</div>
</div>
<!-- Search Modal -->
<div class="modal-overlay" id="searchModalOverlay" onclick="closeSearchModal(event)">
    <div class="search-modal" onclick="event.stopPropagation()">
        <div class="modal-search-header">
            <i class="fa-solid fa-magnifying-glass" style="color:var(--text-faint);"></i>
            <input type="text" id="modalSearchInput" class="modal-search-input" placeholder="Type note
        </div>
        <div class="modal-search-body">
            <div class="modal-results-list" id="modalResultsList"></div>
            <div class="modal-preview-pane">
                <div id="modalPreviewTitle" style="font-weight:700; font-size:12px; color:var(--text-n
                <div id="modalPreviewSnippet"></div>
            </div>
        </div>
    </div>
</div>
</div>
</div>
<script>
    function getSystemTodayDate() {
        const today = new Date();
        const yyyy = today.getFullYear();
        const mm = String(today.getMonth() + 1).padStart(2, '0');
        const dd = String(today.getDate()).padStart(2, '0');
        return `${yyyy}-${mm}-${dd}`;
    }

    const templateTagsList = [
        "Short Form",
        "Long Form",
        "Project",
        "Business",
        "Company",
        "List Of Ideas",
        "Undecided",
        "Music",
        ":: Artwork",
        ":: Random Idea",
        ":: Documentary",
        ":: Event",
        ":: Notes",
        "Journal Entry",
        "Off Frame",
        "Saevin Go Outside Project"
    ];

    let notes = JSON.parse(localStorage.getItem('vaukt_minimal_notes')) || [
        { id: 1, title: '2026-8-5 Building Out the team', content: 'Ok what I want is that I need peop
        { id: 2, title: 'Go Outside Marketing Strategy', content: 'Brainstorming community events and
        { id: 3, title: 'Project Timeline & Milestones', content: 'Documenting content production sche
    ];

    let activeNoteId = notes.find(n => !n.isBinned) ? notes.find(n => !n.isBinned).id : null;
    let openTabs = activeNoteId ? [activeNoteId] : [];
    let selectedNoteIds = new Set(activeNoteId ? [activeNoteId] : []);
    let lastSelectedId = activeNoteId;

    let activeSidebarView = 'notes';
    let activeTagFilter = null;

```

```

let lastShiftTime = 0;
let graphMode = 'links';
let isGraphOpen = true;

let graphZoom = 1;
let graphPanX = 0;
let graphPanY = 0;
let isPanningGraph = false;
let panStartX = 0;
let panStartY = 0;
let selectedGraphTags = new Set();

let calCurrentDate = new Date();
let slashActive = false;
let slashStartPos = -1;
let filteredSlashNotes = [];

window.addEventListener('DOMContentLoaded', () => {
  const savedTheme = localStorage.getItem('vaukt_theme');
  if (savedTheme === 'light') {
    document.body.classList.add('light-theme');
    document.getElementById('themeSelect').value = 'light';
  }

  renderTemplateTagsBar();
  renderNavigatorList();
  renderTags();
  if (activeNoteId) loadNote(activeNoteId);
  else renderEditorState();

  initGraph();
  renderCalendar();

  document.addEventListener('keydown', (e) => {
    if (e.key === 'Shift') {
      const currentTime = new Date().getTime();
      if (currentTime - lastShiftTime < 300) openSearchModal();
      lastShiftTime = currentTime;
    }
  });

  window.addEventListener('resize', () => { if (isGraphOpen) drawGraph(); });
});

function saveStorage() { localStorage.setItem('vaukt_minimal_notes', JSON.stringify(notes)); }

function renderTemplateTagsBar() {
  const bar = document.getElementById('templateTagsBar');
  const labelEl = bar.querySelector('span');
  bar.innerHTML = '';
  bar.appendChild(labelEl);

  templateTagsList.forEach(tagName => {
    const chip = document.createElement('div');
    chip.className = 'template-tag-chip';
    chip.innerHTML = `<i class="fa-solid fa-plus" style="font-size:9px;"></i> ${escapeHtml(tagName)}`;
    chip.onclick = () => insertTagIntoCurrentNote(tagName);
    bar.appendChild(chip);
  });
}

function insertTagIntoCurrentNote(tagName) {
  if (!activeNoteId) return;
  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;

  let currentTags = note.tags ? note.tags.split(',').map(t => t.trim()).filter(Boolean) : [];
  if (!currentTags.includes(tagName)) {
    currentTags.push(tagName);
    note.tags = currentTags.join(', ');
    document.getElementById('propTags').value = note.tags;
    updateCurrentNote();
  }
}

function renderTabs() {
  const container = document.getElementById('tabContainer');
  container.innerHTML = '';

```

```

openTabs.forEach(id => {
  const note = notes.find(n => n.id === id);
  if (!note) return;

  const isActive = id === activeNoteId;
  const tab = document.createElement('div');
  tab.className = `tab ${isActive ? 'active' : ''}`;
  tab.onclick = () => loadNote(id);

  const displayTitle = note.title.trim() === '' ? 'Untitled' : note.title;

  tab.innerHTML = `
    <i class="fa-solid fa-leaf" style="color: ${isActive ? 'var(--accent-purple-light)' : ''}>
    <span class="tab-title">${escapeHtml(displayTitle)}</span>
    <i class="fa-solid fa-xmark tab-close" onclick="event.stopPropagation(); closeTab(${id})"
  `;
  container.appendChild(tab);
});
}

function closeTab(id) {
  openTabs = openTabs.filter(t => t !== id);
  if (id === activeNoteId) {
    if (openTabs.length > 0) {
      loadNote(openTabs[openTabs.length - 1]);
    } else {
      activeNoteId = null;
      renderEditorState();
    }
  } else {
    renderTabs();
  }
}

function renderEditorState() {
  const editorPane = document.getElementById('editorPane');
  const emptyState = document.getElementById('emptyState');

  renderTabs();
  renderNavigatorList();

  if (!activeNoteId) {
    editorPane.style.display = 'none';
    emptyState.style.display = 'flex';
    if (isGraphOpen) drawGraph();
    return;
  }

  editorPane.style.display = 'flex';
  emptyState.style.display = 'none';

  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;

  document.getElementById('noteTitle').value = note.title;
  document.getElementById('noteContent').value = note.content;
  document.getElementById('propTags').value = note.tags || '';
  document.getElementById('propCreated').value = note.created || getSystemTodayDate();

  if (isGraphOpen) drawGraph();
}

function loadNote(id) {
  const note = notes.find(n => n.id === id);
  if (!note) return;

  if (!openTabs.includes(id)) {
    openTabs.push(id);
  }

  activeNoteId = id;
  selectedNoteIds.clear();
  selectedNoteIds.add(id);
  lastSelectedId = id;

  renderEditorState();
}

function createNewNote() {

```

```

    const newNote = { id: Date.now(), title: "", content: "", tags: "", created: getSystemTodayDate() };
    notes.unshift(newNote);
    saveStorage();

    switchSidebarView('notes');
    renderTags();
    loadNote(newNote.id);
    setTimeout(() => { document.getElementById('noteTitle').focus(); }, 50);
}

function updateCurrentNote() {
    if (!activeNoteId) return;
    const note = notes.find(n => n.id === activeNoteId);
    if (!note) return;

    note.title = document.getElementById('noteTitle').value;
    note.content = document.getElementById('noteContent').value;
    note.tags = document.getElementById('propTags').value;
    note.created = document.getElementById('propCreated').value || getSystemTodayDate();

    saveStorage();
    renderTabs();
    renderNavigatorList();
    renderTags();
    if (isGraphOpen) {
        renderTagCheckboxes();
        drawGraph();
    }
}

async function formatLinkAsMarkdown(url) {
    try {
        let title = "";
        if (url.includes("youtube.com") || url.includes("youtu.be")) {
            if (url.includes("07vGsBghWfc")) {
                title = "why I switched to using Obsidian (as a former Notion user)";
            } else {
                title = "YouTube Video";
            }
        } else {
            const domainObj = new URL(url);
            title = domainObj.hostname.replace("www.", "");
        }
        return `[${title}](${url})`;
    } catch (e) {
        return url;
    }
}

async function handleEditorPaste(e) {
    const clipboardData = e.clipboardData || window.clipboardData;
    const pastedText = clipboardData.getData('text');

    if (pastedText && (pastedText.startsWith('http://') || pastedText.startsWith('https://'))) {
        e.preventDefault();
        const textarea = e.target;
        const start = textarea.selectionStart;
        const end = textarea.selectionEnd;
        const val = textarea.value;

        const formattedLink = await formatLinkAsMarkdown(pastedText.trim());

        textarea.value = val.substring(0, start) + formattedLink + val.substring(end);
        textarea.selectionStart = textarea.selectionEnd = start + formattedLink.length;
        updateCurrentNote();
    }
}

async function handleEditorInput(e) {
    updateCurrentNote();
    const textarea = e.target;
    const pos = textarea.selectionStart;
    const val = textarea.value;

    if (val.slice(pos - 1, pos) === ' ' || val.slice(pos - 1, pos) === '\n') {
        const textBeforeCursor = val.substring(0, pos - 1);
        const words = textBeforeCursor.split(/\s+/);
        const lastWord = words[words.length - 1];
    }
}

```

```

        if (lastWord && (lastWord.startsWith('http://') || lastWord.startsWith('https://')) && !la
            const formattedLink = await formatLinkAsMarkdown(lastWord);
            const updatedBefore = textBeforeCursor.substring(0, textBeforeCursor.length - lastWord
            textarea.value = updatedBefore + formattedLink + val.substring(pos - 1);
            textarea.selectionStart = textarea.selectionEnd = updatedBefore.length + formattedLink
            updateCurrentNote();
        }
    }

    if (val.slice(pos - 1, pos) === '/') {
        slashActive = true;
        slashStartPos = pos - 1;
        document.getElementById('slashMenu').style.display = 'block';
        renderSlashList('');
    }
    if (slashActive) renderSlashList(val.substring(slashStartPos + 1, pos).toLowerCase());
}

function switchSidebarView(view) {
    activeSidebarView = view;
    document.getElementById('tabNotesBtn').classList.toggle('active', view === 'notes');
    document.getElementById('tabBinBtn').classList.toggle('active', view === 'bin');
    document.getElementById('ribbonNavBtn').classList.toggle('active', view === 'notes');
    document.getElementById('ribbonCalBtn').classList.toggle('active', view === 'calendar');

    const list = document.getElementById('navigatorList');
    const cal = document.getElementById('calendarView');
    const searchBox = document.getElementById('searchBoxContainer');
    const tagsPane = document.getElementById('tagsPane');

    if (view === 'calendar') {
        list.style.display = 'none';
        searchBox.style.display = 'none';
        tagsPane.style.display = 'none';
        cal.style.display = 'flex';
        document.getElementById('sidebarTitle').innerText = 'Calendar View';
        renderCalendar();
    } else {
        list.style.display = 'flex';
        searchBox.style.display = 'block';
        tagsPane.style.display = 'block';
        cal.style.display = 'none';
        document.getElementById('sidebarTitle').innerText = view === 'bin' ? 'Bin Trash' : 'Notebo
        renderNavigatorList();
    }
}

function handleNoteClick(e, noteId) {
    const visibleNotes = getVisibleNotes();
    if (e.metaKey || e.ctrlKey) {
        if (selectedNoteIds.has(noteId)) selectedNoteIds.delete(noteId);
        else selectedNoteIds.add(noteId);
        lastSelectedId = noteId;
    } else if (e.shiftKey && lastSelectedId !== null) {
        const idx1 = visibleNotes.findIndex(n => n.id === lastSelectedId);
        const idx2 = visibleNotes.findIndex(n => n.id === noteId);
        if (idx1 !== -1 && idx2 !== -1) {
            const start = Math.min(idx1, idx2);
            const end = Math.max(idx1, idx2);
            selectedNoteIds.clear();
            for (let i = start; i <= end; i++) selectedNoteIds.add(visibleNotes[i].id);
        }
    } else {
        selectedNoteIds.clear();
        selectedNoteIds.add(noteId);
        lastSelectedId = noteId;
        loadNote(noteId);
    }
    updateSelectionUI();
}

function updateSelectionUI() {
    const bar = document.getElementById('selectionBar');
    const text = document.getElementById('selectedCountText');
    if (selectedNoteIds.size > 1) {
        bar.style.display = 'flex';
        text.innerText = `${selectedNoteIds.size} notes selected`;
    } else {
        bar.style.display = 'none';
    }
}

```

```

    }
    renderNavigatorList();
}

function clearSelection() {
    selectedNoteIds.clear();
    if (activeNoteId) selectedNoteIds.add(activeNoteId);
    updateSelectionUI();
}

function handleDragStart(e, noteId) {
    if (!selectedNoteIds.has(noteId)) {
        selectedNoteIds.clear();
        selectedNoteIds.add(noteId);
        lastSelectedId = noteId;
        renderNavigatorList();
    }
    const idsArray = Array.from(selectedNoteIds);
    e.dataTransfer.setData('text/plain', JSON.stringify(idsArray));
    const badge = document.getElementById('dragBadge');
    badge.innerText = `${idsArray.length} ${idsArray.length === 1 ? 'Note' : 'Notes'}`;
    e.dataTransfer.setDragImage(badge, 0, 0);
}

function handleDragOverBin(e) { e.preventDefault(); document.getElementById('tabBinBtn').classList
function handleDragLeaveBin(e) { document.getElementById('tabBinBtn').classList.remove('drag-over'

function handleDropToBin(e) {
    e.preventDefault();
    document.getElementById('tabBinBtn').classList.remove('drag-over');
    const rawData = e.dataTransfer.getData('text/plain');
    if (!rawData) return;

    try {
        const idsToBin = JSON.parse(rawData);
        notes.forEach(n => { if (idsToBin.includes(n.id)) n.isBinned = true; });
        saveStorage();

        openTabs = openTabs.filter(id => !idsToBin.includes(id));
        if (idsToBin.includes(activeNoteId)) {
            activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
        }

        selectedNoteIds.clear();
        if (activeNoteId) selectedNoteIds.add(activeNoteId);

        renderEditorState();
        renderTags();
    } catch (err) { console.error(err); }
}

function binSelectedNotes() {
    notes.forEach(n => { if (selectedNoteIds.has(n.id)) n.isBinned = true; });
    saveStorage();

    const binnedIds = Array.from(selectedNoteIds);
    openTabs = openTabs.filter(id => !binnedIds.includes(id));
    if (binnedIds.includes(activeNoteId)) {
        activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
    }

    selectedNoteIds.clear();
    if (activeNoteId) selectedNoteIds.add(activeNoteId);

    renderEditorState();
    renderTags();
}

function getVisibleNotes() {
    const isBinView = activeSidebarView === 'bin';
    return notes.filter(n => {
        if (isBinView) return n.isBinned;
        if (n.isBinned) return false;
        return activeTagFilter ? n.tags.split(',').map(t=>t.trim()).includes(activeTagFilter) : tr
    });
}

function moveActiveNoteToBin() {
    if (!activeNoteId) return;

```

```

        selectedNoteIds.clear();
        selectedNoteIds.add(activeNoteId);
        binSelectedNotes();
    }

    function restoreNoteFromBin(id) {
        const note = notes.find(n => n.id === id);
        if (note) { note.isBinned = false; saveStorage(); renderNavigatorList(); }
    }

    function permanentlyDeleteNote(id) {
        if (confirm("Permanently delete this note?")) {
            notes = notes.filter(n => n.id !== id);
            selectedNoteIds.delete(id);
            openTabs = openTabs.filter(t => t !== id);
            if (activeNoteId === id) activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length-1]
            saveStorage();
            renderEditorState();
        }
    }

    function moveNoteIdToBin(id) {
        const note = notes.find(n => n.id === id);
        if (note) {
            note.isBinned = true;
            saveStorage();
            openTabs = openTabs.filter(t => t !== id);
            if (activeNoteId === id) activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length-1]
            renderEditorState();
        }
    }

    function renderCalendar() {
        const grid = document.getElementById('calendarGrid');
        grid.innerHTML = '';
        const year = calCurrentDate.getFullYear();
        const month = calCurrentDate.getMonth();
        const monthNames = ["January", "February", "March", "April", "May", "June", "July", "August",
        document.getElementById('calMonthYear').innerText = `${monthNames[month]} ${year}`;

        ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'].forEach(d => {
            const l = document.createElement('div');
            l.className = 'cal-day-label';
            l.innerText = d;
            grid.appendChild(l);
        });

        const firstDay = new Date(year, month, 1).getDay();
        const daysInMonth = new Date(year, month + 1, 0).getDate();
        for (let i = 0; i < firstDay; i++) grid.appendChild(document.createElement('div'));

        const todayStr = getSystemTodayDate();
        for (let day = 1; day <= daysInMonth; day++) {
            const cell = document.createElement('div');
            cell.className = 'cal-cell';
            const mm = String(month + 1).padStart(2, '0');
            const dd = String(day).padStart(2, '0');
            const dateStr = `${year}-${mm}-${dd}`;

            cell.innerText = day;
            if (dateStr === todayStr) cell.classList.add('today');
            if (notes.some(n => !n.isBinned && n.created === dateStr)) cell.classList.add('has-note');

            cell.onclick = () => {
                const matchedNote = notes.find(n => !n.isBinned && n.created === dateStr);
                if (matchedNote) {
                    switchSidebarView('notes');
                    loadNote(matchedNote.id);
                } else if (confirm(`Create new note for date ${dateStr}?`)) {
                    const newNote = { id: Date.now(), title: `Note ${dateStr}`, content: '', tags: 'Jo
                    notes.unshift(newNote);
                    saveStorage();
                    switchSidebarView('notes');
                    loadNote(newNote.id);
                }
            };
            grid.appendChild(cell);
        }
    }
}

```

```

function changeMonth(dir) { calCurrentDate.setMonth(calCurrentDate.getMonth() + dir); renderCalend
function triggerICSImport() { document.getElementById('icsImporter').click(); }

function handleImportICS(e) {
  const file = e.target.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = function(evt) {
    const text = evt.target.result;
    const summaryMatch = text.match(/SUMMARY:(.*)/);
    const title = summaryMatch ? summaryMatch[1].trim() : "Imported Event";
    const newNote = { id: Date.now(), title: title, content: "Imported from Apple Calendar\n\n"
    notes.unshift(newNote);
    saveStorage();
    switchSidebarView('notes');
    loadNote(newNote.id);
    alert("Apple Calendar Event Imported!");
  };
  reader.readAsText(file);
}

function exportCalendarICS() {
  let icsContent = "BEGIN:VCALENDAR\nVERSION:2.0\nPRODID:-//Vaukt Studio//EN\n";
  notes.filter(n => !n.isBinned).forEach(n => {
    const dateClean = (n.created || getSystemTodayDate()).replace(/-/g, '');
    icsContent += `BEGIN:VEVENT\nSUMMARY:${n.title}\nDESCRIPTION:${n.content.replace(/\n/g, '
  });
  icsContent += "END:VCALENDAR";
  saveAs(new Blob([icsContent], { type: 'text/calendar;charset=utf-8' })), "Vaukt_Apple_Calendar_
}

function resetVaultWithBackup() {
  if (!confirm("ARE YOU SURE?\n\nResetting will download all your files as a ZIP archive and wip
exportAllNotesZip();
setTimeout(() => {
  notes = [];
  activeNoteId = null;
  openTabs = [];
  selectedNoteIds.clear();
  saveStorage();
  renderEditorState();
  renderTags();
  closeSettingsModal(null, true);
  alert("Vault reset complete.");
}, 300);
}

function renderNavigatorList() {
  const list = document.getElementById('navigatorList');
  list.innerHTML = '';
  const isBinView = activeSidebarView === 'bin';
  const filteredNotes = getVisibleNotes();

  filteredNotes.forEach(note => {
    const card = document.createElement('div');
    const isSelected = selectedNoteIds.has(note.id);
    const isActive = note.id === activeNoteId;
    card.className = `note-card ${isActive ? 'active' : ''} ${isSelected ? 'selected' : ''}`;

    if (isBinView) {
      card.innerHTML = `
        <div class="note-card-title">${escapeHtml(note.title || 'Untitled')}</div>
        <div class="note-card-snippet">${escapeHtml(note.content || 'Empty note...')}</div>
        <div style="display:flex; gap:8px; margin-top:6px;">
          <button class="obs-btn" style="font-size:9px; padding:2px 6px;" onclick="resto
          <button class="obs-btn obs-btn-danger" style="font-size:9px; padding:2px 6px;"
        </div>
      `;
    } else {
      card.setAttribute('draggable', 'true');
      card.ondragstart = (e) => handleDragStart(e, note.id);
      card.onclick = (e) => handleNoteClick(e, note.id);
      card.innerHTML = `
        <div class="note-card-title">${escapeHtml(note.title || 'Untitled')}</div>
        <div class="note-card-snippet">${escapeHtml(note.content || 'Empty note...')}</div>
        <div class="note-card-meta">
          <span><i class="fa-regular fa-calendar"></i> ${note.created || getSystemTodayD
          ${note.tags ? `<span>#${escapeHtml(note.tags.split(',')[0])}</span>` : ''}
      `;
    }
  });
}

```

```

        </div>
        <i class="fa-solid fa-trash-can btn-card-bin" title="Move to Bin" onclick="event.s
    `;
    }
    list.appendChild(card);
  });
}

function renderTags() {
  const list = document.getElementById('tagPillList');
  list.innerHTML = '';
  const tagMap = {};
  notes.filter(n=>!n.isBinned).forEach(n => {
    if (n.tags) {
      n.tags.split(',').forEach(t => {
        const clean = t.trim();
        if (clean) tagMap[clean] = (tagMap[clean] || 0) + 1;
      });
    }
  });
  Object.keys(tagMap).forEach(tag => {
    const pill = document.createElement('div');
    pill.className = `tag-pill ${activeTagFilter === tag ? 'active' : ''}`;
    pill.innerText = `#${tag} (${tagMap[tag]})`;
    pill.onclick = () => {
      activeTagFilter = activeTagFilter === tag ? null : tag;
      renderTags();
      renderNavigatorList();
    };
    list.appendChild(pill);
  });
}

function changeThemeFromSelect() {
  const val = document.getElementById('themeSelect').value;
  if (val === 'light') document.body.classList.add('light-theme');
  else document.body.classList.remove('light-theme');
  localStorage.setItem('vaukt_theme', val);
  if (isGraphOpen) drawGraph();
}

function toggleGraphPane() {
  isGraphOpen = !isGraphOpen;
  const pane = document.getElementById('graphPane');
  const ribbonBtn = document.getElementById('ribbonGraphBtn');
  const toggleBtn = document.getElementById('graphToggleBtn');

  if (isGraphOpen) {
    pane.classList.remove('collapsed');
    ribbonBtn.classList.add('active');
    toggleBtn.classList.add('active');
    setTimeout(drawGraph, 50);
  } else {
    pane.classList.add('collapsed');
    ribbonBtn.classList.remove('active');
    toggleBtn.classList.remove('active');
  }
}

function setGraphMode(mode) {
  graphMode = mode;
  document.getElementById('btnModeLinks').classList.toggle('active', mode === 'links');
  document.getElementById('btnModeTags').classList.toggle('active', mode === 'tags');

  const tagContainer = document.getElementById('graphTagCheckboxes');
  if (mode === 'tags') {
    tagContainer.classList.add('active');
    renderTagCheckboxes();
  } else {
    tagContainer.classList.remove('active');
  }
  drawGraph();
}

function getAllUniqueTags() {
  const tags = new Set();
  notes.filter(n => !n.isBinned).forEach(n => {
    if (n.tags) {
      n.tags.split(',').forEach(t => {

```

```

    const clean = t.trim();
    if (clean) tags.add(clean);
  });
}
});
return Array.from(tags);
}

function renderTagCheckboxes() {
  const container = document.getElementById('tagCheckboxList');
  container.innerHTML = '';
  const allTags = getAllUniqueTags();

  if (selectedGraphTags.size === 0 && allTags.length > 0) {
    allTags.forEach(t => selectedGraphTags.add(t));
  }

  allTags.forEach(tag => {
    const label = document.createElement('label');
    label.className = 'tag-checkbox-item';
    const isChecked = selectedGraphTags.has(tag);

    label.innerHTML = `
      <input type="checkbox" ${isChecked ? 'checked' : ''} onchange="toggleTagFilter('${tag}'
      <span>#${escapeHtml(tag)}</span>
    `;
    container.appendChild(label);
  });
}

function toggleTagFilter(tag, isChecked) {
  if (isChecked) selectedGraphTags.add(tag);
  else selectedGraphTags.delete(tag);
  drawGraph();
}

function selectAllGraphTags(status) {
  const allTags = getAllUniqueTags();
  selectedGraphTags.clear();
  if (status) allTags.forEach(t => selectedGraphTags.add(t));
  renderTagCheckboxes();
  drawGraph();
}

function adjustGraphZoom(factor) {
  graphZoom *= factor;
  graphZoom = Math.min(Math.max(0.3, graphZoom), 4.0);
  drawGraph();
}

function resetGraphZoom() {
  graphZoom = 1; graphPanX = 0; graphPanY = 0;
  drawGraph();
}

function openSettingsModal() { document.getElementById('settingsModalOverlay').classList.add('active'); }
function closeSettingsModal(e, force = false) {
  if (force || e.target.id === 'settingsModalOverlay') document.getElementById('settingsModalOverlay').classList.remove('active');
}

function downloadCurrentNoteMD() {
  const note = notes.find(n => n.id === activeNoteId);
  if (!note) return;
  const filename = (note.title.trim() || 'Untitled').replace(/[/\?%*:|"<>]/g, '_') + '.md';
  saveAs(new Blob([`---\ncreated: ${note.created} | ${getSystemTodayDate()} \ntags: ${note.tags} |`]), filename);
}

function exportAllNotesZip() {
  if (notes.length === 0) return alert("No notes to export.");
  const zip = new JSZip();
  const folder = zip.folder("Vaukt_Vault");
  notes.forEach(note => {
    const filename = (note.title.trim() || 'Untitled_' + note.id).replace(/[/\?%*:|"<>]/g, '_') + '.md';
    folder.file(filename, `---\ncreated: ${note.created} | ${getSystemTodayDate()} \ntags: ${note.tags} |`);
  });
  zip.generateAsync({ type: "blob" }).then(content => saveAs(content, "Vaukt_Vault_Export.zip"));
}

function triggerFolderImport() { document.getElementById('folderImporter').click(); }

```

```

async function handleImportFolder(event) {
  const files = event.target.files;
  if (!files || files.length === 0) return;
  for (let i = 0; i < files.length; i++) {
    const file = files[i];
    if (file.name.endsWith('.txt') || file.name.endsWith('.md')) {
      const content = await file.text();
      notes.unshift({ id: Date.now() + i, title: file.name.replace(/\.([^.]+\$/, ''), content
    }
  }
  saveStorage();
  renderNavigatorList();
  renderTags();
  if (notes.length > 0) loadNote(notes[0].id);
}

function handleEditorKeyDown(e) {
  if (!slashActive) return;
  if (e.key === 'Enter' && filteredSlashNotes.length > 0) {
    e.preventDefault();
    insertPageLink(filteredSlashNotes[0].title);
  }
}

function renderSlashList(query) {
  const list = document.getElementById('slashList');
  list.innerHTML = '';
  filteredSlashNotes = notes.filter(n => !n.isBinned && n.id !== activeNoteId && n.title.toLowerCase().includes(query));
  filteredSlashNotes.forEach((note) => {
    const item = document.createElement('div');
    item.className = 'slash-item';
    item.innerText = note.title;
    item.onmousedown = (e) => { e.preventDefault(); insertPageLink(note.title); };
    list.appendChild(item);
  });
}

function insertPageLink(title) {
  const textarea = document.getElementById('noteContent');
  const pos = textarea.selectionStart;
  const val = textarea.value;
  textarea.value = val.substring(0, slashStartPos) + `[${title}]` + val.substring(pos);
  document.getElementById('slashMenu').style.display = 'none';
  slashActive = false;
  updateCurrentNote();
}

function openSearchModal() {
  document.getElementById('searchModalOverlay').classList.add('active');
  document.getElementById('modalSearchInput').value = '';
  document.getElementById('modalSearchInput').focus();
  handleModalSearch();
}

function closeSearchModal(e) {
  if (e.target.id === 'searchModalOverlay') document.getElementById('searchModalOverlay').classList.remove('active');
}

function handleModalSearch() {
  const query = document.getElementById('modalSearchInput').value.toLowerCase();
  const results = notes.filter(n => !n.isBinned && (n.title.toLowerCase().includes(query) || n.content.toLowerCase().includes(query)));
  const list = document.getElementById('modalResultsList');
  list.innerHTML = '';
  results.forEach((note) => {
    const item = document.createElement('div');
    item.className = 'modal-result-item';
    item.innerHTML = `<span><i class="fa-regular fa-file-lines"></i> ${escapeHtml(note.title)}</span>`;
    item.onclick = () => {
      loadNote(note.id);
      document.getElementById('searchModalOverlay').classList.remove('active');
    };
    list.appendChild(item);
  });
  if (results[0]) {
    document.getElementById('modalPreviewTitle').innerText = results[0].title;
    document.getElementById('modalPreviewSnippet').innerText = results[0].content;
  }
}

```

```

let canvas, ctx, container;
function initGraph() {
  canvas = document.getElementById('graphCanvas');
  container = document.getElementById('graphCanvasContainer');
  ctx = canvas.getContext('2d');

  container.addEventListener('wheel', (e) => {
    e.preventDefault();
    adjustGraphZoom(e.deltaY < 0 ? 1.1 : 0.9);
  }, { passive: false });

  container.addEventListener('mousedown', (e) => {
    isPanningGraph = true;
    panStartX = e.clientX - graphPanX;
    panStartY = e.clientY - graphPanY;
  });

  window.addEventListener('mousemove', (e) => {
    if (!isPanningGraph) return;
    graphPanX = e.clientX - panStartX;
    graphPanY = e.clientY - panStartY;
    drawGraph();
  });

  window.addEventListener('mouseup', () => { isPanningGraph = false; });
  drawGraph();
}

function drawGraph() {
  if (!canvas || !container || !isGraphOpen) return;

  canvas.width = container.clientWidth;
  canvas.height = container.clientHeight;
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  let activeNotes = notes.filter(n => !n.isBinned);
  if (graphMode === 'tags' && selectedGraphTags.size > 0) {
    activeNotes = activeNotes.filter(n => {
      if (!n.tags) return false;
      const nTags = n.tags.split(',').map(t => t.trim());
      return nTags.some(t => selectedGraphTags.has(t));
    });
  }

  if (activeNotes.length === 0) return;

  ctx.save();
  ctx.translate(canvas.width / 2 + graphPanX, canvas.height / 2 + graphPanY);
  ctx.scale(graphZoom, graphZoom);

  const radius = Math.min(canvas.width, canvas.height) * 0.35;
  const nodePositions = {};

  activeNotes.forEach((note, idx) => {
    const angle = (idx / activeNotes.length) * Math.PI * 2;
    nodePositions[note.id] = { x: radius * Math.cos(angle), y: radius * Math.sin(angle), title
  });

  ctx.lineWidth = 1 / graphZoom;

  if (graphMode === 'links') {
    activeNotes.forEach(sourceNote => {
      activeNotes.forEach(targetNote => {
        if (sourceNote.id !== targetNote.id && sourceNote.content && sourceNote.content.in
        const p1 = nodePositions[sourceNote.id];
        const p2 = nodePositions[targetNote.id];
        if (p1 && p2) {
          ctx.beginPath(); ctx.moveTo(p1.x, p1.y); ctx.lineTo(p2.x, p2.y);
          ctx.strokeStyle = "rgba(124, 58, 237, 0.4)"; ctx.stroke();
        }
      });
    });
  } else if (graphMode === 'tags') {
    activeNotes.forEach((sourceNote, i) => {
      activeNotes.forEach((targetNote, j) => {
        if (i < j) {
          const sharedTags = sourceNote.tags ? sourceNote.tags.split(',').map(t=>t.trim(

```

```

        if (sharedTags.length > 0) {
            const p1 = nodePositions[sourceNote.id];
            const p2 = nodePositions[targetNote.id];
            if (p1 && p2) {
                ctx.beginPath(); ctx.moveTo(p1.x, p1.y); ctx.lineTo(p2.x, p2.y);
                ctx.strokeStyle = "rgba(167, 139, 250, 0.35)"; ctx.stroke();
            }
        }
    });
}

activeNotes.forEach((note) => {
    const pos = nodePositions[note.id];
    const isActive = note.id === activeNoteId;

    ctx.beginPath();
    ctx.arc(pos.x, pos.y, isActive ? 6 : 4, 0, Math.PI * 2);
    ctx.fillStyle = isActive ? "#a78bfa" : "#7c3aed";
    ctx.fill();

    ctx.font = isActive ? "bold 10px sans-serif" : "9px sans-serif";
    ctx.fillStyle = isActive ? "ffffff" : "#8e8e93";
    ctx.fillText(note.title.length > 14 ? note.title.substring(0, 12) + '...' : note.title, po
});

ctx.restore();
}

function escapeHtml(str) { return str.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;")
</script>
</body>
</html>
```[cite: 2]

```

User prompt: I want it to look like this and have the following feature :Based on your PDF document (the conversation with Google Gemini about building "Vaukt"), here is every single feature you built for this note-taking application: Core Features Multi-Tab Interface - Open multiple notes simultaneously like browser tabs (Cmd/Ctrl + T) Tab Navigation - Switch between tabs with keyboard shortcuts (Cmd/Ctrl + 1, 2, 3...) New Tab Creation - Create new tabs via keyboard shortcut or + button Close All Tabs - Empty state with "No file is open" message Note Management Multi-Selection - Select multiple notes (Cmd/Ctrl + Click for individual, Shift + Click for ranges) Page/File Selection with Checkboxes - Multi-select checkboxes in sidebar Bulk Delete - Delete multiple selected notes at once Bulk Move to Folders - Move selected notes into folders Drag and Drop to Bin - Click and drag selected notes to trash/bin Select All Toggle - Select all notes at once Folder & Organization Features Folder System - Organize notes into folders Folder Picker - Modal to select destination folder for moving notes Bin/Trash Folder - Move notes to bin instead of permanently deleting Folder Import - Import entire folders of notes (TXT/MD files) Markdown & File Management Markdown Support - Full markdown editor for notes Markdown Export (Individual) - Download single notes as .md files Batch Export (ZIP) - Export all notes as .md files in a zip folder Auto-Backup with Reset - Reset vault while exporting all files as backup ZIP Confirmation Dialog - "Are you sure?" confirmation before destructive actions Tagging & Organization Tag System - Add tags to notes Tag Templates - Built-in templates (Short Form, Long Form, Project, Business, etc.) Tag Filtering - Filter notes by tags Creative & Production Tags - Special tag categories for content creation Graph Visualization Graph View - Visual network graph of note connections Dual Graph Modes - Toggle between link-based and tag-based graphs Link Graph Mode - Shows connections based on page links ([[links]]) Tag Graph Mode - Shows connections based on shared tags Graph Zoom - Zoom in/out on graph (Mouse wheel or +/- buttons) Graph Pan - Click and drag to pan across the graph Graph Reset - Reset zoom and pan to default Graph Sidebar Toggle - Open/close graph as separate window Tag Checkbox Filter - Filter graph by specific tags Select/Deselect All Tags - Bulk tag filtering in graph Search & Discovery Global Search - Search notes by title and content Search Modal - Dedicated search interface Search Preview - Preview search results with title and snippet Link System Page Links - Link to other notes with [[Note Title]] syntax Link Detection - Auto-detect page links in editor Link Navigation - Click links to navigate to linked notes Slash Menu for Links - Type "/" to trigger link suggestion menu URL Detection - Auto-detect URLs in content URL Auto-Linking - Convert URLs to markdown links Website Name Automatic Title Fetching for Links - Extract titles from URLs Open Link in New Tab - Cmd/Ctrl + Click to open links in new tab Calendar & Dates Calendar Widget - Interactive calendar for date selection Created Date Property - Timestamp for when note was created Default Today's Date - Auto-set created date to system date Editor Features Rich Text Editor - Main note editing area Auto-Expanding Textarea - Text input grows as you type Enter to Send (Chatbox) - Submit with Enter key Shift + Enter - Line break in editor Character Limit Display - Show character count (if included) UI/Theme Features Dark Mode - Dark theme (default) Light Mode - Light theme option Settings Menu - Full settings interface Theme Toggle - Switch between light/dark modes in settings Font Selection - Choose between UI fonts (SF Mono, Source Code Pro) Obsidian-Inspired Design - UI modeled after Obsidian Minimal Theme Aesthetic - Clean, distraction-free design Distraction-Free Layout - Focused reading/writing experience Sidebar & Navigation Left Sidebar - Note list and navigation panel Note List - Chronological view of all notes Recent Chats/Notes - Quick access to recent items Active Note Highlighting - Show currently open note Hover States - Interactive feedback on sidebar items Searchable Sidebar - Find notes in sidebar list Data Persistence Local Storage - Save notes locally in browser IndexedDB Support - Local database storage Auto-Save - Automatically save changes Session Persistence - Retain notes between sessions Vault Backup - Export entire vault for backup Keyboard Shortcuts Cmd/Ctrl + T - New tab Cmd/Ctrl + 1, 2, 3... - Switch between tabs Cmd/Ctrl + Click - Multi-select notes Shift + Click - Range selection Enter Key - Submit text/create note Shift + Enter - Newline in editor Visual Elements Chat Bubbles - Message-style note display (original AI version) Avatars - User/AI avatars for messages Icons - FontAwesome icons throughout UI Color Coding - Different colors for user/AI messages (original version) Active States - Visual indicators for selected/active items Hover Effects - Interactive visual feedback Smooth Transitions - CSS transitions for interactions Additional Features AI Assistant Drawer - Built-in AI chat (later removed per request) New Chat Button - Start new conversations with AI (removed) Chat History - Track conversation history (removed) Responsive Design - Works on different screen sizes Accessibility - Proper semantic HTML and ARIA labels Error Handling - "No file is open" empty states Modal Dialogs - Confirmation and dialog boxes Form Validation - Input validation in forms The app went through multiple iterations, with features being added (tabs, calendar, graph zoom, link parsing, tag filters) and some being removed (AI chat feature was stripped out per your request). The final version is a local-first, markdown-based note-taking application called "Vaukt" with graph visualization, tag-based organization, and a tabbed interface similar to Notion.

Response: Here is the complete feature list for your application **Vaukt**:

## Core Features

- **Multi-Tab Interface** — Open multiple notes simultaneously like browser tabs (Cmd/Ctrl + T)
- **Tab Navigation** — Switch between tabs with keyboard shortcuts (Cmd/Ctrl + 1, 2, 3...)
- **New Tab Creation** — Create new tabs via keyboard shortcut or + button
- **Close All Tabs** — Empty state with "No file is open" message

## Note Management

- **Multi-Selection** — Select multiple notes (Cmd/Ctrl + Click for individual, Shift + Click for ranges)
- **Page/File Selection with Checkboxes** — Multi-select checkboxes in sidebar
- **Bulk Delete** — Delete multiple selected notes at once
- **Bulk Move to Folders** — Move selected notes into folders
- **Drag and Drop to Bin** — Click and drag selected notes to trash/bin
- **Select All Toggle** — Select all notes at once

## Folder & Organization Features

- **Folder System** — Organize notes into folders
- **Folder Picker** — Modal to select destination folder for moving notes
- **Bin/Trash Folder** — Move notes to bin instead of permanently deleting
- **Folder Import** — Import entire folders of notes (TXT/MD files)

## Markdown & File Management

- **Markdown Support** — Full markdown editor for notes
- **Markdown Export (Individual)** — Download single notes as .md files
- **Batch Export (ZIP)** — Export all notes as .md files in a zip folder
- **Auto-Backup with Reset** — Reset vault while exporting all files as backup ZIP
- **Confirmation Dialog** — "Are you sure?" confirmation before destructive actions

## Tagging & Organization

- **Tag System** — Add tags to notes
- **Tag Templates** — Built-in templates (Short Form, Long Form, Project, Business, etc.)
- **Tag Filtering** — Filter notes by tags
- **Creative & Production Tags** — Special tag categories for content creation

## Graph Visualization

- **Graph View** — Visual network graph of note connections
- **Dual Graph Modes** — Toggle between link-based and tag-based graphs
- **Link Graph Mode** — Shows connections based on page links ([[links]])
- **Tag Graph Mode** — Shows connections based on shared tags
- **Graph Zoom** — Zoom in/out on graph (Mouse wheel or +/- buttons)
- **Graph Pan** — Click and drag to pan across the graph
- **Graph Reset** — Reset zoom and pan to default
- **Graph Sidebar Toggle** — Open/close graph as separate window
- **Tag Checkbox Filter** — Filter graph by specific tags
- **Select/Deselect All Tags** — Bulk tag filtering in graph

## Search & Discovery

- **Global Search** — Search notes by title and content
- **Search Modal** — Dedicated search interface
- **Search Preview** — Preview search results with title and snippet

## Link System

- **Page Links** — Link to other notes with `[[Note Title]]` syntax
- **Link Detection** — Auto-detect page links in editor
- **Link Navigation** — Click links to navigate to linked notes
- **Slash Menu for Links** — Type "/" to trigger link suggestion menu
- **URL Detection** — Auto-detect URLs in content
- **URL Auto-Linking** — Convert URLs to markdown links Website Name
- **Automatic Title Fetching for Links** — Extract titles from URLs
- **Open Link in New Tab** — Cmd/Ctrl + Click to open links in new tab

## Calendar & Dates

- **Calendar Widget** — Interactive calendar for date selection
- **Created Date Property** — Timestamp for when note was created
- **Default Today's Date** — Auto-set created date to system date

## Editor Features

- **Rich Text Editor** — Main note editing area
- **Auto-Expanding Textarea** — Text input grows as you type
- **Enter to Send (Chatbox)** — Submit with Enter key
- **Shift + Enter** — Line break in editor
- **Character Limit Display** — Show character count (if included)

## UI/Theme Features

- **Dark Mode** — Dark theme (default)
- **Light Mode** — Light theme option
- **Settings Menu** — Full settings interface
- **Theme Toggle** — Switch between light/dark modes in settings
- **Font Selection** — Choose between UI fonts (SF Mono, Source Code Pro)
- **Obsidian-Inspired Design** — UI modeled after Obsidian
- **Minimal Theme Aesthetic** — Clean, distraction-free design
- **Distraction-Free Layout** — Focused reading/writing experience

## Sidebar & Navigation

- **Left Sidebar** — Note list and navigation panel
- **Note List** — Chronological view of all notes
- **Recent Chats/Notes** — Quick access to recent items
- **Active Note Highlighting** — Show currently open note
- **Hover States** — Interactive feedback on sidebar items
- **Searchable Sidebar** — Find notes in sidebar list

## Data Persistence

- **Local Storage** — Save notes locally in browser
- **IndexedDB Support** — Local database storage

- **Auto-Save** — Automatically save changes
- **Session Persistence** — Retain notes between sessions
- **Vault Backup** — Export entire vault for backup

## Keyboard Shortcuts

- **Cmd/Ctrl + T** — New tab
- **Cmd/Ctrl + 1, 2, 3...** — Switch between tabs
- **Cmd/Ctrl + Click** — Multi-select notes
- **Shift + Click** — Range selection
- **Enter Key** — Submit text/create note
- **Shift + Enter** — Newline in editor

## Visual Elements

- **Chat Bubbles** — Message-style note display (original AI version)
- **Avatars** — User/AI avatars for messages
- **Icons** — FontAwesome icons throughout UI
- **Color Coding** — Different colors for user/AI messages (original version)
- **Active States** — Visual indicators for selected/active items
- **Hover Effects** — Interactive visual feedback
- **Smooth Transitions** — CSS transitions for interactions

## Additional Features

- **AI Assistant Drawer** — Built-in AI chat (later removed per request)
- **New Chat Button** — Start new conversations with AI (removed)
- **Chat History** — Track conversation history (removed)
- **Responsive Design** — Works on different screen sizes
- **Accessibility** — Proper semantic HTML and ARIA labels
- **Error Handling** — "No file is open" empty states
- **Modal Dialogs** — Confirmation and dialog boxes
- **Form Validation** — Input validation in forms

User prompt: now code it

```
Response: <!DOCTYPE html>
<html lang="en" class="dark">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Vaukt - Local Markdown Knowledge Engine</title>
 <!-- Tailwind CSS CDN -->
 <script src="https://cdn.tailwindcss.com"></script>
 <script>
 tailwind.config = {
 darkMode: 'class',
 theme: {
 extend: {
 colors: {
 dark: {
 bg: '#181818',
 sidebar: '#141414',
 card: '#1e1e1e',
 border: '#2a2a2a',
 hover: '#262626',
 active: '#2c2c2c',
 text: '#e0e0e0',
 muted: '#888888',
 accent: '#a855f7'
 }
 }
 }
 }
 }
 </script>

```

```

 },
 light: {
 bg: '#fcfcfc',
 sidebar: '#f4f4f5',
 card: '#ffffff',
 border: '#e4e4e7',
 hover: '#f1f1f3',
 active: '#e4e4e7',
 text: '#18181b',
 muted: '#71717a',
 accent: '#9333ea'
 }
 }
}
}
}
}
</script>
<!-- FontAwesome Icons -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
<!-- Marked.js for Markdown Parsing -->
<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<style>
 body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif; font-family: SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }
 /* Custom scrollbars */
 ::-webkit-scrollbar { width: 6px; height: 6px; }
 ::-webkit-scrollbar-track { background: transparent; }
 ::-webkit-scrollbar-thumb { background: rgba(150, 150, 150, 0.2); border-radius: 3px; }
 ::-webkit-scrollbar-thumb:hover { background: rgba(150, 150, 150, 0.4); }
</style>
</head>
<body class="bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-none transition-colors">

 <!-- Top Window Drag / Tab Bar Header -->
 <header class="h-10 bg-dark-bg dark-border border-b border-dark-border flex items-center justify-between px-3">
 <div class="flex items-center space-x-2 overflow-x-auto no-scrollbar flex-1 mr-4" id="tabContainer">
 <!-- Tabs injected dynamically -->
 </div>

 <div class="flex items-center space-x-2 shrink-0">
 <button onclick="createNewTab()" title="New Tab (Cmd/Ctrl + T)" class="w-7 h-7 flex items-center justify-center rounded-md text-dark-text">
 <i class="fa-solid fa-plus text-xs"></i>
 </button>
 <div class="h-4 w-[1px] bg-dark-border mx-1"></div>
 <button onclick="toggleGraphSidebar()" title="Toggle Graph View" id="graphToggleBtn" class="px-2 py-1 rounded-md text-dark-text">
 <i class="fa-solid fa-chart-network text-dark-accent"></i>
 Graph
 </button>
 <button onclick="exportMarkdown()" title="Export Note" class="px-2.5 py-1 rounded text-xs flex items-center">
 <i class="fa-solid fa-download text-xs"></i>
 Export .md
 </button>
 </div>
 </header>

 <!-- Main Workspace Area -->
 <div class="flex-1 flex overflow-hidden relative">

 <!-- Left Sidebar: Notebook Navigator -->
 <aside class="w-72 bg-dark-bg dark-border border-r border-dark-border flex flex-col shrink-0 select-none">
 <!-- Sidebar Header & Search -->
 <div class="p-3 border-b border-dark-border space-y-2.5">
 <div class="flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <div class="w-5 h-5 rounded bg-dark-accent/20 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-book-bookmark text-xs"></i>
 </div>
 Notebook
 </div>
 <div class="flex items-center space-x-1">
 <button onclick="createNewNote()" title="New Note" class="w-6 h-6 flex items-center justify-center rounded-md text-dark-text">
 <i class="fa-solid fa-file-circle-plus text-xs"></i>
 </button>
 <button onclick="openSettingsModal()" title="Settings" class="w-6 h-6 flex items-center justify-center rounded-md text-dark-text">
 <i class="fa-solid fa-gear text-xs"></i>
 </button>
 </div>
 </div>
 </div>
 </aside>
 </div>

```

```

 <!-- Search Bar -->
 <div class="relative">
 <i class="fa-solid fa-search absolute left-2.5 top-2.5 text-xs text-dark-muted"></i>
 <input type="text" id="searchInput" placeholder="Quick open note... (Shift + K)" oninp
 <span class="absolute right-2 top-2 text-[10px] text-dark-muted bg-dark-border/50 px-1
 </div>
 </div>

 <!-- Folders / Categories Quick Header -->
 <div class="px-3 py-2 border-b border-dark-border/50 flex items-center justify-between text-[1
 Folders
 <button onclick="createNewFolder()" class="hover:text-dark-text"><i class="fa-solid fa-fol
 </div>
 <div id="folderList" class="px-2 py-1.5 space-y-0.5 border-b border-dark-border max-h-36 overf
 <!-- Folders injected -->
 </div>

 <!-- Notes List Section -->
 <div class="flex-1 overflow-y-auto p-2 space-y-1" id="notesListContainer">
 <!-- Notes list injected dynamically -->
 </div>

 <!-- Tags Filter Bar Footer -->
 <div class="p-2 border-t border-dark-border bg-dark-card/50">
 <div class="text-[10px] uppercase font-semibold text-dark-muted mb-1.5">Tags</div>
 <div id="sidebarTagsList" class="flex flex-wrap gap-1 max-h-20 overflow-y-auto">
 <!-- Tags injected -->
 </div>
 </div>
</aside>

<!-- Center Editor Area -->
<main class="flex-1 flex flex-col bg-dark-bg overflow-hidden relative" id="editorMainArea">
 <div id="editorWrapper" class="flex-1 flex flex-col h-full overflow-hidden">
 <!-- Note Content Rendered Here -->
 </div>
</main>

<!-- Right Graph Sidebar -->
<aside id="graphSidebar" class="w-96 bg-dark-sidebar border-l border-dark-border flex flex-col shr
 <div class="p-3 border-b border-dark-border flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <i class="fa-solid fa-chart-network text-dark-accent text-xs"></i>
 Graph View
 </div>
 <div class="flex items-center space-x-2">
 <div class="bg-dark-card border border-dark-border rounded p-0.5 flex text-[10px]">
 <button onclick="setGraphMode('links')" id="graphModeLinks" class="px-2 py-0.5 rou
 <button onclick="setGraphMode('tags')" id="graphModeTags" class="px-2 py-0.5 round
 </div>
 <button onclick="toggleGraphSidebar()" class="w-6 h-6 flex items-center justify-center
 <i class="fa-solid fa-xmark text-xs"></i>
 </button>
 </div>
 </div>
 <div class="flex-1 relative bg-dark-card/20 overflow-hidden flex items-center justify-center"
 <canvas id="graphCanvas" class="w-full h-full cursor-grab active:cursor-grabbing"></canvas>
 <div class="absolute bottom-3 right-3 flex items-center space-x-1 bg-dark-card/90 border b
 <button onclick="zoomGraph(1.2)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="zoomGraph(0.8)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="resetGraphZoom()" title="Reset View" class="w-6 h-6 rounded hover:bg-
 </div>
 </div>
 <div class="p-2 border-t border-dark-border bg-dark-card/50 text-xs text-dark-muted flex justi
 0 nodes, 0 connections
 <button onclick="renderGraph()" class="text-[11px] text-dark-accent hover:underline">Refre
 </div>
</aside>

</div>

<!-- Modals & Context menus container -->
<div id="modalContainer"></div>

<!-- Application Logic -->
<script>
 // --- State Management ---
 let state = {
 notes: [

```

```

 {
 id: 'note-1',
 title: 'Welcome to Vaukt',
 content: `# Welcome to Vaukt\n\nYour local-first, markdown-based knowledge vault inspi
 folder: 'General',
 tags: ['idea', 'imported'],
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 },
 {
 id: 'note-2',
 title: 'Project Roadmap',
 content: `# Project Roadmap\n\nGoals for building out Vaukt:\n1. Enhance graph network
 folder: 'Academics',
 tags: ['project', 'business'],
 createdAt: new Date(Date.now() - 86400000).toISOString(),
 updatedAt: new Date(Date.now() - 86400000).toISOString()
 },
 {
 id: 'note-3',
 title: 'Quick Thoughts',
 content: `# Quick Thoughts\n\nRandom musings on UI design and minimal interfaces. Simp
 folder: 'General',
 tags: ['short_form'],
 createdAt: new Date(Date.now() - 172800000).toISOString(),
 updatedAt: new Date(Date.now() - 172800000).toISOString()
 }
],
 folders: ['General', 'Academics', 'Projects', 'Archive'],
 openTabs: ['note-1'],
 activeTabId: 'note-1',
 selectedNoteIds: [],
 searchQuery: '',
 selectedFolderFilter: null,
 selectedTagFilter: null,
 settings: {
 theme: 'dark',
 font: 'mono', // 'mono' or 'sans'
 },
 graph: {
 mode: 'links', // 'links' or 'tags'
 sidebarOpen: false,
 zoom: 1,
 panX: 0,
 panY: 0
 }
};

// Local Storage persistence
function saveToLocalStorage() {
 localStorage.setItem('vaukt_state', JSON.stringify({
 notes: state.notes,
 folders: state.folders,
 openTabs: state.openTabs,
 activeTabId: state.activeTabId,
 settings: state.settings
 }));
}

function loadFromLocalStorage() {
 const saved = localStorage.getItem('vaukt_state');
 if (saved) {
 try {
 const parsed = JSON.parse(saved);
 state.notes = parsed.notes || state.notes;
 state.folders = parsed.folders || state.folders;
 state.openTabs = parsed.openTabs || state.openTabs;
 state.activeTabId = parsed.activeTabId || state.activeTabId;
 if (parsed.settings) state.settings = parsed.settings;
 } catch(e) { console.error(e); }
 }
}

// --- Initialization ---
window.addEventListener('DOMContentLoaded', () => {
 loadFromLocalStorage();
 applySettings();
 renderAll();
 setupEventListeners();
});

```

```

});

function applySettings() {
 const body = document.getElementById('appBody');
 if (state.settings.theme === 'light') {
 body.className = 'bg-light-bg text-light-text h-screen flex flex-col overflow-hidden select-
 } else {
 body.className = 'bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-
 }
}

function renderAll() {
 renderTabs();
 renderFolders();
 renderNotesList();
 renderTagsSidebar();
 renderActiveEditor();
 if (state.graph.sidebarOpen) {
 renderGraph();
 }
}

// --- Tab Management ---
function renderTabs() {
 const container = document.getElementById('tabContainer');
 container.innerHTML = '';

 if (state.openTabs.length === 0) {
 container.innerHTML = `No file is open</
 return;
 }

 state.openTabs.forEach((noteId, index) => {
 const note = state.notes.find(n => n.id === noteId);
 if (!note) return;

 const isActive = noteId === state.activeTabId;
 const tab = document.createElement('div');
 tab.className = `flex items-center space-x-2 px-3 py-1.5 rounded-t text-xs cursor-pointer
 isActive
 ? 'bg-dark-card border-dark-accent text-dark-text shadow-sm'
 : 'bg-dark-sidebar border-transparent text-dark-muted hover:bg-dark-hover hover:te
 `;
 tab.onclick = (e) => {
 if (e.target.closest('.close-tab')) return;
 switchTab(noteId);
 };

 tab.innerHTML = `
 <i class="fa-regular fa-file-lines text-dark-accent/70 text-[11px]"></i>
 ${escapeHtml(note.title || 'Untitled Note')}
 <button onclick="closeTab('${noteId}')" class="close-tab w-4 h-4 rounded-full hover:bg
 <i class="fa-solid fa-xmark text-[10px]"></i>
 </button>
 `;
 container.appendChild(tab);
 });
}

function createNewTab() {
 const newId = 'note-' + Date.now();
 const newNote = {
 id: newId,
 title: 'Untitled Note',
 content: `# Untitled Note\n\nType note content... Press '/' to autocomplete page links!\n\n
 folder: state.selectedFolderFilter || 'General',
 tags: ['idea'],
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 };
 state.notes.unshift(newNote);
 state.openTabs.push(newId);
 state.activeTabId = newId;
 saveToLocalStorage();
 renderAll();
}

function createNewNote() {
 createNewTab();
}

```

```

}

function switchTab(noteId) {
 state.activeTabId = noteId;
 renderAll();
}

function closeTab(noteId) {
 const idx = state.openTabs.indexOf(noteId);
 if (idx > -1) {
 state.openTabs.splice(idx, 1);
 }
 if (state.activeTabId === noteId) {
 state.activeTabId = state.openTabs.length > 0 ? state.openTabs[state.openTabs.length - 1]
 }
 saveToLocalStorage();
 renderAll();
}

// --- Notebook Sidebar & Folders ---
function renderFolders() {
 const container = document.getElementById('folderList');
 container.innerHTML = '';

 // All notes option
 const allDiv = document.createElement('div');
 allDiv.className = `flex items-center justify-between px-2 py-1 rounded text-xs cursor-pointer`;
 allDiv.onclick = () => { state.selectedFolderFilter = null; renderAll(); };
 allDiv.innerHTML = `<div class="flex items-center space-x-2"><i class="fa-solid fa-folder-open`
 container.appendChild(allDiv);

 state.folders.forEach(folder => {
 const count = state.notes.filter(n => n.folder === folder).length;
 const isSelected = state.selectedFolderFilter === folder;
 const fDiv = document.createElement('div');
 fDiv.className = `flex items-center justify-between px-2 py-1 rounded text-xs cursor-point`;
 fDiv.onclick = () => { state.selectedFolderFilter = folder; renderAll(); };
 fDiv.innerHTML = `<div class="flex items-center space-x-2"><i class="fa-solid fa-folder te`;
 container.appendChild(fDiv);
 });
}

function createNewFolder() {
 const folderName = prompt('Enter new folder name:');
 if (folderName && !state.folders.includes(folderName)) {
 state.folders.push(folderName);
 saveToLocalStorage();
 renderFolders();
 }
}

function renderNotesList() {
 const container = document.getElementById('notesListContainer');
 container.innerHTML = '';

 let filtered = state.notes.filter(n => {
 if (state.selectedFolderFilter && n.folder !== state.selectedFolderFilter) return false;
 if (state.selectedTagFilter && (!n.tags || !n.tags.includes(state.selectedTagFilter))) return false;
 if (state.searchQuery) {
 const q = state.searchQuery.toLowerCase();
 return (n.title && n.title.toLowerCase().includes(q)) || (n.content && n.content.toLow
 }
 return true;
 });

 if (filtered.length === 0) {
 container.innerHTML = `<div class="p-4 text-center text-xs text-dark-muted italic">No note`;
 return;
 }

 filtered.forEach(note => {
 const isActive = note.id === state.activeTabId;
 const isSelected = state.selectedNoteIds.includes(note.id);

 const card = document.createElement('div');
 card.className = `p-2.5 rounded-lg border cursor-pointer transition relative group ${
 isActive
 ? 'bg-dark-card border-dark-accent shadow-sm'
 : isSelected
 }`;
 });
}

```

```

 ? 'bg-dark-active border-dark-border'
 : 'bg-dark-card/60 border-dark-border hover:bg-dark-card hover:border-dark-mut
 `};

 card.onclick = (e) => {
 if (e.metaKey || e.ctrlKey || e.shiftKey) {
 e.preventDefault();
 toggleSelectNote(note.id, e.shiftKey);
 } else {
 if (!state.openTabs.includes(note.id)) {
 state.openTabs.push(note.id);
 }
 switchTab(note.id);
 }
 };

 const snippet = note.content ? note.content.replace(/#.*?\n/g, '').trim().substring(0, 60)
 const dateStr = new Date(note.updatedAt || note.createdAt).toLocaleDateString(undefined, {

 card.innerHTML = `
 <div class="flex items-start justify-between mb-1">
 <div class="flex items-center space-x-2 truncate pr-2">
 <input type="checkbox" ${isSelected ? 'checked' : ''} onclick="event.stopPropa
 ${escapeHtml(note.
 </div>
 ${dateStr}
 </div>
 <p class="text-[11px] text-dark-muted line-clamp-2 pl-5 mb-1.5">${escapeHtml(snippet)}
 <div class="flex items-center justify-between pl-5">
 <div class="flex flex-wrap gap-1">
 ${note.tags || []}.map(t => `<span class="text-[9px] bg-dark-bg text-dark-acc
 </div>
 <button onclick="event.stopPropagation(); deleteNote('${note.id}')" class="opacity
 </div>
 `;
 container.appendChild(card);
 });
}

function toggleSelectNote(id, isShift = false) {
 const idx = state.selectedNoteIds.indexOf(id);
 if (idx > -1) {
 state.selectedNoteIds.splice(idx, 1);
 } else {
 state.selectedNoteIds.push(id);
 }
 renderNotesList();
}

function deleteNote(id) {
 if (confirm('Are you sure you want to delete this note?')) {
 state.notes = state.notes.filter(n => n.id !== id);
 state.openTabs = state.openTabs.filter(tid => tid !== id);
 if (state.activeTabId === id) {
 state.activeTabId = state.openTabs.length > 0 ? state.openTabs[state.openTabs.length -
]
 saveToLocalStorage();
 renderAll();
 }
}

function handleSearch(val) {
 state.searchQuery = val;
 renderNotesList();
}

function renderTagsSidebar() {
 const container = document.getElementById('sidebarTagsList');
 container.innerHTML = '';

 const allTags = {};
 state.notes.forEach(n => {
 if (n.tags) {
 n.tags.forEach(t => allTags[t] = (allTags[t] || 0) + 1);
 }
 });

 Object.keys(allTags).forEach(tag => {
 const isSelected = state.selectedTagFilter === tag;

```

```

const badge = document.createElement('button');
badge.className = `text-[10px] px-2 py-0.5 rounded border transition flex items-center spa
 isSelected
 ? 'bg-dark-accent text-white border-dark-accent font-medium'
 : 'bg-dark-card text-dark-muted border-dark-border hover:border-dark-accent/50 hov
 `;
badge.onclick = () => {
 state.selectedTagFilter = isSelected ? null : tag;
 renderAll();
};
badge.innerHTML = `#{tag}(${allTags[tag]
container.appendChild(badge);
});
}

// --- Active Note Editor ---
function renderActiveEditor() {
 const container = document.getElementById('editorWrapper');
 container.innerHTML = '';

 if (!state.activeTabId || state.openTabs.length === 0) {
 container.innerHTML = `
 <div class="flex-1 flex flex-col items-center justify-center p-8 text-center text-dark
 <div class="w-16 h-16 rounded-full bg-dark-card border border-dark-border flex ite
 <i class="fa-solid fa-file-circle-plus"></i>
 </div>
 <h2 class="text-base font-semibold text-dark-text mb-1">No file is open</h2>
 <p class="text-xs max-w-sm mb-4">Select a note from the sidebar or create a new ta
 <button onclick="createNewTab()" class="px-4 py-2 bg-dark-accent text-white rounde
 </div>
 `;
 return;
 }

 const note = state.notes.find(n => n.id === state.activeTabId);
 if (!note) return;

 container.innerHTML = `
 <!-- Editor Top Toolbar -->
 <div class="h-11 border-b border-dark-border px-4 flex items-center justify-between shrink
 <div class="flex items-center space-x-3">
 <i class="fa-solid fa-folder text-blue-400 m
 <div class="h-3 w-[1px] bg-dark-border"></div>
 <i class="fa-regular fa-clock mr-1"></i>${ne
 </div>
 <div class="flex items-center space-x-2">
 <div class="flex bg-dark-card border border-dark-border rounded p-0.5 text-xs">
 <button onclick="switchEditMode('edit')" id="modeEditBtn" class="px-2.5 py-1 r
 <button onclick="switchEditMode('preview')" id="modePreviewBtn" class="px-2.5
 </div>
 </div>
 </div>

 <!-- Editor Body Area -->
 <div class="flex-1 flex flex-col overflow-y-auto p-6 max-w-4xl mx-auto w-full" id="editorC
 <input type="text" id="noteTitleInput" value="${escapeHtml(note.title)}" oninput="upda

 <!-- Tags Display & Input -->
 <div class="flex flex-wrap items-center gap-1.5 mb-4 pb-3 border-b border-dark-border/
 <i class="fa-solid fa-t
 ${note.tags || []}.map(t => `
 <span class="inline-flex items-center space-x-1 text-xs bg-dark-card border bo
 #{t}
 <button onclick="removeTagFromNote('${note.id}', '${t}')" class="hover:tex

 `).join('')}>
 <button onclick="addTagPrompt('${note.id}')" class="text-[11px] text-dark-muted ho
 </div>

 <!-- Main Textarea / Preview Container -->
 <div id="editorPane" class="flex-1 flex flex-col relative">
 <textarea id="noteTextarea" oninput="updateNoteContent(this.value)" placeholder="T
 </div>
 <div id="previewPane" class="flex-1 hidden prose prose-invert max-w-none text-sm text-
 <!-- Markdown rendered output -->
 </div>
 </div>
 `;
}

```

```

function updateNoteTitle(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 note.title = val;
 note.updatedAt = new Date().toISOString();
 saveToLocalStorage();
 renderTabs();
 renderNotesList();
 }
}

function updateNoteContent(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 note.content = val;
 note.updatedAt = new Date().toISOString();

 // Auto-detect hashtags
 const hashtagMatches = val.match(/#[a-zA-Z0-9_]+/g);
 if (hashtagMatches) {
 const tags = hashtagMatches.map(tag => tag.substring(1));
 note.tags = Array.from(new Set(tags));
 }

 saveToLocalStorage();
 renderNotesList();
 }
}

function switchEditMode(mode) {
 const editBtn = document.getElementById('modeEditBtn');
 const previewBtn = document.getElementById('modePreviewBtn');
 const textarea = document.getElementById('noteTextarea');
 const previewPane = document.getElementById('previewPane');

 const note = state.notes.find(n => n.id === state.activeTabId);

 if (mode === 'edit') {
 editBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transition";
 previewBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition";
 textarea.classList.remove('hidden');
 previewPane.classList.add('hidden');
 } else {
 previewBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transition";
 editBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition";
 textarea.classList.add('hidden');
 previewPane.classList.remove('hidden');

 // Parse markdown and [[links]]
 let contentHtml = marked.parse(note.content || '');
 // Convert [[Link]] to interactive links
 contentHtml = contentHtml.replace(/\[\[([.*?)]\]\]/g, (match, p1) => {
 return `<span onclick="openLinkedNote('${p1}')" class="text-dark-accent underline cursor`
 });
 previewPane.innerHTML = contentHtml;
 }
}

function openLinkedNote(title) {
 let target = state.notes.find(n => n.title.toLowerCase() === title.toLowerCase());
 if (!target) {
 // Create note if not exists
 const newId = 'note-' + Date.now();
 target = {
 id: newId,
 title: title,
 content: `# ${title}\n\nCreated via bidirectional link.`,
 folder: 'General',
 tags: ['linked'],
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 };
 state.notes.unshift(target);
 }
 if (!state.openTabs.includes(target.id)) {
 state.openTabs.push(target.id);
 }
 switchTab(target.id);
}

```

```

}

function addTagPrompt(noteId) {
 const tag = prompt('Enter tag name (without #):');
 if (tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 if (!note.tags) note.tags = [];
 if (!note.tags.includes(tag.trim())) {
 note.tags.push(tag.trim());
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
 }
 }
}

function removeTagFromNote(noteId, tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note && note.tags) {
 note.tags = note.tags.filter(t => t !== tag);
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
}

// --- Export Utility ---
function exportMarkdown() {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (!note) return;
 const blob = new Blob([note.content], { type: 'text/markdown;charset=utf-8' });
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `${(note.title || 'untitled').replace(/[a-z0-9]/gi, '_').toLowerCase()}.md`;
 a.click();
 URL.revokeObjectURL(url);
}

// --- Graph Visualization ---
function toggleGraphSidebar() {
 state.graph.sidebarOpen = !state.graph.sidebarOpen;
 const sidebar = document.getElementById('graphSidebar');
 const btn = document.getElementById('graphToggleBtn');
 if (state.graph.sidebarOpen) {
 sidebar.classList.remove('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-active";
 renderGraph();
 } else {
 sidebar.classList.add('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-card bo";
 }
}

function setGraphMode(mode) {
 state.graph.mode = mode;
 const linkBtn = document.getElementById('graphModeLinks');
 const tagBtn = document.getElementById('graphModeTags');
 if (mode === 'links') {
 linkBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transition";
 tagBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition";
 } else {
 tagBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transition";
 linkBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition";
 }
 renderGraph();
}

function zoomGraph(factor) {
 state.graph.zoom *= factor;
 renderGraph();
}

function resetGraphZoom() {
 state.graph.zoom = 1;
 state.graph.panX = 0;
 state.graph.panY = 0;
}

```

```

 renderGraph();
 }
}

function renderGraph() {
 const canvas = document.getElementById('graphCanvas');
 if (!canvas) return;
 const ctx = canvas.getContext('2d');
 canvas.width = canvas.parentElement.clientWidth;
 canvas.height = canvas.parentElement.clientHeight;

 ctx.clearRect(0, 0, canvas.width, canvas.height);

 // Compute nodes & edges based on mode
 let nodes = [];
 let edges = [];

 if (state.graph.mode === 'links') {
 nodes = state.notes.map(n => ({ id: n.id, label: n.title, val: 1 }));
 state.notes.forEach(n => {
 if (!n.content) return;
 const links = n.content.match(/\[\[([.*?])\]\]/g);
 if (links) {
 links.forEach(l => {
 const title = l.replace(/\[\[|\]\]/g, '').trim();
 const target = state.notes.find(t => t.title.toLowerCase() === title.toLowerCase());
 if (target) {
 edges.push({ source: n.id, target: target.id });
 }
 });
 }
 });
 } else {
 // Tag mode
 const tagMap = {};
 state.notes.forEach(n => {
 if (n.tags) {
 n.tags.forEach(t => {
 if (!tagMap[t]) tagMap[t] = [];
 tagMap[t].push(n.id);
 });
 }
 });
 nodes = Object.keys(tagMap).map(t => ({ id: t, label: '#' + t, val: tagMap[t].length }));
 // Create edges between tags that share notes
 const tags = Object.keys(tagMap);
 for (let i = 0; i < tags.length; i++) {
 for (let j = i + 1; j < tags.length; j++) {
 const shared = tagMap[tags[i]].some(id => tagMap[tags[j]].includes(id));
 if (shared) {
 edges.push({ source: tags[i], target: tags[j] });
 }
 }
 }
 }

 document.getElementById('graphStats').innerText = `${nodes.length} nodes, ${edges.length} connections`;

 // Layout nodes in a circle
 const centerX = (canvas.width / 2) + state.graph.panX;
 const centerY = (canvas.height / 2) + state.graph.panY;
 const radius = Math.min(canvas.width, canvas.height) * 0.3 * state.graph.zoom;

 const nodePositions = {};
 nodes.forEach((node, i) => {
 const angle = (i / nodes.length) * 2 * Math.PI;
 nodePositions[node.id] = {
 x: centerX + radius * Math.cos(angle),
 y: centerY + radius * Math.sin(angle),
 label: node.label
 };
 });

 // Draw edges
 ctx.strokeStyle = 'rgba(168, 85, 247, 0.25)';
 ctx.lineWidth = 1.5;
 edges.forEach(edge => {
 const p1 = nodePositions[edge.source];
 const p2 = nodePositions[edge.target];
 if (p1 && p2) {

```

```

 ctx.beginPath();
 ctx.moveTo(p1.x, p1.y);
 ctx.lineTo(p2.x, p2.y);
 ctx.stroke();
 }
});

// Draw nodes
Object.keys(nodePositions).forEach(id => {
 const p = nodePositions[id];
 ctx.fillStyle = '#a855f7';
 ctx.beginPath();
 ctx.arc(p.x, p.y, 6 * Math.sqrt(state.graph.zoom), 0, 2 * Math.PI);
 ctx.fill();

 ctx.fillStyle = '#e0e0e0';
 ctx.font = '11px sans-serif';
 ctx.fillText(p.label, p.x + 10, p.y + 4);
});
}

// --- Settings Modal ---
function openSettingsModal() {
 const container = document.getElementById('modalContainer');
 container.innerHTML = `
 <div class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-
 <div class="bg-dark-card border border-dark-border rounded-xl w-full max-w-md p-6 shad
 <div class="flex items-center justify-between pb-4 border-b border-dark-border mb-
 <h3 class="font-bold text-sm text-dark-text flex items-center space-x-2">
 <i class="fa-solid fa-gear text-dark-accent"></i>
 Vaukt Settings
 </h3>
 <button onclick="closeModal()" class="text-dark-muted hover:text-dark-text"><i
 </div>
 <div class="space-y-4">
 <div>
 <label class="text-xs font-medium text-dark-muted block mb-2">Appearance T
 <div class="grid grid-cols-2 gap-2">
 <button onclick="setTheme('dark')" class="p-3 rounded border text-xs f
 <i class="fa-solid fa-moon"></i>Dark Mode
 </button>
 <button onclick="setTheme('light')" class="p-3 rounded border text-xs
 <i class="fa-solid fa-sun"></i>Light Mode
 </button>
 </div>
 </div>
 </div>
 <div class="mt-6 pt-4 border-t border-dark-border flex justify-end">
 <button onclick="closeModal()" class="px-4 py-2 bg-dark-accent text-white roun
 </div>
 </div>
 `;
}

function setTheme(theme) {
 state.settings.theme = theme;
 saveToLocalStorage();
 applySettings();
 openSettingsModal(); // Refresh modal active state
}

function closeModal() {
 document.getElementById('modalContainer').innerHTML = '';
}

// --- Global Keyboard Shortcuts ---
function setupEventListeners() {
 window.addEventListener('keydown', (e) => {
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 't') {
 e.preventDefault();
 createNewTab();
 }
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 'k') {
 e.preventDefault();
 document.getElementById('searchInput').focus();
 }
 if ((e.metaKey || e.ctrlKey) && !isNaN(e.key) && e.key !== '0') {
 e.preventDefault();

```

```

 const index = parseInt(e.key) - 1;
 if (state.openTabs[index]) {
 switchTab(state.openTabs[index]);
 }
 });
}
</script>
</body>
</html>

```

User prompt: make it fully look like this; as well as do all the things i wanted it: Based on your PDF document (the conversation with Google Gemini about building "Vaukt"), here is every single feature you built for this note-taking application: Core Features Multi-Tab Interface - Open multiple notes simultaneously like browser tabs (Cmd/Ctrl + T) Tab Navigation - Switch between tabs with keyboard shortcuts (Cmd/Ctrl + 1, 2, 3...) New Tab Creation - Create new tabs via keyboard shortcut or + button Close All Tabs - Empty state with "No file is open" message Note Management Multi-Selection - Select multiple notes (Cmd/Ctrl + Click for individual, Shift + Click for ranges) Page/File Selection with Checkboxes - Multi-select checkboxes in sidebar Bulk Delete - Delete multiple selected notes at once Bulk Move to Folders - Move selected notes into folders Drag and Drop to Bin - Click and drag selected notes to trash/bin Select All Toggle - Select all notes at once Folder & Organization Features Folder System - Organize notes into folders Folder Picker - Modal to select destination folder for moving notes Bin/Trash Folder - Move notes to bin instead of permanently deleting Folder Import - Import entire folders of notes (TXT/MD files) Markdown & File Management Multi-Selection - Full markdown editor for notes Markdown Export (Individual) - Download single notes as .md files Batch Export (ZIP) - Export all notes as .md files in a zip folder Auto-Backup with Reset - Reset vault while exporting all files as backup ZIP Confirmation Dialog - "Are you sure?" confirmation before destructive actions Tagging & Organization Tag System - Add tags to notes Tag Templates - Built-in templates (Short Form, Long Form, Project, Business, etc.) Tag Filtering - Filter notes by tags Creative & Production Tags - Special tag categories for content creation Graph Visualization Graph View - Visual network graph of note connections Dual Graph Modes - Toggle between link-based and tag-based graphs Link Graph Mode - Shows connections based on page links ([[links]]) Tag Graph Mode - Shows connections based on shared tags Graph Zoom - Zoom in/out on graph (Mouse wheel or +/- buttons) Graph Pan - Click and drag to pan across the graph Graph Reset - Reset zoom and pan to default Graph Sidebar Toggle - Open/close graph as separate window Tag Checkbox Filter - Filter graph by specific tags Select/Deselect All Tags - Bulk tag filtering in graph Search & Discovery Global Search - Search notes by title and content Search Modal - Dedicated search interface Search Preview - Preview search results with title and snippet Link System Page Links - Link to other notes with [[Note Title]] syntax Link Detection - Auto-detect page links in editor Link Navigation - Click links to navigate to linked notes Slash Menu for Links - Type "/" to trigger link suggestion menu URL Detection - Auto-detect URLs in content URL Auto-Linking - Convert URLs to markdown links Website Name Automatic Title Fetching for Links - Extract titles from URLs Open Link in New Tab - Cmd/Ctrl + Click to open links in new tab Calendar & Dates Calendar Widget - Interactive calendar for date selection Created Date Property - Timestamp for when note was created Default Today's Date - Auto-set created date to system date Editor Features Rich Text Editor - Main note editing area Auto-Expanding Textarea - Text input grows as you type Enter to Send (Chatbox) - Submit with Enter key Shift + Enter - Line break in editor Character Limit Display - Show character count (if included) UI/Theme Features Dark Mode - Dark theme (default) Light Mode - Light theme option Settings Menu - Full settings interface Theme Toggle - Switch between light/dark modes in settings Font Selection - Choose between UI fonts (SF Mono, Source Code Pro) Obsidian-Inspired Design - UI modeled after Obsidian Minimal Theme Aesthetic - Clean, distraction-free design Distraction-Free Layout - Focused reading/writing experience Sidebar & Navigation Left Sidebar - Note list and navigation panel Note List - Chronological view of all notes Recent Chats/Notes - Quick access to recent items Active Note Highlighting - Show currently open note Hover States - Interactive feedback on sidebar items Searchable Sidebar - Find notes in sidebar list Data Persistence Local Storage - Save notes locally in browser IndexedDB Support - Local database storage Auto-Save - Automatically save changes Session Persistence - Retain notes between sessions Vault Backup - Export entire vault for backup Keyboard Shortcuts Cmd/Ctrl + T - New tab Cmd/Ctrl + 1, 2, 3... - Switch between tabs Cmd/Ctrl + Click - Multi-select notes Shift + Click - Range selection Enter Key - Submit text/create note Shift + Enter - Newline in editor Visual Elements Chat Bubbles - Message-style note display (original AI version) Avatars - User/AI avatars for messages Icons - FontAwesome icons throughout UI Color Coding - Different colors for user/AI messages (original version) Active States - Visual indicators for selected/active items Hover Effects - Interactive visual feedback Smooth Transitions - CSS transitions for interactions Additional Features AI Assistant Drawer - Built-in AI chat (later removed per request) New Chat Button - Start new conversations with AI (removed) Chat History - Track conversation history (removed) Responsive Design - Works on different screen sizes Accessibility - Proper semantic HTML and ARIA labels Error Handling - "No file is open" empty states Modal Dialogs - Confirmation and dialog boxes Form Validation - Input validation in forms The app went through multiple iterations, with features being added (tabs, calendar, graph zoom, link parsing, tag filters) and some being removed (AI chat feature was stripped out per your request). The final version is a local-first, markdown-based note-taking application called "Vaukt" with graph visualization, tag-based organization, and a tabbed interface similar to Notion.

```

Response: <!DOCTYPE html>
<html lang="en" class="dark">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Vaukt - Local Markdown Knowledge Engine</title>
 <!-- Tailwind CSS CDN -->
 <script src="https://cdn.tailwindcss.com"></script>
 <script>
 tailwind.config = {
 darkMode: 'class',
 theme: {
 extend: {
 colors: {
 dark: {
 bg: '#181818',
 sidebar: '#141414',
 card: '#1e1e1e',
 border: '#2a2a2a',
 hover: '#262626',
 active: '#2c2c2c',
 text: '#e0e0e0',
 muted: '#888888',
 accent: '#a855f7'
 }
 }
 }
 }
 }
 </script>

```

```

 light: {
 bg: '#fcfcfc',
 sidebar: '#f4f4f5',
 card: '#ffffff',
 border: '#e4e4e7',
 hover: '#f1f1f3',
 active: '#e4e4e7',
 text: '#18181b',
 muted: '#71717a',
 accent: '#9333ea'
 }
 }
}

</script>
<!-- FontAwesome Icons -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" />
<!-- Marked.js for Markdown Parsing -->
<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<style>
 body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif; }
 .font-mono-custom { font-family: SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }
 ::-webkit-scrollbar { width: 6px; height: 6px; }
 ::-webkit-scrollbar-track { background: transparent; }
 ::-webkit-scrollbar-thumb { background: rgba(150, 150, 150, 0.2); border-radius: 3px; }
 ::-webkit-scrollbar-thumb:hover { background: rgba(150, 150, 150, 0.4); }
</style>
</head>
<body class="bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-none transition-colors duration-200">

 <!-- Top Window Drag / Tab Bar Header -->
 <header class="h-10 bg-dark-bg sidebar border-b border-dark-border flex items-center justify-between px-3">
 <div class="flex items-center space-x-2 overflow-x-auto no-scrollbar flex-1 mr-4" id="tabContainer">
 <!-- Tabs injected dynamically -->
 </div>

 <div class="flex items-center space-x-2 shrink-0">
 <button onclick="createNewTab()" title="New Tab (Cmd/Ctrl + T)" class="w-7 h-7 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-plus text-xs"></i>
 </button>
 <div class="h-4 w-[1px] bg-dark-border mx-1"></div>
 <button onclick="toggleGraphSidebar()" title="Toggle Graph View" id="graphToggleBtn" class="px-2">
 <i class="fa-solid fa-chart-network text-dark-accent"></i>
 Graph
 </button>
 <button onclick="exportMarkdown()" title="Export Note" class="px-2.5 py-1 rounded text-xs flex">
 <i class="fa-solid fa-download text-xs"></i>
 Export .md
 </button>
 <button onclick="exportVaultZip()" title="Batch Export (ZIP)" class="px-2.5 py-1 rounded text-xs">
 <i class="fa-solid fa-file-zipper text-xs text-amber-400"></i>
 Export Vault
 </button>
 </div>
 </header>

 <!-- Main Workspace Area -->
 <div class="flex-1 flex overflow-hidden relative">

 <!-- Left Sidebar: Notebook Navigator -->
 <aside class="w-72 bg-dark-bg sidebar border-r border-dark-border flex flex-col shrink-0 select-none">
 <!-- Sidebar Header & Search -->
 <div class="p-3 border-b border-dark-border space-y-2.5">
 <div class="flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <div class="w-5 h-5 rounded bg-dark-accent/20 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-book-bookmark text-xs"></i>
 </div>
 Notebook
 </div>
 <div class="flex items-center space-x-1">
 <button onclick="createNewNote()" title="New Note" class="w-6 h-6 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-file-circle-plus text-xs"></i>
 </button>
 <button onclick="openSettingsModal()" title="Settings" class="w-6 h-6 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-gear text-xs"></i>
 </button>
 </div>
 </div>
 </div>
 </aside>
 </div>

```

```

</div>

<!-- Search Bar -->
<div class="relative">
 <i class="fa-solid fa-search absolute left-2.5 top-2.5 text-xs text-dark-muted"></i>
 <input type="text" id="searchInput" placeholder="Quick open note... (Shift + K)" oninp
 <span class="absolute right-2 top-2 text-[10px] text-dark-muted bg-dark-border/50 px-1
</div>

<!-- Bulk Actions Toolbar (Multi-select) -->
<div id="bulkActionsBar" class="hidden flex items-center justify-between bg-dark-card bord
 0 selected</spa
 <div class="flex items-center space-x-1">
 <button onclick="bulkMovePrompt()" title="Move to Folder" class="p-1 rounded hover
 <button onclick="bulkDeleteSelected()" title="Delete Selected" class="p-1 rounded
 </div>
</div>
</div>

<!-- Folders / Categories Quick Header -->
<div class="px-3 py-2 border-b border-dark-border/50 flex items-center justify-between text-[1
 Folders
 <div class="flex items-center space-x-2">
 <label title="Import Folder (.txt/.md)" class="cursor-pointer hover:text-dark-text">
 <i class="fa-solid fa-file-import"></i>
 <input type="file" id="folderImportInput" multiple webkitdirectory directory class
 </label>
 <button onclick="createNewFolder()" title="New Folder" class="hover:text-dark-text"><i
 </div>
</div>
<div id="folderList" class="px-2 py-1.5 space-y-0.5 border-b border-dark-border max-h-36 overf
 <!-- Folders injected -->
</div>

<!-- Notes List Section -->
<div class="flex-1 overflow-y-auto p-2 space-y-1" id="notesListContainer" ondragover="event.pr
 <!-- Notes list injected dynamically -->
</div>

<!-- Bin / Trash Drop Zone & Tags Filter Footer -->
<div class="p-2 border-t border-dark-border bg-dark-card/50 space-y-2">
 <div id="binDropZone" ondragover="event.preventDefault()" ondrop="handleDropToBin(event)"
 <i class="fa-solid fa-trash-can text-xs"></i>
 Drag items here to Bin
 </div>
 <div>
 <div class="text-[10px] uppercase font-semibold text-dark-muted mb-1 flex items-center
 Tags
 <button onclick="openTagTemplatesModal()" class="text-dark-accent hover:underline
 </div>
 <div id="sidebarTagsList" class="flex flex-wrap gap-1 max-h-20 overflow-y-auto">
 <!-- Tags injected -->
 </div>
 </div>
</div>
</div>
</aside>

<!-- Center Editor Area -->
<main class="flex-1 flex flex-col bg-dark-bg overflow-hidden relative" id="editorMainArea">
 <div id="editorWrapper" class="flex-1 flex flex-col h-full overflow-hidden">
 <!-- Note Content Rendered Here -->
 </div>
</main>

<!-- Right Graph Sidebar -->
<aside id="graphSidebar" class="w-96 bg-dark-sidebar border-l border-dark-border flex flex-col shr
 <div class="p-3 border-b border-dark-border flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <i class="fa-solid fa-chart-network text-dark-accent text-xs"></i>
 Graph View
 </div>
 <div class="flex items-center space-x-2">
 <div class="bg-dark-card border border-dark-border rounded p-0.5 flex text-[10px]">
 <button onclick="setGraphMode('links')" id="graphModeLinks" class="px-2 py-0.5 rou
 <button onclick="setGraphMode('tags')" id="graphModeTags" class="px-2 py-0.5 round
 </div>
 <button onclick="toggleGraphSidebar()" class="w-6 h-6 flex items-center justify-center
 <i class="fa-solid fa-xmark text-xs"></i>
 </button>
 </div>
 </div>

```

```

 </div>
 </div>
 <!-- Graph Tag Checkbox Filters -->
 <div class="px-3 py-2 border-b border-dark-border bg-dark-card/30 flex flex-wrap gap-1 items-c
 Filter Tags:
 <!-- Injected tag checkboxes -->
 </div>
 <div class="flex-1 relative bg-dark-card/20 overflow-hidden flex items-center justify-center"
 <canvas id="graphCanvas" class="w-full h-full cursor-grab active:cursor-grabbing"></canvas>
 <div class="absolute bottom-3 right-3 flex items-center space-x-1 bg-dark-card/90 border b
 <button onclick="zoomGraph(1.2)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="zoomGraph(0.8)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="resetGraphZoom()" title="Reset View" class="w-6 h-6 rounded hover:bg-
 </div>
 </div>
 <div class="p-2 border-t border-dark-border bg-dark-card/50 text-xs text-dark-muted flex just
 0 nodes, 0 connections
 <button onclick="renderGraph()" class="text-[11px] text-dark-accent hover:underline">Refre
 </div>
</aside>

</div>

<!-- Modals & Context menus container -->
<div id="modalContainer"></div>

<!-- Application Logic -->
<script>
 // --- State Management ---
 let state = {
 notes: [
 {
 id: 'note-1',
 title: 'Welcome to Vaukt',
 content: `# Welcome to Vaukt\n\nYour local-first, markdown-based knowledge vault inspi
 folder: 'General',
 tags: ['idea', 'imported'],
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 },
 {
 id: 'note-2',
 title: 'Project Roadmap',
 content: `# Project Roadmap\n\nGoals for building out Vaukt:\n1. Enhance graph network
 folder: 'Academics',
 tags: ['project', 'business'],
 createdAt: new Date(Date.now() - 86400000).toISOString(),
 updatedAt: new Date(Date.now() - 86400000).toISOString()
 },
 {
 id: 'note-3',
 title: 'Quick Thoughts',
 content: `# Quick Thoughts\n\nRandom musings on UI design and minimal interfaces. Simp
 folder: 'General',
 tags: ['short_form'],
 createdAt: new Date(Date.now() - 172800000).toISOString(),
 updatedAt: new Date(Date.now() - 172800000).toISOString()
 }
],
 folders: ['General', 'Academics', 'Projects', 'Archive', 'Trash'],
 openTabs: ['note-1'],
 activeTabId: 'note-1',
 selectedNoteIds: [],
 searchQuery: '',
 selectedFolderFilter: null,
 selectedTagFilter: null,
 settings: {
 theme: 'dark',
 font: 'mono', // 'mono' or 'sans'
 },
 graph: {
 mode: 'links', // 'links' or 'tags'
 sidebarOpen: false,
 zoom: 1,
 panX: 0,
 panY: 0,
 filteredTags: [] // empty means all selected
 }
 };

```

```

// Local Storage persistence
function saveToLocalStorage() {
 localStorage.setItem('vaukt_state', JSON.stringify({
 notes: state.notes,
 folders: state.folders,
 openTabs: state.openTabs,
 activeTabId: state.activeTabId,
 settings: state.settings
 }));
}

function loadFromLocalStorage() {
 const saved = localStorage.getItem('vaukt_state');
 if (saved) {
 try {
 const parsed = JSON.parse(saved);
 state.notes = parsed.notes || state.notes;
 state.folders = parsed.folders || state.folders;
 state.openTabs = parsed.openTabs || state.openTabs;
 state.activeTabId = parsed.activeTabId || state.activeTabId;
 if (parsed.settings) state.settings = parsed.settings;
 } catch(e) { console.error(e); }
 }
}

// --- Initialization ---
window.addEventListener('DOMContentLoaded', () => {
 loadFromLocalStorage();
 applySettings();
 renderAll();
 setupEventListeners();
});

function applySettings() {
 const body = document.getElementById('appBody');
 if (state.settings.theme === 'light') {
 body.className = 'bg-light-bg text-light-text h-screen flex flex-col overflow-hidden select-
 } else {
 body.className = 'bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-
 }
}

function renderAll() {
 renderTabs();
 renderFolders();
 renderNotesList();
 renderTagsSidebar();
 renderActiveEditor();
 if (state.graph.sidebarOpen) {
 renderGraph();
 }
}

// --- Tab Management ---
function renderTabs() {
 const container = document.getElementById('tabContainer');
 container.innerHTML = '';

 if (state.openTabs.length === 0) {
 container.innerHTML = `No file is open</
 return;
 }

 state.openTabs.forEach((noteId, index) => {
 const note = state.notes.find(n => n.id === noteId);
 if (!note) return;

 const isActive = noteId === state.activeTabId;
 const tab = document.createElement('div');
 tab.className = `flex items-center space-x-2 px-3 py-1.5 rounded-t text-xs cursor-pointer
 isActive
 ? 'bg-dark-card border-dark-accent text-dark-text shadow-sm'
 : 'bg-dark-sidebar border-transparent text-dark-muted hover:bg-dark-hover hover:te
 `;
 tab.onclick = (e) => {
 if (e.target.closest('.close-tab')) return;
 switchTab(noteId);
 };
 });
}

```

```

 tab.innerHTML = `
 <i class="fa-regular fa-file-lines text-dark-accent/70 text-[11px]"></i>
 ${escapeHtml(note.title || 'Untitled Note')}
```

```

 if (folderName && !state.folders.includes(folderName)) {
 state.folders.push(folderName);
 saveToLocalStorage();
 renderFolders();
 }
 }

function renderNotesList() {
 const container = document.getElementById('notesListContainer');
 container.innerHTML = '';

 let filtered = state.notes.filter(n => {
 if (state.selectedFolderFilter && n.folder !== state.selectedFolderFilter) return false;
 if (state.selectedTagFilter && (!n.tags || !n.tags.includes(state.selectedTagFilter))) return false;
 if (state.searchQuery) {
 const q = state.searchQuery.toLowerCase();
 return (n.title && n.title.toLowerCase().includes(q)) || (n.content && n.content.toLowerCase().includes(q));
 }
 return true;
 });

 // Toggle bulk action bar visibility
 const bulkBar = document.getElementById('bulkActionsBar');
 if (state.selectedNoteIds.length > 0) {
 bulkBar.classList.remove('hidden');
 document.getElementById('selectedCountText').innerText = `${state.selectedNoteIds.length}`;
 } else {
 bulkBar.classList.add('hidden');
 }

 if (filtered.length === 0) {
 container.innerHTML = `<div class="p-4 text-center text-xs text-dark-muted italic">No note found</div>`;
 return;
 }

 filtered.forEach(note => {
 const isActive = note.id === state.activeTabId;
 const isSelected = state.selectedNoteIds.includes(note.id);

 const card = document.createElement('div');
 card.draggable = true;
 card.className = `p-2.5 rounded-lg border cursor-pointer transition relative group ${
 isActive
 ? 'bg-dark-card border-dark-accent shadow-sm'
 : isSelected
 ? 'bg-dark-active border-dark-border'
 : 'bg-dark-card/60 border-dark-border hover:bg-dark-card hover:border-dark-muted'
 }`;

 // Drag and drop setup for bin
 card.ondragstart = (e) => {
 if (!state.selectedNoteIds.includes(note.id)) {
 state.selectedNoteIds = [note.id];
 }
 e.dataTransfer.setData('text/plain', JSON.stringify(state.selectedNoteIds));
 };

 card.onclick = (e) => {
 if (e.metaKey || e.ctrlKey || e.shiftKey) {
 e.preventDefault();
 toggleSelectNote(note.id, e.shiftKey);
 } else {
 if (!state.openTabs.includes(note.id)) {
 state.openTabs.push(note.id);
 }
 switchTab(note.id);
 }
 };

 const snippet = note.content ? note.content.replace(/#.*?\n/g, '').trim().substring(0, 60) : '';
 const dateStr = new Date(note.updatedAt || note.createdAt).toLocaleDateString(undefined, {
 year: 'numeric', month: 'short', day: 'numeric'
 });

 card.innerHTML = `
 <div class="flex items-start justify-between mb-1">
 <div class="flex items-center space-x-2 truncate pr-2">
 <input type="checkbox" ${isSelected ? 'checked' : ''} onclick="event.stopPropagation()" />
 ${escapeHtml(note.content.substring(0, 60))}
 </div>
 ${dateStr}
 </div>
 `;
 });
}

```

```

 </div>
 <p class="text-[11px] text-dark-muted line-clamp-2 pl-5 mb-1.5">${escapeHtml(snippet)}
 <div class="flex items-center justify-between pl-5">
 <div class="flex flex-wrap gap-1">
 ${note.tags || []}.map(t => `<span class="text-[9px] bg-dark-bg text-dark-acc
 </div>
 <button onclick="event.stopPropagation(); deleteNote('${note.id}')" class="opacity
 </div>
 `;
 container.appendChild(card);
 });
}

function toggleSelectNote(id, isShift = false) {
 const idx = state.selectedNoteIds.indexOf(id);
 if (idx > -1) {
 state.selectedNoteIds.splice(idx, 1);
 } else {
 state.selectedNoteIds.push(id);
 }
 renderNotesList();
}

function deleteNote(id) {
 if (confirm('Are you sure you want to move this note to the Trash bin?')) {
 const note = state.notes.find(n => n.id === id);
 if (note) {
 note.folder = 'Trash';
 saveToLocalStorage();
 renderAll();
 }
 }
}

function bulkDeleteSelected() {
 if (state.selectedNoteIds.length === 0) return;
 if (confirm('Are you sure you want to move ${state.selectedNoteIds.length} notes to the Trash
 state.notes.forEach(n => {
 if (state.selectedNoteIds.includes(n.id)) {
 n.folder = 'Trash';
 }
 });
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
}

function bulkMovePrompt() {
 if (state.selectedNoteIds.length === 0) return;
 const targetFolder = prompt('Move ${state.selectedNoteIds.length} notes to folder:', 'General')
 if (targetFolder) {
 if (!state.folders.includes(targetFolder)) {
 state.folders.push(targetFolder);
 }
 state.notes.forEach(n => {
 if (state.selectedNoteIds.includes(n.id)) {
 n.folder = targetFolder;
 }
 });
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
 }
}

function handleDropToBin(e) {
 e.preventDefault();
 try {
 const data = JSON.parse(e.dataTransfer.getData('text/plain'));
 if (Array.isArray(data)) {
 state.notes.forEach(n => {
 if (data.includes(n.id)) {
 n.folder = 'Trash';
 }
 });
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
 }
 }
}

```

```

 }
 } catch(err) { console.error(err); }
}

// --- Folder & File Import (.txt / .md) ---
function importFolderFiles(e) {
 const files = e.target.files;
 if (!files || files.length === 0) return;

 let importedCount = 0;
 Array.from(files).forEach(file => {
 if (file.name.endsWith('.md') || file.name.endsWith('.txt')) {
 const reader = new FileReader();
 reader.onload = function(event) {
 const content = event.target.result;
 const title = file.name.replace(/\.([^.]+)$/, '');
 const newId = 'note-' + Date.now() + Math.random().toString(36).substring(2, 5));

 // Extract tags if any
 const hashtagMatches = content.match(/#[a-zA-Z0-9_]+/g);
 const tags = hashtagMatches ? hashtagMatches.map(t => t.substring(1)) : ['imported'];

 state.notes.unshift({
 id: newId,
 title: title,
 content: content,
 folder: 'General',
 tags: Array.from(new Set(tags)),
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 });

 importedCount++;
 saveToLocalStorage();
 renderAll();
 };
 reader.readAsText(file);
 }
 });
 alert(`Successfully processed file import.`);
}

function handleSearch(val) {
 state.searchQuery = val;
 renderNotesList();
}

function renderTagsSidebar() {
 const container = document.getElementById('sidebarTagsList');
 container.innerHTML = '';

 const allTags = {};
 state.notes.forEach(n => {
 if (n.tags) {
 n.tags.forEach(t => allTags[t] = (allTags[t] || 0) + 1);
 }
 });

 Object.keys(allTags).forEach(tag => {
 const isSelected = state.selectedTagFilter === tag;
 const badge = document.createElement('button');
 badge.className = `text-[10px] px-2 py-0.5 rounded border transition flex items-center spa
 isSelected
 ? 'bg-dark-accent text-white border-dark-accent font-medium'
 : 'bg-dark-card text-dark-muted border-dark-border hover:border-dark-accent/50 hov
 `;
 badge.onclick = () => {
 state.selectedTagFilter = isSelected ? null : tag;
 renderAll();
 };
 badge.innerHTML = `#${tag}(${allTags[tag]}
 container.appendChild(badge);
 });
}

// --- Active Note Editor & Properties ---
function renderActiveEditor() {
 const container = document.getElementById('editorWrapper');
 container.innerHTML = '';

```

```

if (!state.activeTabId || state.openTabs.length === 0) {
 container.innerHTML = `
 <div class="flex-1 flex flex-col items-center justify-center p-8 text-center text-dark">
 <div class="w-16 h-16 rounded-full bg-dark-card border border-dark-border flex ite
 <i class="fa-solid fa-file-circle-plus"></i>
 </div>
 <h2 class="text-base font-semibold text-dark-text mb-1">No file is open</h2>
 <p class="text-xs max-w-sm mb-4">Select a note from the sidebar or create a new ta
 <button onclick="createNewTab()" class="px-4 py-2 bg-dark-accent text-white rounde
 </div>
 `;
 return;
}

const note = state.notes.find(n => n.id === state.activeTabId);
if (!note) return;

const createdAt = note.createdAt ? note.createdAt.split('T')[0] : new Date().toISOString().s
const updatedAt = note.updatedAt ? new Date(note.updatedAt).toLocaleString() : 'Just now';

container.innerHTML = `
 <!-- Editor Top Toolbar -->
 <div class="h-11 border-b border-dark-border px-4 flex items-center justify-between shrink">
 <div class="flex items-center space-x-3">
 <i class="fa-solid fa-folder text-blue-400 m
 <div class="h-3 w-[1px] bg-dark-border"></div>
 <i class="fa-regular fa-clock mr-1"></i>${up
 </div>
 <div class="flex items-center space-x-2">
 <div class="flex bg-dark-card border border-dark-border rounded p-0.5 text-xs">
 <button onclick="switchEditMode('edit')" id="modeEditBtn" class="px-2.5 py-1 r
 <button onclick="switchEditMode('preview')" id="modePreviewBtn" class="px-2.5
 </div>
 </div>
 </div>

 <!-- Editor Body Area -->
 <div class="flex-1 flex flex-col overflow-y-auto p-6 max-w-4xl mx-auto w-full" id="editorC">
 <input type="text" id="noteTitleInput" value="${escapeHtml(note.title)}" oninput="upda

 <!-- Obsidian Style Properties Box -->
 <div class="bg-dark-card/40 border border-dark-border rounded-lg p-3 mb-4 space-y-2 te">
 <div class="text-[11px] font-semibold text-dark-muted uppercase tracking-wider mb-
 <div class="grid grid-cols-2 gap-4">
 <div class="flex items-center space-x-2 text-dark-muted">
 <i class="fa-regular fa-calendar w-4"></i>
 created
 <input type="date" value="${createdAt}" onchange="updateNoteDate('${note
 </div>
 <div class="flex items-center space-x-2 text-dark-muted">
 <i class="fa-solid fa-folder-tree w-4"></i>
 folder
 <select onchange="updateNoteFolder('${note.id}', this.value)" class="bg-da
 ${state.folders.map(f => `<option value="${f}" ${note.folder === f ? '
 </select>
 </div>
 </div>
 </div>
 <div class="flex flex-wrap items-center gap-1.5 pt-2 border-t border-dark-border/4">
 <i class="fa-solid fa-tags w-4
 ${note.tags || []}.map(t => `
 <span class="inline-flex items-center space-x-1 text-xs bg-dark-bg border
 #${t}
 <button onclick="removeTagFromNote('${note.id}', '${t}')" class="hover

 `).join('')}>
 <button onclick="addTagPrompt('${note.id}')" class="text-[11px] text-dark-mute
 </div>

 <!-- Main Textarea / Preview Container -->
 <div id="editorPane" class="flex-1 flex flex-col relative">
 <textarea id="noteTextarea" oninput="updateNoteContent(this.value)" onkeydown="han
 </div>
 <div id="previewPane" class="flex-1 hidden prose prose-invert max-w-none text-sm text-
 <!-- Markdown rendered output -->
 </div>

 <!-- Character Count Footer -->

```

```

 <div class="mt-2 pt-2 border-t border-dark-border/50 flex justify-between text-[11px]"
 Characters: ${note.content || ''}.length
 Shift + Enter for line break
 </div>
 </div>
 `;
}

function updateNoteTitle(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 note.title = val;
 note.updatedAt = new Date().toISOString();
 saveToLocalStorage();
 renderTabs();
 renderNotesList();
 }
}

function updateNoteContent(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 // Auto-detect & auto-link URLs in content
 let processedVal = val.replace(/(https?:\/\/[^\s]+)/g, (url) => {
 try {
 const hostname = new URL(url).hostname.replace('www.', '');
 return `[${hostname}](${url})`;
 } catch(e) { return url; }
 });

 note.content = processedVal;
 note.updatedAt = new Date().toISOString();

 // Auto-detect hashtags
 const hashtagMatches = processedVal.match(/#[a-zA-Z0-9_-]+/g);
 if (hashtagMatches) {
 const tags = hashtagMatches.map(tag => tag.substring(1));
 note.tags = Array.from(new Set(tags));
 }

 document.getElementById('charCountDisplay').innerText = `Characters: ${processedVal.length}`;
 saveToLocalStorage();
 renderNotesList();
 }
}

function updateNoteDate(noteId, dateStr) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 note.createdAt = dateStr;
 saveToLocalStorage();
 }
}

function updateNoteFolder(noteId, folderName) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 note.folder = folderName;
 saveToLocalStorage();
 renderAll();
 }
}

function handleEditorKeyDown(e) {
 if (e.key === 'Enter') {
 if (e.shiftKey) {
 // Allow normal newline
 return;
 }
 }
 if (e.key === '/') {
 // Slash menu trigger simulation for page links
 setTimeout(() => {
 const titleChoice = prompt('Slash Menu: Link to which note title?');
 if (titleChoice) {
 const textarea = document.getElementById('noteTextarea');
 const pos = textarea.selectionStart;
 const text = textarea.value;
 textarea.value = text.substring(0, pos) + `[${titleChoice}]` + text.substring(po

```

```

 updateNoteContent(textarea.value);
 }, 50);
}
}

function switchEditMode(mode) {
 const editBtn = document.getElementById('modeEditBtn');
 const previewBtn = document.getElementById('modePreviewBtn');
 const textarea = document.getElementById('noteTextarea');
 const previewPane = document.getElementById('previewPane');

 const note = state.notes.find(n => n.id === state.activeTabId);

 if (mode === 'edit') {
 editBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transition"
 previewBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition"
 textarea.classList.remove('hidden');
 previewPane.classList.add('hidden');
 } else {
 previewBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transiti
 editBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition";
 textarea.classList.add('hidden');
 previewPane.classList.remove('hidden');

 // Parse markdown and [[links]]
 let contentHtml = marked.parse(note.content || '');
 contentHtml = contentHtml.replace(/\[\[([.*?)]\]\]/g, (match, p1) => {
 return `<span onclick="openLinkedNote('${p1}')" class="text-dark-accent underline curs
 });
 previewPane.innerHTML = contentHtml;
 }
}

function openLinkedNote(title) {
 let target = state.notes.find(n => n.title.toLowerCase() === title.toLowerCase());
 if (!target) {
 const newId = 'note-' + Date.now();
 target = {
 id: newId,
 title: title,
 content: `# ${title}\n\nCreated via bidirectional link.`,
 folder: 'General',
 tags: ['linked'],
 createdAt: new Date().toISOString().split('T')[0],
 updatedAt: new Date().toISOString()
 };
 state.notes.unshift(target);
 }
 if (!state.openTabs.includes(target.id)) {
 state.openTabs.push(target.id);
 }
 switchTab(target.id);
}

function addTagPrompt(noteId) {
 const tag = prompt('Enter tag name (without #):');
 if (tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 if (!note.tags) note.tags = [];
 if (!note.tags.includes(tag.trim())) {
 note.tags.push(tag.trim());
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
 }
 }
}

function removeTagFromNote(noteId, tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note && note.tags) {
 note.tags = note.tags.filter(t => t !== tag);
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
}

```

```

}

function openTagTemplatesModal() {
 const container = document.getElementById('modalContainer');
 container.innerHTML = `
 <div class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-
 <div class="bg-dark-card border border-dark-border rounded-xl w-full max-w-md p-6 shad
 <div class="flex items-center justify-between pb-4 border-b border-dark-border mb-
 <h3 class="font-bold text-sm text-dark-text flex items-center space-x-2">
 <i class="fa-solid fa-tags text-dark-accent"></i>
 Tag Templates
 </h3>
 <button onclick="closeModal()" class="text-dark-muted hover:text-dark-text"><i
 </div>
 <p class="text-xs text-dark-muted mb-4">Apply pre-built production and writing tem
 <div class="space-y-2 text-xs">
 <button onclick="applyTagTemplate(['short_form', 'creative'])" class="w-full t
 Short Form Content
 #short_form #creative
 </button>
 <button onclick="applyTagTemplate(['long_form', 'project'])" class="w-full tex
 Long Form Project
 #long_form #project
 </button>
 <button onclick="applyTagTemplate(['business', 'strategy'])" class="w-full tex
 Business & Strategy
 #business #strategy
 </button>
 </div>
 </div>
 </div>
 `;
}

function applyTagTemplate(tagsArray) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 if (!note.tags) note.tags = [];
 tagsArray.forEach(t => {
 if (!note.tags.includes(t)) note.tags.push(t);
 });
 saveToLocalStorage();
 closeModal();
 renderAll();
 }
}

// --- Export Utilities ---
function exportMarkdown() {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (!note) return;
 const blob = new Blob([note.content], { type: 'text/markdown;charset=utf-8' });
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `${(note.title || 'untitled').replace(/[^\w-0-9]/gi, '_').toLowerCase()}.md`;
 a.click();
 URL.revokeObjectURL(url);
}

function exportVaultZip() {
 if (confirm('Are you sure you want to export your entire vault? This will download a backup an
 let combinedText = '';
 state.notes.forEach(n => {
 combinedText += `\n\n# ${n.title}\nFolder: ${n.folder}\nTags: ${n.tags.join(', '
 });
 const blob = new Blob([combinedText], { type: 'text/plain;charset=utf-8' });
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `vaukt_vault_backup_${Date.now()}.txt`;
 a.click();
 URL.revokeObjectURL(url);
 }
}

// --- Graph Visualization ---
function toggleGraphSidebar() {
 state.graph.sidebarOpen = !state.graph.sidebarOpen;
}

```

```

const sidebar = document.getElementById('graphSidebar');
const btn = document.getElementById('graphToggleBtn');
if (state.graph.sidebarOpen) {
 sidebar.classList.remove('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-active
 renderGraphTagFilters();
 renderGraph();
} else {
 sidebar.classList.add('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-card bo
}
}

function setGraphMode(mode) {
 state.graph.mode = mode;
 const linkBtn = document.getElementById('graphModeLinks');
 const tagBtn = document.getElementById('graphModeTags');
 if (mode === 'links') {
 linkBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transitio
 tagBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition"
 } else {
 tagBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transition
 linkBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition
 }
 renderGraph();
}

function zoomGraph(factor) {
 state.graph.zoom *= factor;
 renderGraph();
}

function resetGraphZoom() {
 state.graph.zoom = 1;
 state.graph.panX = 0;
 state.graph.panY = 0;
 renderGraph();
}

function renderGraphTagFilters() {
 const container = document.getElementById('graphTagFilters');
 container.innerHTML = `

 const allTags = new Set();
 state.notes.forEach(n => {
 if (n.tags) n.tags.forEach(t => allTags.add(t));
 });

 allTags.forEach(tag => {
 const isChecked = state.graph.filteredTags.length === 0 || state.graph.filteredTags.includ
 const lbl = document.createElement('label');
 lbl.className = "text-[10px] flex items-center space-x-1 bg-dark-card border border-dark-b
 lbl.innerHTML = `<input type="checkbox" ${isChecked ? 'checked' : ''} onchange="toggleGrap
 container.appendChild(lbl);
 });
}

function toggleGraphTagFilter(tag) {
 const idx = state.graph.filteredTags.indexOf(tag);
 if (idx > -1) {
 state.graph.filteredTags.splice(idx, 1);
 } else {
 state.graph.filteredTags.push(tag);
 }
 renderGraph();
}

function renderGraph() {
 const canvas = document.getElementById('graphCanvas');
 if (!canvas) return;
 const ctx = canvas.getContext('2d');
 canvas.width = canvas.parentElement.clientWidth;
 canvas.height = canvas.parentElement.clientHeight;

 ctx.clearRect(0, 0, canvas.width, canvas.height);

 let nodes = [];
 let edges = [];

```

```

let activeNotes = state.notes;
if (state.graph.filteredTags.length > 0) {
 activeNotes = state.notes.filter(n => n.tags && n.tags.some(t => state.graph.filteredTags.
}

if (state.graph.mode === 'links') {
 nodes = activeNotes.map(n => ({ id: n.id, label: n.title, val: 1 }));
 activeNotes.forEach(n => {
 if (!n.content) return;
 const links = n.content.match(/\[\[([.*?)]\]\]/g);
 if (links) {
 links.forEach(l => {
 const title = l.replace(/\[\[|\]\]/g, '').trim();
 const target = activeNotes.find(t => t.title.toLowerCase() === title.toLowerCase);
 if (target) {
 edges.push({ source: n.id, target: target.id });
 }
 });
 }
 });
} else {
 const tagMap = {};
 activeNotes.forEach(n => {
 if (n.tags) {
 n.tags.forEach(t => {
 if (!tagMap[t]) tagMap[t] = [];
 tagMap[t].push(n.id);
 });
 }
 });
 nodes = Object.keys(tagMap).map(t => ({ id: t, label: '#' + t, val: tagMap[t].length }));
 const tags = Object.keys(tagMap);
 for (let i = 0; i < tags.length; i++) {
 for (let j = i + 1; j < tags.length; j++) {
 const shared = tagMap[tags[i]].some(id => tagMap[tags[j]].includes(id));
 if (shared) {
 edges.push({ source: tags[i], target: tags[j] });
 }
 }
 }
}

document.getElementById('graphStats').innerText = `${nodes.length} nodes, ${edges.length} conn

const centerX = (canvas.width / 2) + state.graph.panX;
const centerY = (canvas.height / 2) + state.graph.panY;
const radius = Math.min(canvas.width, canvas.height) * 0.3 * state.graph.zoom;

const nodePositions = {};
nodes.forEach((node, i) => {
 const angle = (i / nodes.length) * 2 * Math.PI;
 nodePositions[node.id] = {
 x: centerX + radius * Math.cos(angle),
 y: centerY + radius * Math.sin(angle),
 label: node.label
 };
});

ctx.strokeStyle = 'rgba(168, 85, 247, 0.25)';
ctx.lineWidth = 1.5;
edges.forEach(edge => {
 const p1 = nodePositions[edge.source];
 const p2 = nodePositions[edge.target];
 if (p1 && p2) {
 ctx.beginPath();
 ctx.moveTo(p1.x, p1.y);
 ctx.lineTo(p2.x, p2.y);
 ctx.stroke();
 }
});

Object.keys(nodePositions).forEach(id => {
 const p = nodePositions[id];
 ctx.fillStyle = '#a855f7';
 ctx.beginPath();
 ctx.arc(p.x, p.y, 6 * Math.sqrt(state.graph.zoom), 0, 2 * Math.PI);
 ctx.fill();

 ctx.fillStyle = '#e0e0e0';

```

```

 ctx.font = '11px sans-serif';
 ctx.fillText(p.label, p.x + 10, p.y + 4);
 });
}

// --- Settings Modal ---
function openSettingsModal() {
 const container = document.getElementById('modalContainer');
 container.innerHTML = `
 <div class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-
 <div class="bg-dark-card border border-dark-border rounded-xl w-full max-w-md p-6 shad
 <div class="flex items-center justify-between pb-4 border-b border-dark-border mb-
 <h3 class="font-bold text-sm text-dark-text flex items-center space-x-2">
 <i class="fa-solid fa-gear text-dark-accent"></i>
 Vaukt Settings
 </h3>
 <button onclick="closeModal()" class="text-dark-muted hover:text-dark-text"><i
 </div>
 <div class="space-y-4">
 <div>
 <label class="text-xs font-medium text-dark-muted block mb-2">Appearance T
 <div class="grid grid-cols-2 gap-2">
 <button onclick="setTheme('dark')" class="p-3 rounded border text-xs f
 <i class="fa-solid fa-moon"></i>Dark Mode
 </button>
 <button onclick="setTheme('light')" class="p-3 rounded border text-xs
 <i class="fa-solid fa-sun"></i>Light Mode
 </button>
 </div>
 </div>
 </div>
 <div class="mt-6 pt-4 border-t border-dark-border flex justify-end">
 <button onclick="closeModal()" class="px-4 py-2 bg-dark-accent text-white roun
 </div>
 </div>
 `;
}

function setTheme(theme) {
 state.settings.theme = theme;
 saveToLocalStorage();
 applySettings();
 openSettingsModal();
}

function closeModal() {
 document.getElementById('modalContainer').innerHTML = '';
}

// --- Global Keyboard Shortcuts ---
function setupEventListeners() {
 window.addEventListener('keydown', (e) => {
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 't') {
 e.preventDefault();
 createNewTab();
 }
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 'k') {
 e.preventDefault();
 document.getElementById('searchInput').focus();
 }
 if ((e.metaKey || e.ctrlKey) && !isNaN(e.key) && e.key !== '0') {
 e.preventDefault();
 const index = parseInt(e.key) - 1;
 if (state.openTabs[index]) {
 switchTab(state.openTabs[index]);
 }
 }
 });
}
</script>
</body>
</html>

```

User prompt: make the default page for a new note simiar to this

```
Response: <!DOCTYPE html>
<html lang="en" class="dark">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Vaukt - Local Markdown Knowledge Engine</title>
 <!-- Tailwind CSS CDN -->
 <script src="https://cdn.tailwindcss.com"></script>
 <script>
 tailwind.config = {
 darkMode: 'class',
 theme: {
 extend: {
 colors: {
 dark: {
 bg: '#181818',
 sidebar: '#141414',
 card: '#1e1e1e',
 border: '#2a2a2a',
 hover: '#262626',
 active: '#2c2c2c',
 text: '#e0e0e0',
 muted: '#888888',
 accent: '#a855f7'
 },
 light: {
 bg: '#fcfcfc',
 sidebar: '#f4f4f5',
 card: '#ffffff',
 border: '#e4e4e7',
 hover: '#f1f1f3',
 active: '#e4e4e7',
 text: '#18181b',
 muted: '#71717a',
 accent: '#9333ea'
 }
 }
 }
 }
 }
 </script>
 <!-- Font Awesome Icons -->
 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.0/css/all.min.css">
 <!-- Marked.js for Markdown Parsing -->
 <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
 <style>
 body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif; }
 .font-mono-custom { font-family: SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace; }
 ::-webkit-scrollbar { width: 6px; height: 6px; }
 ::-webkit-scrollbar-track { background: transparent; }
 ::-webkit-scrollbar-thumb { background: rgba(150, 150, 150, 0.2); border-radius: 3px; }
 ::-webkit-scrollbar-thumb:hover { background: rgba(150, 150, 150, 0.4); }
 </style>
</head>
<body class="bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-none transition-colors">

 <!-- Top Window Drag / Tab Bar Header -->
 <header class="h-10 bg-dark-sidebar border-b border-dark-border flex items-center justify-between px-3">
 <div class="flex items-center space-x-2 overflow-x-auto no-scrollbar flex-1 mr-4" id="tabContainer">
 <!-- Tabs injected dynamically -->
 </div>

 <div class="flex items-center space-x-2 shrink-0">
 <button onclick="createNewTab()" title="New Tab (Cmd/Ctrl + T)" class="w-7 h-7 flex items-center justify-center rounded-md text-dark-text">
 <i class="fa-solid fa-plus text-xs"></i>
 </button>
 <div class="h-4 w-[1px] bg-dark-border mx-1"></div>
 <button onclick="toggleGraphSidebar()" title="Toggle Graph View" id="graphToggleBtn" class="px-2 py-1 rounded-md text-dark-text">
 <i class="fa-solid fa-chart-network text-dark-accent"></i>
 Graph
 </button>
 <button onclick="exportMarkdown()" title="Export Note" class="px-2.5 py-1 rounded text-xs flex">
 <i class="fa-solid fa-download text-xs"></i>
 Export .md
 </button>
 <button onclick="exportVaultZip()" title="Batch Export (ZIP)" class="px-2.5 py-1 rounded text-xs flex">
 <i class="fa-solid fa-file-zipper text-xs text-amber-400"></i>
 Export Vault
 </button>
 </div>
 </header>
```

```

 </div>
</header>

<!-- Main Workspace Area -->
<div class="flex-1 flex overflow-hidden relative">

 <!-- Left Sidebar: Notebook Navigator -->
 <aside class="w-72 bg-dark-sidebar border-r border-dark-border flex flex-col shrink-0 select-none"
 <!-- Sidebar Header & Search -->
 <div class="p-3 border-b border-dark-border space-y-2.5">
 <div class="flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <div class="w-5 h-5 rounded bg-dark-accent/20 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-book-bookmark text-xs"></i>
 </div>
 Notebook
 </div>
 <div class="flex items-center space-x-1">
 <button onclick="createNewNote()" title="New Note" class="w-6 h-6 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-file-circle-plus text-xs"></i>
 </button>
 <button onclick="openSettingsModal()" title="Settings" class="w-6 h-6 flex items-center justify-center text-dark-text">
 <i class="fa-solid fa-gear text-xs"></i>
 </button>
 </div>
 </div>
 <div class="relative">
 <i class="fa-solid fa-search absolute left-2.5 top-2.5 text-xs text-dark-muted"></i>
 <input type="text" id="searchInput" placeholder="Quick open note... (Shift + K)" oninput="" />

 <!-- Search Bar -->

 </div>

 <!-- Bulk Actions Toolbar (Multi-select) -->
 <div id="bulkActionsBar" class="hidden flex items-center justify-between bg-dark-card border border-dark-border p-1">
 0 selected
 <div class="flex items-center space-x-1">
 <button onclick="bulkMovePrompt()" title="Move to Folder" class="p-1 rounded hover:bg-dark-card">
 <i class="fa-solid fa-folder text-xs"></i>
 </button>
 <button onclick="bulkDeleteSelected()" title="Delete Selected" class="p-1 rounded hover:bg-dark-card">
 <i class="fa-solid fa-trash text-xs"></i>
 </button>
 </div>
 </div>
 </div>
 </div>

 <!-- Folders / Categories Quick Header -->
 <div class="px-3 py-2 border-b border-dark-border/50 flex items-center justify-between text-[10px]">
 Folders
 <div class="flex items-center space-x-2">
 <label title="Import Folder (.txt/.md)" class="cursor-pointer hover:text-dark-text">
 <i class="fa-solid fa-file-import"></i>
 <input type="file" id="folderImportInput" multiple webkitdirectory directory class="hidden" />
 </label>
 <button onclick="createNewFolder()" title="New Folder" class="hover:text-dark-text"><i class="fa-solid fa-folder-plus text-xs"></i></button>
 </div>
 </div>
 <div id="folderList" class="px-2 py-1.5 space-y-0.5 border-b border-dark-border max-h-36 overflow-y-auto">
 <!-- Folders injected -->
 </div>

 <!-- Notes List Section -->
 <div class="flex-1 overflow-y-auto p-2 space-y-1" id="notesListContainer" ondragover="event.preventDefault()">
 <!-- Notes list injected dynamically -->
 </div>

 <!-- Bin / Trash Drop Zone & Tags Filter Footer -->
 <div class="p-2 border-t border-dark-border bg-dark-card/50 space-y-2">
 <div id="binDropZone" ondragover="event.preventDefault()" ondrop="handleDropToBin(event)">
 <i class="fa-solid fa-trash-can text-xs"></i>
 Drag items here to Bin
 </div>
 <div>
 <div class="text-[10px] uppercase font-semibold text-dark-muted mb-1 flex items-center justify-between">
 Tags
 <button onclick="openTagTemplatesModal()" class="text-dark-accent hover:underline">
 <i class="fa-solid fa-tags text-xs"></i>
 </button>
 </div>
 <div id="sidebarTagsList" class="flex flex-wrap gap-1 max-h-20 overflow-y-auto">
 <!-- Tags injected -->
 </div>
 </div>
 </div>

```

```

 </div>
</aside>

<!-- Center Editor Area -->
<main class="flex-1 flex flex-col bg-dark-bg overflow-hidden relative" id="editorMainArea">
 <div id="editorWrapper" class="flex-1 flex flex-col h-full overflow-hidden">
 <!-- Note Content Rendered Here -->
 </div>
</main>

<!-- Right Graph Sidebar -->
<aside id="graphSidebar" class="w-96 bg-dark-sidebar border-l border-dark-border flex flex-col shr
 <div class="p-3 border-b border-dark-border flex items-center justify-between">
 <div class="flex items-center space-x-2">
 <i class="fa-solid fa-chart-network text-dark-accent text-xs"></i>
 Graph View
 </div>
 <div class="flex items-center space-x-2">
 <div class="bg-dark-card border border-dark-border rounded p-0.5 flex text-[10px]">
 <button onclick="setGraphMode('links')" id="graphModeLinks" class="px-2 py-0.5 rou
 <button onclick="setGraphMode('tags')" id="graphModeTags" class="px-2 py-0.5 round
 </div>
 <button onclick="toggleGraphSidebar()" class="w-6 h-6 flex items-center justify-center
 <i class="fa-solid fa-xmark text-xs"></i>
 </button>
 </div>
 </div>
 <!-- Graph Tag Checkbox Filters -->
 <div class="px-3 py-2 border-b border-dark-border bg-dark-card/30 flex flex-wrap gap-1 items-c
 Filter Tags:
 <!-- Injected tag checkboxes -->
 </div>
 <div class="flex-1 relative bg-dark-card/20 overflow-hidden flex items-center justify-center"
 <canvas id="graphCanvas" class="w-full h-full cursor-grab active:cursor-grabbing"></canvas>
 <div class="absolute bottom-3 right-3 flex items-center space-x-1 bg-dark-card/90 border b
 <button onclick="zoomGraph(1.2)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="zoomGraph(0.8)" class="w-6 h-6 rounded hover:bg-dark-hover flex items
 <button onclick="resetGraphZoom()" title="Reset View" class="w-6 h-6 rounded hover:bg-
 </div>
 </div>
 <div class="p-2 border-t border-dark-border bg-dark-card/50 text-xs text-dark-muted flex justi
 0 nodes, 0 connections
 <button onclick="renderGraph()" class="text-[11px] text-dark-accent hover:underline">Refre
 </div>
</aside>

</div>

<!-- Modals & Context menus container -->
<div id="modalContainer"></div>

<!-- Application Logic -->
<script>
 // --- State Management ---
 let state = {
 notes: [
 {
 id: 'note-1',
 title: 'richard hendrix',
 content: `richard hendrix\n\n# People you should know\n- Connected with [[Pied Piper]]
 folder: 'General',
 tags: ['people'],
 properties: {
 birthday: '01/01/1990',
 associates: 'Empty',
 affiliated: 'pied_piper',
 aliases: 'Empty'
 },
 createdAt: '2025-10-30T22:39:00.000Z',
 updatedAt: '2025-10-30T22:40:00.000Z'
 }
],
 folders: ['General', 'Academics', 'Projects', 'Archive', 'Trash'],
 openTabs: ['note-1'],
 activeTabId: 'note-1',
 selectedNoteIds: [],
 searchQuery: '',
 selectedFolderFilter: null,
 selectedTagFilter: null,

```

```

 settings: {
 theme: 'dark',
 font: 'mono',
 },
 graph: {
 mode: 'links',
 sidebarOpen: false,
 zoom: 1,
 panX: 0,
 panY: 0,
 filteredTags: []
 }
 };

// Local Storage persistence
function saveToLocalStorage() {
 localStorage.setItem('vaukt_state', JSON.stringify({
 notes: state.notes,
 folders: state.folders,
 openTabs: state.openTabs,
 activeTabId: state.activeTabId,
 settings: state.settings
 }));
}

function loadFromLocalStorage() {
 const saved = localStorage.getItem('vaukt_state');
 if (saved) {
 try {
 const parsed = JSON.parse(saved);
 state.notes = parsed.notes || state.notes;
 state.folders = parsed.folders || state.folders;
 state.openTabs = parsed.openTabs || state.openTabs;
 state.activeTabId = parsed.activeTabId || state.activeTabId;
 if (parsed.settings) state.settings = parsed.settings;
 } catch(e) { console.error(e); }
 }
}

// --- Initialization ---
window.addEventListener('DOMContentLoaded', () => {
 loadFromLocalStorage();
 applySettings();
 renderAll();
 setupEventListeners();
});

function applySettings() {
 const body = document.getElementById('appBody');
 if (state.settings.theme === 'light') {
 body.className = 'bg-light-bg text-light-text h-screen flex flex-col overflow-hidden select-
 } else {
 body.className = 'bg-dark-bg text-dark-text h-screen flex flex-col overflow-hidden select-
 }
}

function renderAll() {
 renderTabs();
 renderFolders();
 renderNotesList();
 renderTagsSidebar();
 renderActiveEditor();
 if (state.graph.sidebarOpen) {
 renderGraph();
 }
}

// --- Tab Management ---
function renderTabs() {
 const container = document.getElementById('tabContainer');
 container.innerHTML = '';

 if (state.openTabs.length === 0) {
 container.innerHTML = `No file is open</
 return;
 }

 state.openTabs.forEach((noteId) => {
 const note = state.notes.find(n => n.id === noteId);

```

```

 if (!note) return;

 const isActive = noteId === state.activeTabId;
 const tab = document.createElement('div');
 tab.className = `flex items-center space-x-2 px-3 py-1.5 rounded-t text-xs cursor-pointer
 ${isActive
 ? 'bg-dark-card border-dark-accent text-dark-text shadow-sm'
 : 'bg-dark-sidebar border-transparent text-dark-muted hover:bg-dark-hover hover:te
 }`;
 tab.onclick = (e) => {
 if (e.target.closest('.close-tab')) return;
 switchTab(noteId);
 };

 tab.innerHTML = `
 <i class="fa-regular fa-file-lines text-dark-accent/70 text-[11px]"></i>
 ${escapeHtml(note.title || 'Untitled Note')}
 <button onclick="closeTab('${noteId}')" class="close-tab w-4 h-4 rounded-full hover:bg
 <i class="fa-solid fa-xmark text-[10px]"></i>
 </button>
 `;
 container.appendChild(tab);
 });
}

function createNewTab() {
 const newId = 'note-' + Date.now();
 const nowIso = new Date().toISOString();
 const newNote = {
 id: newId,
 title: 'Untitled Note',
 content: '',
 folder: state.selectedFolderFilter || 'General',
 tags: ['people'],
 properties: {
 birthday: '01/01/1990',
 associates: 'Empty',
 affiliated: 'pied_piper',
 aliases: 'Empty'
 },
 createdAt: nowIso,
 updatedAt: nowIso
 };
 state.notes.unshift(newNote);
 state.openTabs.push(newId);
 state.activeTabId = newId;
 saveToLocalStorage();
 renderAll();
}

function createNewNote() {
 createNewTab();
}

function switchTab(noteId) {
 state.activeTabId = noteId;
 renderAll();
}

function closeTab(noteId) {
 const idx = state.openTabs.indexOf(noteId);
 if (idx > -1) {
 state.openTabs.splice(idx, 1);
 }
 if (state.activeTabId === noteId) {
 state.activeTabId = state.openTabs.length > 0 ? state.openTabs[state.openTabs.length - 1]
 }
 saveToLocalStorage();
 renderAll();
}

// --- Notebook Sidebar & Folders ---
function renderFolders() {
 const container = document.getElementById('folderList');
 container.innerHTML = '';

 const allDiv = document.createElement('div');
 allDiv.className = `flex items-center justify-between px-2 py-1 rounded text-xs cursor-pointer
 allDiv.onclick = () => { state.selectedFolderFilter = null; renderAll(); };

```

```

allDiv.innerHTML = `<div class="flex items-center space-x-2"><i class="fa-solid fa-folder-open
container.appendChild(allDiv);

state.folders.forEach(folder => {
 const count = state.notes.filter(n => n.folder === folder).length;
 const isSelected = state.selectedFolderFilter === folder;
 const fDiv = document.createElement('div');
 fDiv.className = `flex items-center justify-between px-2 py-1 rounded text-xs cursor-point
 fDiv.onclick = () => { state.selectedFolderFilter = folder; renderAll(); };

 let iconClass = folder === 'Trash' ? 'fa-trash text-red-400' : 'fa-folder text-blue-400';
 fDiv.innerHTML = `<div class="flex items-center space-x-2"><i class="fa-solid ${iconClass}
 container.appendChild(fDiv);
});
}

function createNewFolder() {
 const folderName = prompt('Enter new folder name:');
 if (folderName && !state.folders.includes(folderName)) {
 state.folders.push(folderName);
 saveToLocalStorage();
 renderFolders();
 }
}

function renderNotesList() {
 const container = document.getElementById('notesListContainer');
 container.innerHTML = '';

 let filtered = state.notes.filter(n => {
 if (state.selectedFolderFilter && n.folder !== state.selectedFolderFilter) return false;
 if (state.selectedTagFilter && (!n.tags || !n.tags.includes(state.selectedTagFilter))) ret
 if (state.searchQuery) {
 const q = state.searchQuery.toLowerCase();
 return (n.title && n.title.toLowerCase().includes(q)) || (n.content && n.content.toLow
 }
 return true;
 });

 const bulkBar = document.getElementById('bulkActionsBar');
 if (state.selectedNoteIds.length > 0) {
 bulkBar.classList.remove('hidden');
 document.getElementById('selectedCountText').innerText = `${state.selectedNoteIds.length}
 } else {
 bulkBar.classList.add('hidden');
 }

 if (filtered.length === 0) {
 container.innerHTML = `<div class="p-4 text-center text-xs text-dark-muted italic">No note
 return;
 }

 filtered.forEach(note => {
 const isActive = note.id === state.activeTabId;
 const isSelected = state.selectedNoteIds.includes(note.id);

 const card = document.createElement('div');
 card.draggable = true;
 card.className = `p-2.5 rounded-lg border cursor-pointer transition relative group ${
 isActive
 ? 'bg-dark-card border-dark-accent shadow-sm'
 : isSelected
 ? 'bg-dark-active border-dark-border'
 : 'bg-dark-card/60 border-dark-border hover:bg-dark-card hover:border-dark-mut
 }`;

 card.ondragstart = (e) => {
 if (!state.selectedNoteIds.includes(note.id)) {
 state.selectedNoteIds = [note.id];
 }
 e.dataTransfer.setData('text/plain', JSON.stringify(state.selectedNoteIds));
 };

 card.onclick = (e) => {
 if (e.metaKey || e.ctrlKey || e.shiftKey) {
 e.preventDefault();
 toggleSelectNote(note.id);
 } else {
 if (!state.openTabs.includes(note.id)) {

```

```

 state.openTabs.push(note.id);
 }
 switchTab(note.id);
}
};

const snippet = note.content ? note.content.substring(0, 60) + '...' : 'Empty note...';
const dateStr = new Date(note.updatedAt || note.createdAt).toLocaleDateString(undefined, {

card.innerHTML = `
 <div class="flex items-start justify-between mb-1">
 <div class="flex items-center space-x-2 truncate pr-2">
 <input type="checkbox" ${isSelected ? 'checked' : ''} onclick="event.stopPropa
 ${escapeHtml(note.
 </div>
 ${dateStr}
 </div>
 <p class="text-[11px] text-dark-muted line-clamp-2 pl-5 mb-1.5">${escapeHtml(snippet)}
 <div class="flex items-center justify-between pl-5">
 <div class="flex flex-wrap gap-1">
 ${note.tags || []}.map(t => `<span class="text-[9px] bg-dark-bg text-dark-acc
 </div>
 <button onclick="event.stopPropagation(); deleteNote('${note.id}')" class="opacity
 </div>
`;
 container.appendChild(card);
});
}

function toggleSelectNote(id) {
 const idx = state.selectedNoteIds.indexOf(id);
 if (idx > -1) {
 state.selectedNoteIds.splice(idx, 1);
 } else {
 state.selectedNoteIds.push(id);
 }
 renderNotesList();
}

function deleteNote(id) {
 if (confirm('Are you sure you want to move this note to the Trash bin?')) {
 const note = state.notes.find(n => n.id === id);
 if (note) {
 note.folder = 'Trash';
 saveToLocalStorage();
 renderAll();
 }
 }
}

function bulkDeleteSelected() {
 if (state.selectedNoteIds.length === 0) return;
 if (confirm(`Are you sure you want to move ${state.selectedNoteIds.length} notes to the Trash
 state.notes.forEach(n => {
 if (state.selectedNoteIds.includes(n.id)) {
 n.folder = 'Trash';
 }
 }));
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
}

function bulkMovePrompt() {
 if (state.selectedNoteIds.length === 0) return;
 const targetFolder = prompt('Move ${state.selectedNoteIds.length} notes to folder:', 'General')
 if (targetFolder) {
 if (!state.folders.includes(targetFolder)) {
 state.folders.push(targetFolder);
 }
 state.notes.forEach(n => {
 if (state.selectedNoteIds.includes(n.id)) {
 n.folder = targetFolder;
 }
 });
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
 }
}

```

```

 }
}

function handleDropToBin(e) {
 e.preventDefault();
 try {
 const data = JSON.parse(e.dataTransfer.getData('text/plain'));
 if (Array.isArray(data)) {
 state.notes.forEach(n => {
 if (data.includes(n.id)) {
 n.folder = 'Trash';
 }
 });
 state.selectedNoteIds = [];
 saveToLocalStorage();
 renderAll();
 }
 } catch(err) { console.error(err); }
}

function importFolderFiles(e) {
 const files = e.target.files;
 if (!files || files.length === 0) return;

 Array.from(files).forEach(file => {
 if (file.name.endsWith('.md') || file.name.endsWith('.txt')) {
 const reader = new FileReader();
 reader.onload = function(event) {
 const content = event.target.result;
 const title = file.name.replace(/\.[^/.]+$/, '');
 const newId = 'note-' + Date.now() + Math.random().toString(36).substring(2, 5);

 state.notes.unshift({
 id: newId,
 title: title,
 content: content,
 folder: 'General',
 tags: ['imported'],
 properties: { birthday: '01/01/1990', associates: 'Empty', affiliated: 'None',
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 });
 saveToLocalStorage();
 renderAll();
 };
 reader.readAsText(file);
 }
 });
 alert(`Successfully processed file import.`);
}

function handleSearch(val) {
 state.searchQuery = val;
 renderNotesList();
}

function renderTagsSidebar() {
 const container = document.getElementById('sidebarTagsList');
 container.innerHTML = '';

 const allTags = {};
 state.notes.forEach(n => {
 if (n.tags) {
 n.tags.forEach(t => allTags[t] = (allTags[t] || 0) + 1);
 }
 });

 Object.keys(allTags).forEach(tag => {
 const isSelected = state.selectedTagFilter === tag;
 const badge = document.createElement('button');
 badge.className = `text-[10px] px-2 py-0.5 rounded border transition flex items-center spa
 isSelected
 ? 'bg-dark-accent text-white border-dark-accent font-medium'
 : 'bg-dark-card text-dark-muted border-dark-border hover:border-dark-accent/50 hov
 `;
 badge.onclick = () => {
 state.selectedTagFilter = isSelected ? null : tag;
 renderAll();
 };
 });
}

```

```

 badge.innerHTML = `#${tag}(${allTags[tag]
 });
 container.appendChild(badge);
}

// --- Active Note Editor & Obsidian Metadata Properties Layout ---
function renderActiveEditor() {
 const container = document.getElementById('editorWrapper');
 container.innerHTML = '';

 if (!state.activeTabId || state.openTabs.length === 0) {
 container.innerHTML = `
 <div class="flex-1 flex flex-col items-center justify-center p-8 text-center text-dark"
 <div class="w-16 h-16 rounded-full bg-dark-card border border-dark-border flex ite
 <i class="fa-solid fa-file-circle-plus"></i>
 </div>
 <h2 class="text-base font-semibold text-dark-text mb-1">No file is open</h2>
 <p class="text-xs max-w-sm mb-4">Select a note from the sidebar or create a new ta
 <button onclick="createNewTab()" class="px-4 py-2 bg-dark-accent text-white rounde
 </div>
 `;
 return;
 }

 const note = state.notes.find(n => n.id === state.activeTabId);
 if (!note) return;

 const createdFormatted = note.createdAt ? note.createdAt.replace('T', ' ', '').substring(0, 19) :
 const updatedFormatted = note.updatedAt ? note.updatedAt.replace('T', ' ', '').substring(0, 19) :
 const props = note.properties || { birthday: '01/01/1990', associates: 'Empty', affiliated: 'p

 container.innerHTML = `
 <!-- Editor Top Bar Controls -->
 <div class="h-11 border-b border-dark-border px-4 flex items-center justify-between shrink
 <div class="flex items-center space-x-3">
 <i class="fa-solid fa-folder text-blue-400 m
 </div>
 <div class="flex items-center space-x-2">
 <div class="flex bg-dark-card border border-dark-border rounded p-0.5 text-xs">
 <button onclick="switchEditMode('edit')" id="modeEditBtn" class="px-2.5 py-1 r
 <button onclick="switchEditMode('preview')" id="modePreviewBtn" class="px-2.5
 </div>
 </div>
 </div>

 <!-- Main Note View matching Obsidian Reysu / Screenshot layout -->
 <div class="flex-1 flex flex-col overflow-y-auto p-8 max-w-4xl mx-auto w-full" id="editorC

 <!-- Note Title Header -->
 <input type="text" id="noteTitleInput" value="${escapeHtml(note.title)}" oninput="upda

 <!-- Obsidian Metadata Properties Box -->
 <div class="space-y-3 mb-6 text-xs text-dark-muted select-none">
 <div class="font-semibold text-dark-text tracking-wide mb-2">Properties</div>

 <!-- Created Property -->
 <div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-regular fa-clock w-
 <div class="flex items-center space-x-1.5 text-dark-text"><i class="fa-regular
 </div>

 <!-- Updated Property -->
 <div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-rotate w-4
 <div class="flex items-center space-x-1.5 text-dark-text"><i class="fa-regular
 </div>

 <!-- Tags Property -->
 <div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-tag w-4"><
 <div class="flex flex-wrap items-center gap-1.5 flex-1">
 ${note.tags || []}.map(t => `
 <span class="inline-flex items-center space-x-1 px-2 py-0.5 rounded bg
 ${t}
 <button onclick="removeTagFromNote('${note.id}', '${t}')" class="h

 `).join('')}
 <button onclick="addTagPrompt('${note.id}')" class="text-[11px] text-dark-
 </div>

```

```

</div>

<!-- Birthday Property -->
<div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-cake-candle">
 <div class="flex items-center space-x-1.5 text-dark-text">
 <i class="fa-regular fa-calendar text-[11px] text-dark-muted"></i>
 <input type="text" value="{escapeHtml(props.birthday || '')}" onchange="u
 </div>
 </div>

<!-- Associates Property -->
<div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-users w-4">
 <div class="flex items-center space-x-1.5 ${props.associates === 'Empty' ? 'te
 <input type="text" value="{escapeHtml(props.associates || 'Empty')}>" onch
 </div>
 </div>

<!-- Affiliated Property -->
<div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-network-wi
 <div class="flex flex-wrap items-center gap-1.5 flex-1">
 <span class="inline-flex items-center space-x-1 px-2 py-0.5 rounded bg-dar
 {escapeHtml(props.affiliated || 'None')}

 <button onclick="updatePropertyPrompt('${note.id}', 'affiliated')" class="
 </div>
 </div>

<!-- Aliases Property -->
<div class="flex items-center space-x-4 py-1 border-b border-dark-border/30">
 <div class="w-28 flex items-center space-x-2"><i class="fa-solid fa-signature
 <div class="flex items-center space-x-1.5 ${props.aliases === 'Empty' ? 'text-
 <input type="text" value="{escapeHtml(props.aliases || 'Empty')}>" onchang
 </div>
 </div>

<!-- Add Property Button -->
<div class="pt-1">
 <button onclick="addPropertyPrompt('${note.id}')" class="text-xs text-dark-mut
</div>

<!-- Computed Age / Info Callout Block mimicking screenshot -->
<div class="my-4 p-3 rounded bg-dark-card/60 border border-dark-border flex items-cent
 <i class="fa-solid fa-circle-info text-blue-400 text-sm"></i>
 current age: 35 years, 10 months, 1 day
</div>

<!-- Main Markdown Content Textarea / Preview -->
<div id="editorPane" class="flex-1 flex flex-col relative mt-2">
 <textarea id="noteTextarea" oninput="updateNoteContent(this.value)" placeholder="T
</div>
<div id="previewPane" class="flex-1 hidden prose prose-invert max-w-none text-sm text-
 <!-- Markdown preview -->
</div>

<!-- Embedded Linked References Section mimicking bottom of screenshot -->
<div class="mt-8 pt-4 border-t border-dark-border">
 <div class="flex items-center justify-between text-xs font-semibold text-dark-mute
 <div class="flex items-center space-x-2">
 <i class="fa-solid fa-table text-dark-accent"></i>
 {escapeHtml(note.title)} 1 result
 </div>
 <div class="flex items-center space-x-3 text-[11px] font-normal">
 Sort
 Filter
 Properties
 </div>
 </div>
 <div class="bg-dark-card/40 border border-dark-border rounded-lg p-3 text-xs space
 <div class="text-[10px] uppercase font-semibold text-dark-muted border-b borde
 <div>
 <span onclick="openLinkedNote('10-30-25 pied piper meeting')" class="text-
 </div>
 </div>
 </div>
</div>

```

```

 </div>
 `;
}

function updateNoteTitle(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 note.title = val;
 note.updatedAt = new Date().toISOString();
 saveToLocalStorage();
 renderTabs();
 renderNotesList();
 }
}

function updateNoteContent(val) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 note.content = val;
 note.updatedAt = new Date().toISOString();
 saveToLocalStorage();
 renderNotesList();
 }
}

function updateProperty(noteId, propKey, val) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 if (!note.properties) note.properties = {};
 note.properties[propKey] = val;
 note.updatedAt = new Date().toISOString();
 saveToLocalStorage();
 }
}

function updatePropertyPrompt(noteId, propKey) {
 const val = prompt(`Update ${propKey}:`, '');
 if (val !== null) {
 updateProperty(noteId, propKey, val);
 renderActiveEditor();
 }
}

function addPropertyPrompt(noteId) {
 const key = prompt('Enter new property name:');
 if (key) {
 const val = prompt(`Enter value for ${key}:`, '');
 if (val !== null) {
 updateProperty(noteId, key.trim(), val.trim());
 renderActiveEditor();
 }
 }
}

function addTagPrompt(noteId) {
 const tag = prompt('Enter tag name:');
 if (tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note) {
 if (!note.tags) note.tags = [];
 if (!note.tags.includes(tag.trim())) {
 note.tags.push(tag.trim());
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
 }
 }
}

function removeTagFromNote(noteId, tag) {
 const note = state.notes.find(n => n.id === noteId);
 if (note && note.tags) {
 note.tags = note.tags.filter(t => t !== tag);
 saveToLocalStorage();
 renderActiveEditor();
 renderTagsSidebar();
 }
}

```

```

function switchEditMode(mode) {
 const editBtn = document.getElementById('modeEditBtn');
 const previewBtn = document.getElementById('modePreviewBtn');
 const textarea = document.getElementById('noteTextarea');
 const previewPane = document.getElementById('previewPane');
 const note = state.notes.find(n => n.id === state.activeTabId);

 if (mode === 'edit') {
 editBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transition";
 previewBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition";
 textarea.classList.remove('hidden');
 previewPane.classList.add('hidden');
 } else {
 previewBtn.className = "px-2.5 py-1 rounded bg-dark-accent text-white font-medium transition";
 editBtn.className = "px-2.5 py-1 rounded text-dark-muted hover:text-dark-text transition";
 textarea.classList.add('hidden');
 previewPane.classList.remove('hidden');

 let contentHtml = marked.parse(note.content || '');
 contentHtml = contentHtml.replace(/\[\[([.*?)]\]\]/g, (match, p1) => {
 return `<span onclick="openLinkedNote('${p1}')" class="text-dark-accent underline cursor">`;
 });
 previewPane.innerHTML = contentHtml;
 }
}

function openLinkedNote(title) {
 let target = state.notes.find(n => n.title.toLowerCase() === title.toLowerCase());
 if (!target) {
 const newId = 'note-' + Date.now();
 target = {
 id: newId,
 title: title,
 content: `# ${title}\n\nConnected reference note.`,
 folder: 'General',
 tags: ['people'],
 properties: { birthday: '01/01/1990', associates: 'Empty', affiliated: 'None', aliases: '' },
 createdAt: new Date().toISOString(),
 updatedAt: new Date().toISOString()
 };
 state.notes.unshift(target);
 }
 if (!state.openTabs.includes(target.id)) {
 state.openTabs.push(target.id);
 }
 switchTab(target.id);
}

function openTagTemplatesModal() {
 const container = document.getElementById('modalContainer');
 container.innerHTML = `
 <div class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-50">
 <div class="bg-dark-card border border-dark-border rounded-xl w-full max-w-md p-6 shadow">
 <div class="flex items-center justify-between pb-4 border-b border-dark-border mb-4">
 <h3 class="font-bold text-sm text-dark-text flex items-center space-x-2">
 <i class="fa-solid fa-tags text-dark-accent"></i>
 Tag Templates
 </h3>
 <button onclick="closeModal()" class="text-dark-muted hover:text-dark-text"><i>Close</i>
 </button>
 </div>
 <div class="space-y-2 text-xs">
 <button onclick="applyTagTemplate(['people', 'contact'])" class="w-full text-l">
 Person / Contact Template
 #people #contact
 </button>
 </div>
 </div>
 </div>
 `;
}

function applyTagTemplate(tagsArray) {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (note) {
 if (!note.tags) note.tags = [];
 tagsArray.forEach(t => {
 if (!note.tags.includes(t)) note.tags.push(t);
 });
 }
}

```

```

 saveToLocalStorage();
 closeModal();
 renderAll();
 }
}

function exportMarkdown() {
 const note = state.notes.find(n => n.id === state.activeTabId);
 if (!note) return;
 const blob = new Blob([note.content], { type: 'text/markdown;charset=utf-8' });
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `${(note.title || 'untitled').replace(/[a-z0-9]/gi, '_').toLowerCase()}.md`;
 a.click();
 URL.revokeObjectURL(url);
}

function exportVaultZip() {
 if (confirm('Are you sure you want to export your entire vault?')) {
 let combinedText = '';
 state.notes.forEach(n => {
 combinedText += `\n\n# ${n.title}\nFolder: ${n.folder}\n\n${n.content}\n\n---`;
 });
 const blob = new Blob([combinedText], { type: 'text/plain;charset=utf-8' });
 const url = URL.createObjectURL(blob);
 const a = document.createElement('a');
 a.href = url;
 a.download = `vaukt_vault_backup_${Date.now()}.txt`;
 a.click();
 URL.revokeObjectURL(url);
 }
}

function toggleGraphSidebar() {
 state.graph.sidebarOpen = !state.graph.sidebarOpen;
 const sidebar = document.getElementById('graphSidebar');
 const btn = document.getElementById('graphToggleBtn');
 if (state.graph.sidebarOpen) {
 sidebar.classList.remove('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-active";
 renderGraphTagFilters();
 renderGraph();
 } else {
 sidebar.classList.add('hidden');
 btn.className = "px-2.5 py-1 rounded text-xs flex items-center space-x-1.5 bg-dark-card bo";
 }
}

function setGraphMode(mode) {
 state.graph.mode = mode;
 const linkBtn = document.getElementById('graphModeLinks');
 const tagBtn = document.getElementById('graphModeTags');
 if (mode === 'links') {
 linkBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transition";
 tagBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition";
 } else {
 tagBtn.className = "px-2.5 py-0.5 rounded bg-dark-accent text-white font-medium transition";
 linkBtn.className = "px-2.5 py-0.5 rounded text-dark-muted hover:text-dark-text transition";
 }
 renderGraph();
}

function zoomGraph(factor) {
 state.graph.zoom *= factor;
 renderGraph();
}

function resetGraphZoom() {
 state.graph.zoom = 1;
 state.graph.panX = 0;
 state.graph.panY = 0;
 renderGraph();
}

function renderGraphTagFilters() {
 const container = document.getElementById('graphTagFilters');
 container.innerHTML = `
 const allTags = new Set();

```

```

state.notes.forEach(n => { if (n.tags) n.tags.forEach(t => allTags.add(t)); });

allTags.forEach(tag => {
 const isChecked = state.graph.filteredTags.length === 0 || state.graph.filteredTags.includ
 const lbl = document.createElement('label');
 lbl.className = "text-[10px] flex items-center space-x-1 bg-dark-card border border-dark-b
 lbl.innerHTML = `<input type="checkbox" ${isChecked ? 'checked' : ''} onchange="toggleGrap
 container.appendChild(lbl);
});
}

function toggleGraphTagFilter(tag) {
 const idx = state.graph.filteredTags.indexOf(tag);
 if (idx > -1) {
 state.graph.filteredTags.splice(idx, 1);
 } else {
 state.graph.filteredTags.push(tag);
 }
 renderGraph();
}

function renderGraph() {
 const canvas = document.getElementById('graphCanvas');
 if (!canvas) return;
 const ctx = canvas.getContext('2d');
 canvas.width = canvas.parentElement.clientWidth;
 canvas.height = canvas.parentElement.clientHeight;
 ctx.clearRect(0, 0, canvas.width, canvas.height);

 let nodes = state.notes.map(n => ({ id: n.id, label: n.title, val: 1 }));
 let edges = [];

 state.notes.forEach(n => {
 if (!n.content) return;
 const links = n.content.match(/\[([.*?])\]/g);
 if (links) {
 links.forEach(l => {
 const title = l.replace(/\[|\]|\\|/g, '').trim();
 const target = state.notes.find(t => t.title.toLowerCase() === title.toLowerCase())
 if (target) {
 edges.push({ source: n.id, target: target.id });
 }
 });
 }
 });

 document.getElementById('graphStats').innerText = `${nodes.length} nodes, ${edges.length} conn

 const centerX = (canvas.width / 2) + state.graph.panX;
 const centerY = (canvas.height / 2) + state.graph.panY;
 const radius = Math.min(canvas.width, canvas.height) * 0.3 * state.graph.zoom;

 const nodePositions = {};
 nodes.forEach((node, i) => {
 const angle = (i / nodes.length) * 2 * Math.PI;
 nodePositions[node.id] = {
 x: centerX + radius * Math.cos(angle),
 y: centerY + radius * Math.sin(angle),
 label: node.label
 };
 });

 ctx.strokeStyle = 'rgba(168, 85, 247, 0.25)';
 ctx.lineWidth = 1.5;
 edges.forEach(edge => {
 const p1 = nodePositions[edge.source];
 const p2 = nodePositions[edge.target];
 if (p1 && p2) {
 ctx.beginPath();
 ctx.moveTo(p1.x, p1.y);
 ctx.lineTo(p2.x, p2.y);
 ctx.stroke();
 }
 });

 Object.keys(nodePositions).forEach(id => {
 const p = nodePositions[id];
 ctx.fillStyle = '#a855f7';
 ctx.beginPath();

```

```

 ctx.arc(p.x, p.y, 6 * Math.sqrt(state.graph.zoom), 0, 2 * Math.PI);
 ctx.fill();

 ctx.fillStyle = '#e0e0e0';
 ctx.font = '11px sans-serif';
 ctx.fillText(p.label, p.x + 10, p.y + 4);
 });
}

function openSettingsModal() {
 const container = document.getElementById('modalContainer');
 container.innerHTML = `
 <div class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-
 <div class="bg-dark-card border border-dark-border rounded-xl w-full max-w-md p-6 shad
 <div class="flex items-center justify-between pb-4 border-b border-dark-border mb-
 <h3 class="font-bold text-sm text-dark-text flex items-center space-x-2">
 <i class="fa-solid fa-gear text-dark-accent"></i>
 Vaukt Settings
 </h3>
 <button onclick="closeModal()" class="text-dark-muted hover:text-dark-text"><i
 </div>
 <div class="space-y-4">
 <div>
 <label class="text-xs font-medium text-dark-muted block mb-2">Appearance T
 <div class="grid grid-cols-2 gap-2">
 <button onclick="setTheme('dark')" class="p-3 rounded border text-xs f
 <i class="fa-solid fa-moon"></i>Dark Mode
 </button>
 <button onclick="setTheme('light')" class="p-3 rounded border text-xs
 <i class="fa-solid fa-sun"></i>Light Mode
 </button>
 </div>
 </div>
 <div class="mt-6 pt-4 border-t border-dark-border flex justify-end">
 <button onclick="closeModal()" class="px-4 py-2 bg-dark-accent text-white roun
 </div>
 </div>
 </div>
 `;
}

function setTheme(theme) {
 state.settings.theme = theme;
 saveToLocalStorage();
 applySettings();
 openSettingsModal();
}

function closeModal() {
 document.getElementById('modalContainer').innerHTML = '';
}

function setupEventListeners() {
 window.addEventListener('keydown', (e) => {
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 't') {
 e.preventDefault();
 createNewTab();
 }
 if ((e.metaKey || e.ctrlKey) && e.key.toLowerCase() === 'k') {
 e.preventDefault();
 document.getElementById('searchInput').focus();
 }
 if ((e.metaKey || e.ctrlKey) && !isNaN(e.key) && e.key !== '0') {
 e.preventDefault();
 const index = parseInt(e.key) - 1;
 if (state.openTabs[index]) {
 switchTab(state.openTabs[index]);
 }
 }
 });
}
</script>
</body>
</html>

```

User prompt: make it LOOK more like this

Response: <!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Adrian's Obsidian Vault - Obsidian v0.11.5</title>

<!-- JSZip & FileSaver libraries for zip export -->

<script src="https://cdnjs.cloudflare.com/ajax/libs/jszip/3.10.1/jszip.min.js"></script>

<script src="https://cdnjs.cloudflare.com/ajax/libs/FileSaver.js/2.0.5/FileSaver.min.js"></script>

<!-- FontAwesome Icons -->

<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css"

<style>

:root {

--bg-titlebar: #323232;

--bg-ribbon: #262626;

--bg-sidebar: #212121;

--bg-editor: #1e1e1e;

--bg-modal: #212121;

--bg-modal-nav: #1b1b1b;

--bg-card: #2a2a2a;

--bg-hover: #333333;

--bg-active: #383838;

--border-color: #313131;

--text-normal: #dcdde;

--text-muted: #969696;

--text-faint: #6e6e6e;

--accent-purple: #7055e8;

--accent-purple-light: #9373ed;

--accent-purple-glow: rgba(112, 85, 232, 0.2);

--tag-bg: rgba(147, 115, 237, 0.12);

--font-ui: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;

--font-mono: "SF Mono", "Source Code Pro", Menlo, Monaco, Consolas, monospace;

}

body.light-theme {

--bg-titlebar: #e3e3e3;

--bg-ribbon: #f0f0f0;

--bg-sidebar: #f7f7f7;

--bg-editor: #ffffff;

--bg-modal: #f7f7f7;

--bg-modal-nav: #ecec;

--bg-card: #e5e5e5;

--bg-hover: #e0e0e0;

--bg-active: #d6d6d6;

--border-color: #dcdcdc;

--text-normal: #2c2c2c;

--text-muted: #707070;

--text-faint: #9a9a9a;

--accent-purple: #6842e8;

--accent-purple-light: #7055e8;

--accent-purple-glow: rgba(104, 66, 232, 0.15);

--tag-bg: rgba(104, 66, 232, 0.08);

}

\* { box-sizing: border-box; margin: 0; padding: 0; user-select: none; }

body { background-color: var(--bg-titlebar); color: var(--text-normal); height: 100vh; display: fl

.window-header { height: 38px; background: var(--bg-titlebar); border-bottom: 1px solid var(--bord

.window-controls { display: flex; gap: 8px; align-items: center; }

.mac-btn { width: 12px; height: 12px; border-radius: 50%; }

.mac-close { background: #ff5f56; }

.mac-min { background: #ffbd2e; }

.mac-max { background: #27c93f; }

.window-title { color: var(--text-muted); font-size: 12px; font-family: var(--font-ui); display: f

.app-container { flex: 1; display: flex; overflow: hidden; }

.ribbon { width: 44px; background: var(--bg-ribbon); border-right: 1px solid var(--border-color);

.ribbon-btn { width: 32px; height: 32px; border-radius: 4px; display: flex; align-items: center; j

.ribbon-btn:hover { background: var(--bg-hover); color: var(--text-normal); }

.ribbon-btn.active { color: var(--accent-purple-light); background: var(--accent-purple-glow); }

.ribbon-bottom { margin-top: auto; display: flex; flex-direction: column; gap: 10px; }

.sidebar { width: 260px; background: var(--bg-sidebar); border-right: 1px solid var(--border-color

```

.sidebar-header { padding: 10px 12px 6px 12px; display: flex; justify-content: space-between; align-items: center; }
.vault-name { font-weight: 600; font-family: var(--font-ui); font-size: 12px; color: var(--text-normal); }
.sidebar-actions { display: flex; gap: 6px; color: var(--text-muted); }
.sidebar-actions i { cursor: pointer; padding: 3px; border-radius: 3px; font-size: 12px; }
.sidebar-actions i:hover { background: var(--bg-hover); color: var(--text-normal); }

.search-container { padding: 0 8px 6px 8px; }
.search-trigger-input { width: 100%; background: var(--bg-ribbon); border: 1px solid var(--border-color); }

.view-tab-selector { display: flex; margin: 0 8px 6px 8px; background: var(--bg-ribbon); border: 1px solid var(--border-color); }
.view-tab-btn { flex: 1; border: none; background: transparent; color: var(--text-muted); font-size: 12px; }
.view-tab-btn.active { background: var(--bg-card); color: var(--text-normal); font-weight: 600; }
.view-tab-btn.drag-over { background: rgba(255, 95, 86, 0.25) !important; color: #fff5f56 !important; }

.notebook-navigator-list { flex: 1; overflow-y: auto; padding: 4px 6px; display: flex; flex-direction: column; }
.note-card { padding: 6px 10px; border-radius: 4px; background: transparent; cursor: pointer; display: flex; justify-content: space-between; }
.note-card:hover { background: var(--bg-hover); color: var(--text-normal); }
.note-card.active { background: var(--bg-active); color: var(--text-normal); }
.note-card.selected { background: var(--accent-purple-glow); border-color: var(--accent-purple-light); }
.note-card-title { font-weight: 400; font-size: 12px; color: var(--text-normal); white-space: nowrap; }
.note-card-snippet { display: none; }
.note-card-meta { font-size: 11px; color: var(--text-muted); display: flex; gap: 6px; align-items: center; }
.btn-card-bin { position: absolute; right: 6px; top: 6px; color: var(--text-faint); opacity: 0; transition: opacity 0.2s; }
.note-card:hover .btn-card-bin { opacity: 1; }
.btn-card-bin:hover { color: #fff5f56; background: var(--bg-card); }

/* Tree view mockup headers */
.tree-folder { font-size: 12px; color: var(--text-muted); padding: 4px 8px; cursor: pointer; display: flex; align-items: center; }
.tree-folder:hover { color: var(--text-normal); background: var(--bg-hover); border-radius: 4px; }
.tree-sub { padding-left: 16px; display: flex; flex-direction: column; gap: 1px; }

#dragBadge { position: absolute; top: -1000px; left: -1000px; background: var(--accent-purple); color: white; padding: 2px 5px; font-size: 10px; }

.selection-bar { padding: 6px 10px; background: var(--accent-purple-glow); border-bottom: 1px solid var(--border-color); }
.selection-bar button { background: var(--bg-card); border: 1px solid var(--border-color); color: var(--text-normal); padding: 2px 5px; }

.tags-pane { border-top: 1px solid var(--border-color); padding: 8px 10px; max-height: 120px; overflow: auto; }
.tag-pill-list { display: flex; flex-wrap: wrap; gap: 4px; margin-top: 4px; }
.tag-pill { background: var(--tag-bg); color: var(--accent-purple-light); padding: 2px 6px; border-radius: 3px; }
.tag-pill.active { background: var(--accent-purple); color: #fff; }

.workspace { flex: 1; display: flex; flex-direction: column; background: var(--bg-editor); overflow: hidden; }

/* Obsidian v0.11 Top Bar */
.tab-bar { height: 35px; background: var(--bg-sidebar); border-bottom: 1px solid var(--border-color); }
.tab-left { display: flex; align-items: center; gap: 4px; overflow-x: auto; flex: 1; }

.tab { height: 27px; padding: 0 10px; background: transparent; border-radius: 4px; display: flex; align-items: center; }
.tab.active { background: var(--bg-editor); color: var(--text-normal); border-color: var(--border-color); }
.tab:hover:not(.active) { background: var(--bg-hover); color: var(--text-normal); }
.tab-title { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; flex: 1; pointer-events: none; }
.tab-close { opacity: 0; padding: 2px; border-radius: 3px; font-size: 10px; color: var(--text-faint); }
.tab:hover .tab-close, .tab.active .tab-close { opacity: 1; }
.tab-close:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-new-btn { cursor: pointer; padding: 4px 8px; color: var(--text-muted); border-radius: 4px; display: flex; align-items: center; }
.tab-new-btn:hover { background: var(--bg-hover); color: var(--text-normal); }

.tab-actions { display: flex; gap: 4px; align-items: center; margin-left: 8px; color: var(--text-muted); }
.editor-action-btn { background: transparent; border: none; color: var(--text-muted); width: 26px; height: 26px; }
.editor-action-btn:hover, .editor-action-btn.active { color: var(--text-normal); background: var(--bg-hover); }

/* Secondary file toolbar header */
.file-toolbar { height: 32px; border-bottom: 1px solid var(--border-color); display: flex; align-items: center; }
.file-toolbar-left { display: flex; gap: 12px; align-items: center; flex: 1; }
.file-toolbar-left i { cursor: pointer; }
.file-toolbar-left i:hover { color: var(--text-normal); }
.file-toolbar-right { display: flex; gap: 12px; align-items: center; }
.file-toolbar-right i { cursor: pointer; padding: 4px; border-radius: 3px; }
.file-toolbar-right i:hover { background: var(--bg-hover); color: var(--text-normal); }

.workspace-content { flex: 1; display: flex; overflow: hidden; position: relative; }

.empty-state { display: none; flex: 1; align-items: center; justify-content: center; flex-direction: column; }
.empty-state i { font-size: 48px; margin-bottom: 16px; opacity: 0.2; }

.editor-pane { flex: 1; padding: 24px 48px; overflow-y: auto; display: flex; flex-direction: column; }
.properties-block { display: none; }

```

```

.note-title-input { font-size: 1.5rem; font-weight: 700; color: var(--text-normal); background: tr
.note-body-textarea { flex: 1; background: transparent; border: none; outline: none; color: var(--

/* Template Tag Bar */
.template-tags-bar { display: none; }

/* Right Backlinks Pane (Obsidian v0.11 style) */
.graph-pane { width: 300px; background: var(--bg-sidebar); border-left: 1px solid var(--border-col
.graph-pane.collapsed { display: none; }
.graph-header { padding: 8px 12px; border-bottom: 1px solid var(--border-color); display: flex; al
.graph-mode-selector { display: flex; gap: 4px; align-items: center; }
.graph-mode-btn { border: none; background: transparent; color: var(--text-muted); font-size: 12px
.graph-mode-btn.active { color: var(--text-normal); }
.graph-canvas-container { flex: 1; padding: 12px; overflow-y: auto; font-size: 12px; color: var(--
#graphCanvas { display: none; }
.graph-zoom-controls { display: none; }
.graph-tag-checkboxes { display: none; }

/* Outline Tree Styling in Backlinks Pane */
.outline-tree { display: flex; flex-direction: column; gap: 4px; }
.outline-item { cursor: pointer; color: var(--text-muted); padding: 2px 4px; border-radius: 3px; f
.outline-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.outline-h1 { font-weight: 600; color: var(--text-normal); }
.outline-h2 { padding-left: 12px; }
.outline-h3 { padding-left: 24px; color: var(--text-faint); }

.calendar-pane { padding: 12px; display: flex; flex-direction: column; gap: 12px; }
.calendar-header { display: flex; justify-content: space-between; align-items: center; font-weight
.calendar-grid { display: grid; grid-template-columns: repeat(7, 1fr); gap: 4px; text-align: cente
.cal-day-label { font-size: 10px; color: var(--text-faint); padding-bottom: 4px; }
.cal-cell { font-size: 11px; padding: 6px 0; border-radius: 4px; cursor: pointer; color: var(--tex
.cal-cell:hover { background: var(--bg-hover); color: var(--text-normal); }
.cal-cell.today { border: 1px solid var(--accent-purple-light); font-weight: bold; color: var(--te
.cal-cell.has-note::after { content: ''; position: absolute; bottom: 2px; left: 50%; transform: tr

/* Modals & Menus */
.modal-overlay { position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; background: rgba(0
.modal-overlay.active { display: flex; }
.obsidian-settings-card { width: 840px; height: 560px; max-width: 95vw; max-height: 90vh; backgrou
.settings-nav { width: 220px; background: var(--bg-modal-nav); border-right: 1px solid var(--borde
.settings-nav-section { display: flex; flex-direction: column; gap: 2px; }
.settings-nav-header { font-size: 10px; font-weight: 700; color: var(--text-faint); padding: 4px 8
.settings-nav-item { padding: 6px 10px; border-radius: 4px; color: var(--text-muted); cursor: poin
.settings-nav-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.settings-nav-item.active { background: var(--bg-active); color: var(--text-normal); font-weight:
.settings-content { flex: 1; padding: 24px 32px; overflow-y: auto; display: flex; flex-direction:
.settings-section-title { font-size: 16px; font-weight: 700; color: var(--text-normal); border-bot
.obs-setting-item { display: flex; justify-content: space-between; align-items: flex-start; paddin
.obs-setting-info { display: flex; flex-direction: column; gap: 4px; max-width: 380px; }
.obs-setting-name { font-size: 12px; font-weight: 600; color: var(--text-normal); }
.obs-setting-desc { font-size: 11px; color: var(--text-muted); line-height: 1.4; }
.obs-setting-control { display: flex; align-items: center; gap: 8px; }
select.obs-select { background: var(--bg-card); border: 1px solid var(--border-color); color: var(
.obs-btn { background: var(--bg-card); border: 1px solid var(--border-color); color: var(--text-no
.obs-btn:hover { background: var(--bg-hover); }
.obs-btn-danger { background: #ff5f56; color: #fff; border: none; }

.search-modal { width: 640px; max-width: 90vw; background: var(--bg-sidebar); border: 1px solid va
.modal-search-header { padding: 14px; border-bottom: 1px solid var(--border-color); display: flex;
.modal-search-input { flex: 1; background: transparent; border: none; outline: none; color: var(--
.modal-search-body { display: flex; height: 340px; }
.modal-results-list { flex: 1; overflow-y: auto; padding: 8px; border-right: 1px solid var(--borde
.modal-result-item { padding: 8px 10px; border-radius: 6px; cursor: pointer; display: flex; align-
.modal-result-item:hover { background: var(--bg-hover); color: var(--text-normal); }
.modal-preview-pane { width: 280px; background: var(--bg-editor); padding: 14px; overflow-y: auto;

.slash-menu { position: absolute; background: var(--bg-card); border: 1px solid var(--border-color
.slash-item { padding: 8px 12px; display: flex; align-items: center; gap: 8px; color: var(--text-n
.slash-item:hover { background: var(--accent-purple); color: #ffffff; }
</style>
</head>
<body>

<input type="file" id="folderImporter" webkitdirectory directory multiple style="display: none;" oncha
<input type="file" id="icsImporter" accept=".ics" style="display: none;" onchange="handleImportICS(eve
<div id="dragBadge">1 Note Selected</div>

<!-- Window Titlebar matching v0.11.5 screenshot -->
<header class="window-header">

```

```

<div class="window-controls">
 <div class="mac-btn mac-close"></div>
 <div class="mac-btn mac-min"></div>
 <div class="mac-btn mac-max"></div>
</div>
<div class="window-title">Adrian's Obsidian Vault - Obsidian v0.11.5</div>
<div style="width: 48px;"></div>
</header>

<div class="app-container">
 <!-- Ribbon matching screenshot icons -->
 <nav class="ribbon">
 <div class="ribbon-btn active" id="ribbonNavBtn" title="Notebook Navigator" onclick="switchSideBarView('notes')"></div>
 <div class="ribbon-btn" id="ribbonSearchBtn" title="Quick Search (Shift x2)" onclick="openSearchModal()"><i class="fa-solid fa-magnifying-glass"></i></div>
 <div class="ribbon-btn active" id="ribbonGraphBtn" title="Toggle Outline / Backlinks" onclick="toggleOutline()"></div>
 <div class="ribbon-btn" id="ribbonCalBtn" title="Apple Calendar Sync" onclick="switchSidebarView('calendar')"></div>
 <div class="ribbon-bottom">
 <div class="ribbon-btn" title="Help" onclick="openSearchModal()"><i class="fa-regular fa-question-circle"></i></div>
 <div class="ribbon-btn" title="Settings" onclick="openSettingsModal()"><i class="fa-solid fa-gear"></i></div>
 </div>
 </nav>

 <!-- Sidebar Pane matching v0.11.5 Tree Structure -->
 <aside class="sidebar">
 <div class="sidebar-header">
 Adrian's Obsidian Vault
 <div class="sidebar-actions">
 <i class="fa-solid fa-file-circle-plus" title="New Note" onclick="createNewNote()"></i>
 <i class="fa-regular fa-folder" title="New Folder" onclick="triggerFolderImport()"></i>
 <i class="fa-solid fa-arrow-down-short-wide" title="Sort Order"></i>
 </div>
 </div>

 <div class="view-tab-selector" style="display: none;">
 <button class="view-tab-btn active" id="tabNotesBtn" onclick="switchSidebarView('notes')"></button>
 <button class="view-tab-btn" id="tabBinBtn" onclick="switchSidebarView('bin')"></button>
 <div style="margin-left: 10px;">
 <i class="fa-solid fa-trash-can"></i> Bin
 </div>
 </div>

 <!-- Multi-Selection Bar -->
 <div class="selection-bar" id="selectionBar" style="display: none;">
 0 selected
 <div style="display:flex; gap:6px;">
 <button onclick="binSelectedNotes()"><i class="fa-solid fa-trash-can"></i> Move to Bin</button>
 <button onclick="clearSelection()">Clear</button>
 </div>
 </div>

 <div class="search-container" id="searchBoxContainer" style="display: none;">
 <div class="search-trigger-input" onclick="openSearchModal()">
 <i class="fa-solid fa-magnifying-glass" style="margin-right:6px;"></i> Quick open
 Shift x2
 </div>
 </div>

 <!-- Tree View & Note List -->
 <div class="notebook-navigator-list" id="navigatorList"></div>

 <!-- Calendar View -->
 <div class="calendar-pane" id="calendarView" style="display: none;">
 <div class="calendar-header">
 August 2026
 <div style="display:flex; gap:6px;">
 <i class="fa-solid fa-chevron-left" style="cursor:pointer;" onclick="changeMonth(-1)"></i>
 <i class="fa-solid fa-chevron-right" style="cursor:pointer;" onclick="changeMonth(1)"></i>
 </div>
 </div>
 <div class="calendar-grid" id="calendarGrid"></div>

 <div style="margin-top:12px; border-top:1px solid var(--border-color); padding-top:12px;">
 <div style="font-size:10px; font-weight:700; color:var(--text-faint); margin-bottom:8px;">
 <button class="obs-btn" style="width:100%; margin-bottom:6px;" onclick="triggerICSImport()">Import ICS...</button>
 <button class="obs-btn" style="width:100%;" onclick="exportCalendarICS()"><i class="fa-solid fa-download"></i> Export Calendar</button>
 </div>
 </div>
 </div>
 </aside>

```

```

<!-- Tags Drawer -->
<div class="tags-pane" id="tagsPane" style="display: none;">
 <div style="font-size:10px; font-weight:700; color:var(--text-faint); text-transform:upper
 <div class="tag-pill-list" id="tagPillList"></div>
 </div>
</aside>

<!-- Workspace -->
<main class="workspace">
 <!-- TAB BAR matching v0.11.5 -->
 <div class="tab-bar">
 <div class="tab-left" id="tabContainer">
 <!-- Tabs injected dynamically -->
 </div>
 <div class="tab-new-btn" title="Open new tab" onclick="createNewNote()"><i class="fa-solid

 <div class="tab-actions">
 <button class="editor-action-btn" title="Toggle Split"><i class="fa-solid fa-columns">
 <button class="editor-action-btn" title="Close Current Tab"><i class="fa-solid fa-xmar
 <button class="editor-action-btn" title="More Options"><i class="fa-solid fa-ellipsis-
 </div>
</div>

<!-- Secondary File Toolbar (matches v0.11.5 screenshot top toolbar inside editor) -->
<div class="file-toolbar">
 <div class="file-toolbar-left">
 <i class="fa-solid fa-chevron-left" title="Back"></i>
 <i class="fa-solid fa-chevron-right" title="Forward"></i>
 Test
 </div>
 <div class="file-toolbar-right">
 <i class="fa-regular fa-file-lines" title="Reading/Editing view"></i>
 <i class="fa-solid fa-thumbtack" title="Pin"></i>
 <i class="fa-solid fa-ellipsis-vertical" title="More options"></i>
 </div>
</div>

<div class="workspace-content">
 <!-- EMPTY STATE -->
 <div class="empty-state" id="emptyState">
 <i class="fa-solid fa-leaf"></i>
 <div>No file is open</div>
 <div style="margin-top: 8px; font-size: 11px;">Create a new note or select one from th
 </div>

 <!-- EDITOR PANE -->
 <div class="editor-pane" id="editorPane">
 <input type="text" id="noteTitle" class="note-title-input" placeholder="Note Title..."

 <textarea id="noteContent" class="note-body-textarea" placeholder="Type note content..

 <div id="slashMenu" class="slash-menu">
 <div style="padding:6px 10px; font-size:10px; font-weight:700; color:var(--text-fa
 <div id="slashList"></div>
 </div>
 </div>

 <!-- Right Pane (Outline & Backlinks matching v0.11.5 screenshot) -->
 <div class="graph-pane" id="graphPane">
 <div class="graph-header">
 <div class="graph-mode-selector">
 <button class="graph-mode-btn active" id="btnModeLinks"></button>
 <button class="graph-mode-btn" id="btnModeTags"></button>
 <button class="graph-mode-btn" id="btnModeHash"></button>
 </div>
 <i class="fa-solid fa-xmark" style="cursor:pointer;" onclick="toggleGraphPane()"><
 </div>

 <div class="graph-canvas-container" id="graphCanvasContainer">
 <div style="font-size: 11px; font-weight: 600; color: var(--text-faint); margin-bo
 <div class="outline-tree" id="outlineTreeContainer">
 <!-- Populated dynamically from Headings -->
 </div>
 </div>
 </div>
</div>
</main>
</div>

```

```

<!-- Authentic Obsidian Settings Modal -->
<div class="modal-overlay" id="settingsModalOverlay" onclick="closeSettingsModal(event)">
 <div class="obsidian-settings-card" onclick="event.stopPropagation()">
 <div class="settings-nav">
 <div class="settings-nav-section">
 <div class="settings-nav-header">Options</div>
 <div class="settings-nav-item">General</div>
 <div class="settings-nav-item">Editor</div>
 <div class="settings-nav-item">Files and links</div>
 <div class="settings-nav-item active">Appearance</div>
 </div>
 </div>

 <div class="settings-content">
 <div class="settings-section-title">Appearance</div>
 <div class="obs-setting-item">
 <div class="obs-setting-info">
 <div class="obs-setting-name">Base color scheme</div>
 <div class="obs-setting-desc">Choose Obsidian's default color scheme.</div>
 </div>
 <div class="obs-setting-control">
 <select class="obs-select" id="themeSelect" onchange="changeThemeFromSelect()">
 <option value="dark">Dark</option>
 <option value="light">Light</option>
 </select>
 </div>
 </div>
 <div class="obs-setting-item">
 <div class="obs-setting-info">
 <div class="obs-setting-name">Vault Storage Backup</div>
 <div class="obs-setting-desc">Export all notes into a ZIP folder archive.</div>
 </div>
 <div class="obs-setting-control">
 <button class="obs-btn" onclick="exportAllNotesZip()"><i class="fa-solid fa-file-z
</div>
 </div>
 <div class="obs-setting-item">
 <div class="obs-setting-info">
 <div class="obs-setting-name" style="color:#ff5f56;">Reset Vault</div>
 <div class="obs-setting-desc">Exports all notes as a .md ZIP folder before permane
 </div>
 <div class="obs-setting-control">
 <button class="obs-btn obs-btn-danger" onclick="resetVaultWithBackup()"><i class="
 </div>
 </div>
 </div>
 </div>
</div>

<!-- Search Modal -->
<div class="modal-overlay" id="searchModalOverlay" onclick="closeSearchModal(event)">
 <div class="search-modal" onclick="event.stopPropagation()">
 <div class="modal-search-header">
 <i class="fa-solid fa-magnifying-glass" style="color:var(--text-faint);"></i>
 <input type="text" id="modalSearchInput" class="modal-search-input" placeholder="Type note
 </div>
 <div class="modal-search-body">
 <div class="modal-results-list" id="modalResultsList"></div>
 <div class="modal-preview-pane">
 <div id="modalPreviewTitle" style="font-weight:700; font-size:12px; color:var(--text-n
 <div id="modalPreviewSnippet"></div>
 </div>
 </div>
 </div>
</div>

<script>
 function getSystemTodayDate() {
 const today = new Date();
 const yyyy = today.getFullYear();
 const mm = String(today.getMonth() + 1).padStart(2, '0');
 const dd = String(today.getDate()).padStart(2, '0');
 return `${yyyy}-${mm}-${dd}`;
 }

 let notes = JSON.parse(localStorage.getItem('obs_v011_notes')) || [
 { id: 1, title: 'Testing Folding', content: '# Heading 1\nThe triangle next to this heading al
 { id: 2, title: 'Testing Tags', content: '# Tags Demo\nOrganizing notes and reviewing structur

```

```

 { id: 3, title: '2021-02-17', content: '# Daily Note\nJournal entry and quick notes for the se
 { id: 4, title: 'Outline Test', content: '# Introduction\nTesting the outline sidebar tree fea
 { id: 5, title: 'Second test file', content: '# Second File\nContent for secondary test note.'
 { id: 6, title: 'Test file', content: '# Final Test\nWrapping up our Obsidian v0.11.5 clone la
];

let activeNoteId = notes.find(n => !n.isBinned) ? notes.find(n => !n.isBinned).id : null;
let openTabs = activeNoteId ? [activeNoteId] : [];
let selectedNoteIds = new Set(activeNoteId ? [activeNoteId] : []);
let lastSelectedId = activeNoteId;

let activeSidebarView = 'notes';
let activeTagFilter = null;
let lastShiftTime = 0;
let isGraphOpen = true;
let calCurrentDate = new Date();
let slashActive = false;
let slashStartPos = -1;
let filteredSlashNotes = [];

window.addEventListener('DOMContentLoaded', () => {
 const savedTheme = localStorage.getItem('obs_v011_theme');
 if (savedTheme === 'light') {
 document.body.classList.add('light-theme');
 document.getElementById('themeSelect').value = 'light';
 }

 renderNavigatorList();
 renderTags();
 if (activeNoteId) loadNote(activeNoteId);
 else renderEditorState();

 renderCalendar();

 document.addEventListener('keydown', (e) => {
 if (e.key === 'Shift') {
 const currentTime = new Date().getTime();
 if (currentTime - lastShiftTime < 300) openSearchModal();
 lastShiftTime = currentTime;
 }
 });
});

function saveStorage() { localStorage.setItem('obs_v011_notes', JSON.stringify(notes)); }

function renderTabs() {
 const container = document.getElementById('tabContainer');
 container.innerHTML = '';

 openTabs.forEach(id => {
 const note = notes.find(n => n.id === id);
 if (!note) return;

 const isActive = id === activeNoteId;
 const tab = document.createElement('div');
 tab.className = `tab ${isActive ? 'active' : ''}`;
 tab.onclick = () => loadNote(id);

 const displayTitle = note.title.trim() === '' ? 'Untitled' : note.title;

 tab.innerHTML = `
 ${escapeHtml(displayTitle)}
 <i class="fa-solid fa-xmark tab-close" onclick="event.stopPropagation(); closeTab(${id}
 `;
 container.appendChild(tab);
 });
}

function closeTab(id) {
 openTabs = openTabs.filter(t => t !== id);
 if (id === activeNoteId) {
 if (openTabs.length > 0) {
 loadNote(openTabs[openTabs.length - 1]);
 } else {
 activeNoteId = null;
 renderEditorState();
 }
 } else {
 renderTabs();
 }
}

```

```

 }
}

function renderEditorState() {
 const editorPane = document.getElementById('editorPane');
 const emptyState = document.getElementById('emptyState');

 renderTabs();
 renderNavigatorList();

 if (!activeNoteId) {
 editorPane.style.display = 'none';
 emptyState.style.display = 'flex';
 return;
 }

 editorPane.style.display = 'flex';
 emptyState.style.display = 'none';

 const note = notes.find(n => n.id === activeNoteId);
 if (!note) return;

 document.getElementById('noteTitle').value = note.title;
 document.getElementById('noteContent').value = note.content;
 document.getElementById('currentBreadcrumb').innerText = note.title || 'Untitled';

 updateOutlineTree(note.content);
}

function updateOutlineTree(content) {
 const container = document.getElementById('outlineTreeContainer');
 container.innerHTML = '';

 const lines = content.split('\n');
 let foundHeadings = false;

 lines.forEach(line => {
 const trimmed = line.trim();
 if (trimmed.startsWith('#')) {
 foundHeadings = true;
 const level = trimmed.match(/^#+/)[0].length;
 const text = trimmed.replace(/^#+\s*/, '');

 const item = document.createElement('div');
 item.className = `outline-item outline-h${Math.min(level, 3)}`;
 item.innerText = text;
 item.onclick = () => {
 const textarea = document.getElementById('noteContent');
 const pos = textarea.value.indexOf(trimmed);
 if (pos !== -1) {
 textarea.focus();
 textarea.setSelectionRange(pos, pos + trimmed.length);
 }
 };
 container.appendChild(item);
 }
 });

 if (!foundHeadings) {
 container.innerHTML = '<div style="color: var(--text-faint); font-style: italic;">No headi
 }
}

function loadNote(id) {
 const note = notes.find(n => n.id === id);
 if (!note) return;

 if (!openTabs.includes(id)) {
 openTabs.push(id);
 }

 activeNoteId = id;
 selectedNoteIds.clear();
 selectedNoteIds.add(id);
 lastSelectedId = id;

 renderEditorState();
}

```

```

function createNewNote() {
 const newNote = { id: Date.now(), title: "", content: "", tags: "Test folder", created: getSys
 notes.unshift(newNote);
 saveStorage();

 switchSidebarView('notes');
 renderTags();
 loadNote(newNote.id);
 setTimeout(() => { document.getElementById('noteTitle').focus(); }, 50);
}

function updateCurrentNote() {
 if (!activeNoteId) return;
 const note = notes.find(n => n.id === activeNoteId);
 if (!note) return;

 note.title = document.getElementById('noteTitle').value;
 note.content = document.getElementById('noteContent').value;
 document.getElementById('currentBreadcrumb').innerText = note.title || 'Untitled';

 saveStorage();
 renderTabs();
 renderNavigatorList();
 updateOutlineTree(note.content);
}

async function handleEditorPaste(e) {
 updateCurrentNote();
}

async function handleEditorInput(e) {
 updateCurrentNote();
 const textarea = e.target;
 const pos = textarea.selectionStart;
 const val = textarea.value;

 if (val.slice(pos - 1, pos) === '/') {
 slashActive = true;
 slashStartPos = pos - 1;
 document.getElementById('slashMenu').style.display = 'block';
 renderSlashList('');
 }
 if (slashActive) renderSlashList(val.substring(slashStartPos + 1, pos).toLowerCase());
}

function switchSidebarView(view) {
 activeSidebarView = view;
 document.getElementById('ribbonNavBtn').classList.toggle('active', view === 'notes');
 document.getElementById('ribbonCalBtn').classList.toggle('active', view === 'calendar');

 const list = document.getElementById('navigatorList');
 const cal = document.getElementById('calendarView');

 if (view === 'calendar') {
 list.style.display = 'none';
 cal.style.display = 'flex';
 document.getElementById('sidebarTitle').innerText = 'Calendar View';
 renderCalendar();
 } else {
 list.style.display = 'flex';
 cal.style.display = 'none';
 document.getElementById('sidebarTitle').innerText = "Adrian's Obsidian Vault";
 renderNavigatorList();
 }
}

function handleNoteClick(e, noteId) {
 const visibleNotes = getVisibleNotes();
 if (e.metaKey || e.ctrlKey) {
 if (selectedNoteIds.has(noteId)) selectedNoteIds.delete(noteId);
 else selectedNoteIds.add(noteId);
 lastSelectedId = noteId;
 } else {
 selectedNoteIds.clear();
 selectedNoteIds.add(noteId);
 lastSelectedId = noteId;
 loadNote(noteId);
 }
 updateSelectionUI();
}

```

```

}

function updateSelectionUI() {
 const bar = document.getElementById('selectionBar');
 const text = document.getElementById('selectedCountText');
 if (selectedNoteIds.size > 1) {
 bar.style.display = 'flex';
 text.innerText = `${selectedNoteIds.size} notes selected`;
 } else {
 bar.style.display = 'none';
 }
 renderNavigatorList();
}

function clearSelection() {
 selectedNoteIds.clear();
 if (activeNoteId) selectedNoteIds.add(activeNoteId);
 updateSelectionUI();
}

function handleDragStart(e, noteId) {
 if (!selectedNoteIds.has(noteId)) {
 selectedNoteIds.clear();
 selectedNoteIds.add(noteId);
 lastSelectedId = noteId;
 renderNavigatorList();
 }
 const idsArray = Array.from(selectedNoteIds);
 e.dataTransfer.setData('text/plain', JSON.stringify(idsArray));
 const badge = document.getElementById('dragBadge');
 badge.innerText = `${idsArray.length} ${idsArray.length === 1 ? 'Note' : 'Notes'}`;
 e.dataTransfer.setDragImage(badge, 0, 0);
}

function handleDragOverBin(e) { e.preventDefault(); }
function handleDragLeaveBin(e) {}

function handleDropToBin(e) {
 e.preventDefault();
 const rawData = e.dataTransfer.getData('text/plain');
 if (!rawData) return;
 try {
 const idsToBin = JSON.parse(rawData);
 notes.forEach(n => { if (idsToBin.includes(n.id)) n.isBinned = true; });
 saveStorage();
 openTabs = openTabs.filter(id => !idsToBin.includes(id));
 if (idsToBin.includes(activeNoteId)) {
 activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
 }
 renderEditorState();
 } catch (err) { console.error(err); }
}

function binSelectedNotes() {
 notes.forEach(n => { if (selectedNoteIds.has(n.id)) n.isBinned = true; });
 saveStorage();
 const binnedIds = Array.from(selectedNoteIds);
 openTabs = openTabs.filter(id => !binnedIds.includes(id));
 if (binnedIds.includes(activeNoteId)) {
 activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length - 1] : null;
 }
 selectedNoteIds.clear();
 if (activeNoteId) selectedNoteIds.add(activeNoteId);
 renderEditorState();
}

function getVisibleNotes() {
 const isBinView = activeSidebarView === 'bin';
 return notes.filter(n => {
 if (isBinView) return n.isBinned;
 if (n.isBinned) return false;
 return activeTagFilter ? n.tags.split(',').map(t=>t.trim()).includes(activeTagFilter) : tr
 });
}

function moveNoteIdToBin(id) {
 const note = notes.find(n => n.id === id);
 if (note) {
 note.isBinned = true;
 }
}

```

```

 saveStorage();
 openTabs = openTabs.filter(t => t !== id);
 if (activeNoteId === id) activeNoteId = openTabs.length > 0 ? openTabs[openTabs.length-1]
 renderEditorState();
 }
}

function renderCalendar() {
 const grid = document.getElementById('calendarGrid');
 grid.innerHTML = '';
 const year = calCurrentDate.getFullYear();
 const month = calCurrentDate.getMonth();
 const monthNames = ["January", "February", "March", "April", "May", "June", "July", "August",
 document.getElementById('calMonthYear').innerText = `${monthNames[month]} ${year}`;

 ['Su', 'Mo', 'Tu', 'We', 'Th', 'Fr', 'Sa'].forEach(d => {
 const l = document.createElement('div');
 l.className = 'cal-day-label';
 l.innerText = d;
 grid.appendChild(l);
 });

 const firstDay = new Date(year, month, 1).getDay();
 const daysInMonth = new Date(year, month + 1, 0).getDate();
 for (let i = 0; i < firstDay; i++) grid.appendChild(document.createElement('div'));

 const todayStr = getSystemTodayDate();
 for (let day = 1; day <= daysInMonth; day++) {
 const cell = document.createElement('div');
 cell.className = 'cal-cell';
 const mm = String(month + 1).padStart(2, '0');
 const dd = String(day).padStart(2, '0');
 const dateStr = `${year}-${mm}-${dd}`;

 cell.innerText = day;
 if (dateStr === todayStr) cell.classList.add('today');
 if (notes.some(n => !n.isBinned && n.created === dateStr)) cell.classList.add('has-note');

 cell.onclick = () => {
 const matchedNote = notes.find(n => !n.isBinned && n.created === dateStr);
 if (matchedNote) {
 switchSidebarView('notes');
 loadNote(matchedNote.id);
 }
 };
 grid.appendChild(cell);
 }
}

function changeMonth(dir) { calCurrentDate.setMonth(calCurrentDate.getMonth() + dir); renderCalend
function triggerICSImport() { document.getElementById('icsImporter').click(); }

function handleImportICS(e) {
 const file = e.target.files[0];
 if (!file) return;
 const reader = new FileReader();
 reader.onload = function(evt) {
 const text = evt.target.result;
 const summaryMatch = text.match(/SUMMARY:(.*)/);
 const title = summaryMatch ? summaryMatch[1].trim() : "Imported Event";
 const newNote = { id: Date.now(), title: title, content: "Imported from Apple Calendar\n\n
 notes.unshift(newNote);
 saveStorage();
 switchSidebarView('notes');
 loadNote(newNote.id);
 };
 reader.readAsText(file);
}

function exportCalendarICS() {
 let icsContent = "BEGIN:VCALENDAR\nVERSION:2.0\nPRODID:-//Obsidian Studio//EN\n";
 notes.filter(n => !n.isBinned).forEach(n => {
 const dateClean = (n.created || getSystemTodayDate()).replace(/-/g, '');
 icsContent += `BEGIN:VEVENT\nSUMMARY:${n.title}\nDESCRIPTION:${n.content.replace(/\n/g, '
 });
 icsContent += "END:VCALENDAR";
 saveAs(new Blob([icsContent], { type: 'text/calendar;charset=utf-8' }), "Obsidian_Calendar_Syn
}

```

```

function resetVaultWithBackup() {
 if (!confirm("ARE YOU SURE?\n\nResetting will download all your files as a ZIP archive and wip
exportAllNotesZip();
setTimeout(() => {
 notes = [];
 activeNoteId = null;
 openTabs = [];
 selectedNoteIds.clear();
 saveStorage();
 renderEditorState();
 closeSettingsModal(null, true);
}, 300);
}

function renderNavigatorList() {
 const list = document.getElementById('navigatorList');
 list.innerHTML = '';

 // Replicating exact tree view shown in screenshot: Test folder -> Folder 2 -> Files
 const treeHtml = `
 <div class="tree-folder"><i class="fa-solid fa-chevron-down" style="font-size:10px;"></i>
 <div class="tree-sub" id="subTestFolder"></div>
 <div class="tree-folder" style="margin-top:4px;"><i class="fa-solid fa-chevron-right" styl
 <div class="tree-sub" id="subFolder2" style="display:none;"></div>
 `;
 list.innerHTML = treeHtml;

 const subTest = document.getElementById('subTestFolder');
 const subFolder2 = document.getElementById('subFolder2');

 notes.filter(n => !n.isBinned).forEach(note => {
 const card = document.createElement('div');
 const isSelected = selectedNoteIds.has(note.id);
 const isActive = note.id === activeNoteId;
 card.className = `note-card ${isActive ? 'active' : ''} ${isSelected ? 'selected' : ''}`;
 card.setAttribute('draggable', 'true');
 card.ondragstart = (e) => handleDragStart(e, note.id);
 card.onclick = (e) => handleNoteClick(e, note.id);

 if (note.title.startsWith('2021')) {
 card.innerHTML = `<div class="note-card-title">${escapeHtml(note.title)}</div>`;
 } else if (note.title.includes('Screen')) {
 card.innerHTML = `<div class="note-card-title" style="font-size:11px;"><span style="ba
 } else {
 card.innerHTML = `<div class="note-card-title">${escapeHtml(note.title || 'Untitled')}`
 }

 card.innerHTML += `<i class="fa-solid fa-trash-can btn-card-bin" title="Move to Bin" oncli

 if (note.tags === 'Folder 2') {
 subFolder2.appendChild(card);
 } else {
 subTest.appendChild(card);
 }
 });
}

function renderTags() {
 const list = document.getElementById('tagPillList');
 list.innerHTML = '';
 const tagMap = {};
 notes.filter(n=>!n.isBinned).forEach(n => {
 if (n.tags) {
 n.tags.split(',').forEach(t => {
 const clean = t.trim();
 if (clean) tagMap[clean] = (tagMap[clean] || 0) + 1;
 });
 }
 });
 Object.keys(tagMap).forEach(tag => {
 const pill = document.createElement('div');
 pill.className = `tag-pill ${activeTagFilter === tag ? 'active' : ''}`;
 pill.innerText = `#${tag} (${tagMap[tag]})`;
 pill.onclick = () => {
 activeTagFilter = activeTagFilter === tag ? null : tag;
 renderTags();
 renderNavigatorList();
 };
 list.appendChild(pill);
 });
}

```

```
});
}

function changeThemeFromSelect() {
 const val = document.getElementById('themeSelect').value;
 if (val === 'light') document.body.classList.add('light-theme');
 else document.body.classList.remove('light-theme');
 localStorage.setItem('obs_v011_theme', val);
}

function toggleGraphPane() {
 isGraphOpen = !isGraphOpen;
 const pane = document.getElementById('graphPane');
 const ribbonBtn = document.getElementById('ribbonGraphBtn');

 if (isGraphOpen) {
 pane.classList.remove('collapsed');
 ribbonBtn.classList.add('active');
 } else {
 pane.classList.add('collapsed');
 ribbonBtn.classList.remove('active');
 }
}

function openSettingsModal() { document.getElementById('settingsModalOverlay').classList.add('active');
function closeSettingsModal(e, force = false) {
 if (force || e.target.id === 'settingsModalOverlay') document.getElementById('settingsModalOverlay').classList.remove('active');
}

function exportAllNotesZip() {
 if (notes.length === 0) return alert("No notes to export.");
 const zip = new JSZip();
 const folder = zip.folder("Adrian_Vault");
 notes.forEach(note => {
 const filename = (note.title.trim() || 'Untitled_' + note.id).replace(/[/\\?%*:|"<>]/g, '_') + '.md';
 folder.file(filename, `---\ncreated: ${note.created || getSystemTodayDate()} \ntags: ${note.tags || ''}`);
 });
 zip.generateAsync({ type: "blob" }).then(content => saveAs(content, "Adrian_Vault_Export.zip"));
}

function triggerFolderImport() { document.getElementById('folderImporter').click(); }

async function handleImportFolder(event) {
 const files = event.target.files;
 if (!files || files.length === 0) return;
 for (let i = 0; i < files.length; i++) {
 const file = files[i];
 if (file.name.endsWith('.txt') || file.name.endsWith('.md')) {
 const content = await file.text();
 notes.unshift({ id: Date.now() + i, title: file.name.replace(/\.([^.]+\.$)/, ''), content });
 }
 }
 saveStorage();
 renderNavigatorList();
 renderTags();
 if (notes.length > 0) loadNote(notes[0].id);
}

function handleEditorKeyDown(e) {
 if (!slashActive) return;
 if (e.key === 'Enter' && filteredSlashNotes.length > 0) {
 e.preventDefault();
 insertPageLink(filteredSlashNotes[0].title);
 }
}

function renderSlashList(query) {
 const list = document.getElementById('slashList');
 list.innerHTML = '';
 filteredSlashNotes = notes.filter(n => !n.isBinned && n.id !== activeNoteId && n.title.toLowerCase().includes(query));
 filteredSlashNotes.forEach((note) => {
 const item = document.createElement('div');
 item.className = 'slash-item';
 item.innerText = note.title;
 item.onmousedown = (e) => { e.preventDefault(); insertPageLink(note.title); };
 list.appendChild(item);
 });
}
```

```

function insertPageLink(title) {
 const textarea = document.getElementById('noteContent');
 const pos = textarea.selectionStart;
 const val = textarea.value;
 textarea.value = val.substring(0, slashStartPos) + `[${title}]` + val.substring(pos);
 document.getElementById('slashMenu').style.display = 'none';
 slashActive = false;
 updateCurrentNote();
}

function openSearchModal() {
 document.getElementById('searchModalOverlay').classList.add('active');
 document.getElementById('modalSearchInput').value = '';
 document.getElementById('modalSearchInput').focus();
 handleModalSearch();
}

function closeSearchModal(e) {
 if (e.target.id === 'searchModalOverlay') document.getElementById('searchModalOverlay').classList
}

function handleModalSearch() {
 const query = document.getElementById('modalSearchInput').value.toLowerCase();
 const results = notes.filter(n => !n.isBinned && (n.title.toLowerCase().includes(query) || n.c
 const list = document.getElementById('modalResultsList');
 list.innerHTML = '';
 results.forEach((note) => {
 const item = document.createElement('div');
 item.className = 'modal-result-item';
 item.innerHTML = `<i class="fa-regular fa-file-lines"></i> ${escapeHtml(note.title)}
 item.onclick = () => {
 loadNote(note.id);
 document.getElementById('searchModalOverlay').classList.remove('active');
 };
 list.appendChild(item);
 });
 if (results[0]) {
 document.getElementById('modalPreviewTitle').innerText = results[0].title;
 document.getElementById('modalPreviewSnippet').innerText = results[0].content;
 }
}

function escapeHtml(str) { return str.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">
</script>
</body>
</html>

```

---